

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO PIAUÍ
CAMPUS PIRIPIRI
TECNÓLOGO EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

FRANCISCO IGOR SILVA SANTOS

CERTIFYHUB: sistema para emissão, gestão e autenticação de certificados

PIRIPIRI – PI
2026

Francisco Igor Silva Santos

CERTIFYHUB: sistema para emissão, gestão e autenticação de certificados

Trabalho de Conclusão de Curso apresentado como exigência parcial para obtenção do diploma do Curso de Tecnólogo em Análise e Desenvolvimento de Sistemas do Instituto Federal de Educação, Ciência e Tecnologia do Piauí, Campus Piripiri.

Orientador: Prof. Me. Mayllon Veras da Silva.

PIRIPIRI – PI
2026

Dados Internacionais de Catalogação na Publicação (CIP) de acordo com ISBD

Santos, Francisco Igor Silva

S237c CertifyHub : Sistema para Emissão, Gestão e Autenticação de Certificados / Francisco Igor Silva Santos. - 2026.

81 f.

Trabalho de Conclusão de Curso (Tecnologia em Análise e Desenvolvimento de Sistemas) - Instituto Federal de Educação, Ciência e Tecnologia do Piauí, Campus Piripiri, 2026.

Orientador : Prof Me. Mayllon Veras da Silva.

1. SaaS. 2. Arquitetura limpa. 3. Autenticação via QR Code. 4. React. I. Título.

CDD - 004.21

Elaborado por Júlio César Oliveira Rodrigues CRB CRB 3/1125

CERTIFYHUB: sistema para emissão, gestão e autenticação de certificados

Francisco Igor Silva Santos
Prof. Me. Mayllon Veras da Silva

RESUMO

Empresas de treinamento e instituições educacionais ainda dependem de processos manuais e soluções improvisadas para emissão de certificados, o que gera retrabalho, inconsistências e ausência de mecanismos confiáveis de validação. Este trabalho apresenta o desenvolvimento de um sistema web no modelo SaaS (*Software as a Service*) para emissão, gestão e autenticação pública de certificados. A solução adota uma arquitetura desacoplada estruturada em duas aplicações independentes: um *frontend* do tipo *Single Page Application* (SPA) desenvolvido com React e TypeScript, e um *backend* (Next.js) orientado aos princípios da Arquitetura Limpa, organizado nas camadas de apresentação, aplicação, domínio e infraestrutura. A persistência de dados ocorre em um banco relacional (PostgreSQL), enquanto a geração de PDFs é processada no servidor. As funcionalidades implementadas incluem *templates* configuráveis com editor visual, emissão individual com código único e *QR Code*, *dashboard* gerencial e portal público de validação. O ciclo de vida da aplicação conta com *pipeline* de CI/CD para integração e implantação contínuas em nuvem. A qualidade do software foi atestada por uma robusta suíte de testes automatizados (290 testes), alcançando cobertura de código de 82,6% no *frontend* e 96,53% no *backend* (medidos por *Statements*). Conclui-se que o produto, na forma de um Produto Mínimo Viável (MVP) funcional e testado, atingiu os objetivos propostos, superando as restrições do fluxo manual por meio de uma plataforma escalável e segura.

Palavras-chave: emissão de certificados; SaaS; arquitetura limpa; React; autenticação via *QR Code*.

ABSTRACT

Training companies and educational institutions still rely on manual processes and improvised solutions for certificate issuance, leading to rework, inconsistencies, and a lack of reliable validation mechanisms. This work presents the development of a SaaS (*Software as a Service*) web system for issuing, managing, and publicly authenticating certificates. The solution adopts a decoupled architecture structured in two independent applications: a *Single Page Application* (SPA) frontend developed with React and TypeScript, and a backend (Next.js) oriented towards Clean Architecture principles, organized into presentation, application, domain, and infrastructure layers. Data persistence is handled by a relational database (PostgreSQL), while PDF generation is processed on the server-side. Implemented features include configurable templates with a visual editor, individual issuance with unique codes and *QR Codes*, a management dashboard, and a public validation portal. The application lifecycle is supported by a CI/CD pipeline for *continuous integration* and cloud deployment. Software quality was ensured by a robust automated testing suite (290 tests), achieving 82.6% code coverage on the frontend and 96.53% on the backend (measured by *Statements*). It is concluded that the product, as a functional and tested *Minimum Viable Product* (MVP), achieved the proposed objectives, overcoming the constraints of the manual workflow through a scalable and secure platform.

Keywords: certificate issuance; SaaS; clean architecture; React; *QR Code* authentication.

Data de aprovação: 15/07/2026

---

1 Acadêmico do curso de Tecnólogo em Análise e Desenvolvimento de Sistemas pelo Instituto Federal do Piauí – IFPI, Campus Piripiri.

2 Mestre em Engenharia de Software pelo Centro de Estudos e Sistemas Avançados do Recife – CESAR School. Professor de Ensino Superior e Professor do Ensino Básico, Técnico e Tecnológico do Instituto Federal do Piauí – IFPI, Campus Piripiri.

LISTA DE FIGURAS

1 Diagrama de casos de uso do sistema de certificados 38
2 Diagrama de Classes — MVP do Sistema de Certificados 53
3 Arquitetura do Backend — Quatro camadas (App, Application, Domain, Infrastructure) com DTOs, Value Objects e fluxo de dependências . . . 55
4 Arquitetura do Frontend — Quatro camadas (View, ViewModel, Service, Model) com React 56
5 Diagrama Entidade-Relacionamento (DER) — MVP do Banco de Dados 57
6 Tela inicial — *Landing page* 58
7 Dashboard — Visão geral do sistema 59
8 Editor visual de template com pré-visualização em tempo real 60
9 Lista de templates cadastrados 60
10 Formulário de emissão de certificado 61
11 Lista de certificados emitidos 62
12 Histórico de envios de e-mail 62
13 Certificado gerado com *QR Code* de verificação 63
14 Página pública de verificação de certificados 64
15 Pipeline de CI do backend no GitHub Actions 67
16 Pipeline de CI do frontend no GitHub Actions 67
17 Diagrama de Implantação — Fluxo de deploy no Render a partir do GitHub, com frontend como Static Site, backend como Web Service e banco PostgreSQL gerenciado pelo Supabase 68
18 Relatório de cobertura dos testes — frontend 77
19 Relatório de cobertura dos testes — backend 78
20 Relatório do teste *end-to-end* gerado pelo Playwright 79

LISTA DE QUADROS

1 Tecnologias utilizadas no desenvolvimento do sistema 25
2 Critérios de aceitação dos requisitos funcionais 34
3 Caso de uso UC001 – Gerenciar Usuários 39
4 Caso de uso UC002 – Gerenciar Templates 40
5 Caso de uso UC003 – Gerenciar Cursos 41
6 Caso de uso UC004 – Emitir Certificado Individual 42
7 Caso de uso UC005 – Emitir Certificados em Lote 43
8 Caso de uso UC006 – Cadastrar Participante 44
9 Caso de uso UC007 – Importar Participantes 45
10 Caso de uso UC008 – Validar Certificado 46
11 Caso de uso UC009 – Buscar Participantes 46
12 Caso de uso UC010 – Visualizar Histórico de Certificados 47
13 Caso de uso UC011 – Gerenciar Organizações 48
14 Caso de uso UC012 – Configurar Sistema 49
15 Caso de uso UC013 – Abrir Chamado de Suporte 50
16 Caso de uso UC014 – Visualizar Dashboard 50
17 Caso de uso UC015 – Consultar Logs 51
18 Matriz de rastreabilidade entre requisitos funcionais, casos de uso, classes e tabelas implementados no MVP 65
19 Plano de testes do sistema 73

LISTA DE TABELAS

1 Resumo geral dos testes 80

LISTA DE ABREVIATURAS E SIGLAS

API	*Application Programming Interface*
BaaS	*Backend as a Service*
CI/CD	*Continuous Integration / Continuous Deployment*
CPF	Cadastro de Pessoa Física
CRUD	*Create, Read, Update, Delete*
CORS	*Cross-Origin Resource Sharing*
CSS	*Cascading Style Sheets*
CT	Caso de Teste
CSV	*Comma-Separated Values*
DDD	*Domain-Driven Design*
DER	Diagrama Entidade-Relacionamento
DTO	*Data Transfer Object*
E2E	*End-to-End*
HMR	*Hot Module Replacement*
HTML	*HyperText Markup Language*
HTTP	*Hypertext Transfer Protocol*
HTTPS	*Hypertext Transfer Protocol Secure*
IFPI	Instituto Federal do Piauí
JSON	*JavaScript Object Notation*
JWT	*JSON Web Token*
LGPD	Lei Geral de Proteção de Dados
MFA	*Multi-Factor Authentication*
MVP	*Minimum Viable Product*
NIT	Núcleo de Inovação Tecnológica
ORM	*Object-Relational Mapping*
OWASP	*Open Web Application Security Project*
PDF	*Portable Document Format*
PNG	*Portable Network Graphics*
QR Code	*Quick Response Code*
RBAC	*Role-Based Access Control*
REST	*Representational State Transfer*
RLS	*Row Level Security*
SaaS	*Software as a Service*
SDK	*Software Development Kit*
SEO	*Search Engine Optimization*
SMTP	*Simple Mail Transfer Protocol*
SPA	*Single Page Application*
SQL	*Structured Query Language*
SSR	*Server-Side Rendering*
SVG	*Scalable Vector Graphics*
TLS	*Transport Layer Security*
UC	Caso de Uso
URL	*Uniform Resource Locator*
UUID	*Universally Unique Identifier*
WCAG	*Web Content Accessibility Guidelines*

SUMÁRIO

1 INTRODUÇÃO 12
1.1 OBJETIVOS 13
1.1.1 Geral 13
1.1.2 Específicos 13

2 TECNOLOGIAS ENVOLVIDAS 14
2.1 FRONTEND 14
2.1.1 React 19 15
2.1.2 TypeScript 15
2.1.3 Tailwind CSS 15
2.1.4 shadcn/ui 16
2.1.5 TanStack Router 16
2.1.6 React Hook Form 17
2.1.7 Sonner 17
2.1.8 Recharts 17
2.1.9 date-fns 18
2.1.10 Vite 18
2.1.11 Arquitetura e Padrões 18
2.1.12 Zod 19
2.1.13 Next.js 16 19
2.1.14 Zod 20
2.1.15 Arquitetura e Padrões 20
2.2 BANCO DE DADOS 21
2.2.1 Supabase e PostgreSQL 21
2.3 GERAÇÃO DE DOCUMENTOS 21
2.3.1 PDFKit 21
2.3.2 qrcode 22
2.3.3 Render 23
2.3.4 Jest 23
2.3.5 ESLint e Prettier 24
2.3.6 Git e GitHub 24

2.3.7 Visual Studio Code 24

3 MODELAGEM DO PROJETO 25
3.1 LEVANTAMENTO DE REQUISITOS 25
3.1.1 Módulo de Autenticação e Controle de Acesso 26
3.1.2 Módulo de Templates de Certificados 26
3.1.3 Módulo de Entrada de Dados 26
3.1.4 Módulo de Geração de Certificados 27
3.1.5 Módulo de Envio e Comunicação 27
3.1.6 Módulo de Validação Pública e Antifraude 27
3.1.7 Módulo de Assinaturas Digitais 27
3.1.8 Módulo de Dashboard e Gestão 27
3.1.9 Módulo de Auditoria e Segurança Operacional 28
3.1.10 Módulo de Gestão de Cursos/Eventos 28
3.1.11 Módulo de Participantes 28
3.1.12 Módulo Multiempresa / Multi-tenant 28
3.1.13 Módulo de Configurações do Sistema 28
3.1.14 Módulo de Armazenamento e Arquivos 29
3.1.15 Módulo de Notificações e Alertas 29
3.1.16 Módulo de Monitoramento e Logs 29
3.1.17 Módulo de Suporte, Operação e Manutenção 29
3.1.18 Segurança 29
3.1.19 Desempenho e Escalabilidade 30
3.1.20 Disponibilidade e Confiabilidade 30
3.1.21 Usabilidade e Acessibilidade 31
3.1.22 Compatibilidade e Integração 31
3.1.23 Armazenamento e Retenção 31
3.1.24 Conformidade Legal e LGPD 31
3.1.25 Arquitetura e Tecnologia 32
3.1.26 Manutenção e Monitoramento 32
3.1.27 Internacionalização 33
3.2 REGRAS DE NEGÓCIO 33
3.2.1 Status de Implementação das Regras de Negócio 33
3.3 CRITÉRIOS DE ACEITAÇÃO 34
3.4 DIAGRAMAS DE CASOS DE USO 37

3.5 DIAGRAMAS DE CLASSE 51
3.6 ARQUITETURA DO SISTEMA 53
3.7 DIAGRAMA ENTIDADE-RELACIONAMENTO 56
3.8 INTERFACES 58
3.8.1 Tela Inicial 58
3.8.2 Dashboard 58
3.8.3 Customização de Templates 59
3.8.4 Lista de Templates 60
3.8.5 Emissão de Certificados 61
3.8.6 Lista de Certificados 61
3.8.7 Envios 62
3.8.8 Certificado Gerado 63
3.8.9 Verificação Pública 63
3.9 MATRIZ DE RASTREABILIDADE 64

4 SOFTWARE 66
4.1 IMPLANTAÇÃO 66
4.1.1 Arquitetura e Execução 66
4.1.2 CI/CD e Pipeline de Implantação 66
4.1.3 Configuração de Ambiente 67
4.1.4 Banco de Dados 68
4.1.5 Monitoramento e Observabilidade 68
4.2 EXEMPLOS DE CÓDIGO 68
4.2.1 Entidade de Domínio: Certificate 68
4.2.2 Route Handler de Emissão 69
4.2.3 Geração de PDF com *QR Code* 70
4.2.4 Validação por Código Único 71
4.3 TESTES 72
4.3.1 Plano de Testes 72
4.3.2 Frontend (CertifyHub) 77
4.3.3 Backend (CertifyHub Server) 77
4.3.4 Testes End-to-End 78
4.3.5 Limitações 79
4.3.6 Resumo Geral 80

5 CONSIDERAÇÕES FINAIS 80
5.1 PRINCIPAIS CONTRIBUIÇÕES 81
5.2 LIMITAÇÕES 81
5.3 TRABALHOS FUTUROS 82

REFERÊNCIAS 86

ANEXOS 87

1 INTRODUÇÃO

A emissão de certificados é uma atividade essencial para empresas de treinamentos, consultorias e instituições educacionais que promovem cursos, capacitações e eventos [1, 2]. Apesar da relevância desse processo, grande parte das organizações ainda depende de procedimentos manuais ou soluções improvisadas, utilizando ferramentas como Google Planilhas, Google Docs, PowerPoint e complementos externos.

Essas práticas resultam em retrabalho, inconsistências, falta de padronização e dificuldade de controle operacional. Além disso, ferramentas como AutoCrat [3] e documentos espalhados entre diferentes plataformas não foram projetados para uso profissional em escala, o que gera limitação, fragilidade e risco de inconsistências.

A ausência de um sistema integrado compromete a rastreabilidade e a credibilidade dos certificados, uma vez que não há mecanismos oficiais de validação pública, histórico centralizado, indicadores gerenciais ou emissão em lote eficiente. Estudos têm mostrado que sistemas baseados em *QR Code* têm se destacado como alternativa eficaz para verificação de credenciais acadêmicas [1], combinando criptografia e códigos bidimensionais para garantir a autenticidade dos documentos [4]. Esse cenário se agrava com o crescimento de cursos online e da necessidade de certificação confiável.

Diante disso, justifica-se o desenvolvimento de uma solução centralizada e automatizada, capaz de reduzir erros humanos, otimizar o tempo operacional e garantir segurança, conformidade e profissionalismo. Recursos como códigos únicos, validação via *QR Code* e assinaturas digitais ampliam a confiabilidade dos certificados e fortalecem a imagem institucional das organizações emissoras [2].

Diante desse contexto, realizou-se uma análise de soluções existentes no mercado. No espectro open-source, o TrueCert [5] oferece stack similar (React + Node.js), mas utiliza MongoDB e Express.js sem seguir Arquitetura Limpa, CI/CD ou testes automatizados. Já soluções brasileiras como AutoCert [6] operam sobre Google Sheets, sem aplicação web própria. Nenhuma dessas alternativas combina simultaneamente código aberto, banco relacional, pipeline de CI/CD com cobertura superior a 82% e arquitetura baseada nos princípios da Arquitetura Limpa, como proposto neste trabalho. Ressalta-se que uma ampla busca bibliográfica foi realizada em bases de dados nacionais (CAPES, SciELO, BDTD e Google Acadêmico com filtros para língua portuguesa) com o intuito de identificar trabalhos correlatos de cunho nacional abordando sistemas de emissão e validação de certificados com arquitetura web moderna; entretanto, não foram encontrados estudos que contemplassem simultaneamente os mesmos pilares técnicos propostos neste trabalho – o que reforça a originalidade e a pertinência da presente contribuição.

O presente trabalho consiste no desenvolvimento de um Sistema Web – produto do tipo SaaS (*Software as a Service*) com arquitetura multi-tenant – destinado à emissão, gestão e autenticação de certificados. O escopo do projeto foi definido a partir de um estudo de caso realizado em uma organização do ramo de treinamentos corporativos, por meio de entrevistas semiestruturadas e observação direta dos processos de emissão vigentes. As dificuldades identificadas – como o tempo elevado para emissão manual, a dificuldade em gerenciar grandes volumes de certificados e a ausência de mecanismos de validação pública – serviram como base para a definição

dos requisitos funcionais e não funcionais do sistema. Dessa forma, o escopo abrange desde a automatização da criação e personalização de certificados até a disponibilização de um portal público de validação, visando substituir processos manuais por uma plataforma profissional, segura e escalável. Os direitos de propriedade intelectual do software desenvolvido são de propriedade do Instituto Federal do Piauí (IFPI).

O desenvolvimento seguiu a abordagem de Produto Mínimo Viável (MVP), priorizando as funcionalidades essenciais para a operação do sistema – emissão de certificados, templates configuráveis, dashboard e validação pública – enquanto funcionalidades complementares, como importação em lote e envio automático por e-mail, foram postergadas para versões futuras. O modelo SaaS adotado permite que o sistema seja oferecido como serviço centralizado, simplificando a manutenção, a escalabilidade e a atualização contínua da plataforma.

Este relatório técnico está organizado da seguinte forma: a Seção 2 apresenta as tecnologias utilizadas no desenvolvimento; a Seção 3 descreve a modelagem do sistema, incluindo levantamento de requisitos, diagramas de casos de uso, classes, arquitetura e entidade-relacionamento; a Seção 4 detalha a implementação do software; e a Seção 5 traz as considerações finais.

1.1 OBJETIVOS

Os objetivos deste trabalho estão organizados em um objetivo geral e em objetivos específicos. O objetivo geral sintetiza a finalidade central do sistema proposto, enquanto os objetivos específicos detalham, de forma objetiva e mensurável, as metas técnicas e funcionais cuja realização é necessária para a concretização do projeto. Em conjunto, eles orientam a condução das etapas de levantamento de requisitos, modelagem, implementação e validação descritas ao longo deste relatório técnico.

1.1.1 Geral

Projetar e implementar um sistema web automatizado para emissão, gestão e validação de certificados, com base projetada para suporte a envio automático por e-mail em versões futuras, destinado a empresas de treinamento corporativo, instituições educacionais e demais organizações que necessitem certificar participantes em cursos, capacitações e eventos, a fim de eliminar a dependência de processos manuais e soluções improvisadas, garantindo padronização dos documentos, rastreabilidade do histórico de emissões, autenticação confiável via código único e *QR Code*, centralização dos dados em uma única plataforma e estabelecendo as bases para emissão em lote eficiente em versões futuras.

1.1.2 Específicos

- implementar um módulo de templates configuráveis para a criação de certificados;
- desenvolver um mecanismo de autenticação pública via código único e *QR Code* para garantir a integridade dos certificados e prevenir fraudes;
- desenvolver um módulo de dashboard com indicadores gerenciais para apoiar o controle operacional e a tomada de decisão das organizações sobre suas emissões, dispondo de filtros por período, curso e status;

- planejar e executar testes unitários e de integração para validação dos módulos implementados;
- configurar o ambiente de implantação do sistema em infraestrutura em nuvem com deploys automáticos a partir do repositório Git;
- projetar as bases conceituais e estruturais para conformidade com a LGPD, incluindo a modelagem do registro de consentimento, diretrizes para anonimização de dados pessoais, controle de acesso a logs sensíveis e trilha de auditoria para operações críticas, a serem refinadas em versões futuras;
- projetar a arquitetura multi-tenant do sistema, estabelecendo as bases conceituais para isolamento de dados entre organizações, segurança por tenant e escalabilidade horizontal em versões futuras.

2 TECNOLOGIAS ENVOLVIDAS

Este capítulo apresenta o conjunto de tecnologias empregadas no desenvolvimento do sistema, organizadas por camada de atuação. A discussão que se segue detalha as opções técnicas adotadas, seus motivos e as alternativas consideradas ao longo do processo de decisão.

2.1 FRONTEND

A camada de apresentação adota a arquitetura de *Single Page Application* (SPA), conforme estabelecido no requisito não funcional RNF041. Nesse modelo, o navegador carrega um único documento HTML e as atualizações de conteúdo ocorrem dinamicamente via JavaScript, sem recarregamento completo da página [7, 8]. A comunicação com o backend é realizada exclusivamente via API REST, o que desacopla as duas aplicações e permite que cada uma evolua com seu próprio ciclo de vida.

A escolha pela arquitetura SPA foi motivada pela natureza do sistema: trata-se de um painel administrativo (*dashboard*) com operações CRUD, formulários de emissão, gráficos gerenciais e filtros em tempo real — cenários nos quais a SPA oferece vantagens significativas em interatividade e fluidez de navegação [9]. Estudos comparativos demonstram que, embora a SPA apresente tempo de carregamento inicial superior ao de abordagens com renderização no servidor (SSR), ela oferece interatividade mais rápida após o carregamento e melhor experiência do usuário em interfaces dinâmicas [10]. Para o contexto deste sistema, as limitações de SEO não são impeditivas: o portal público de validação é acessado via *QR Code* impresso no certificado — não por busca orgânica — e o público interno da plataforma (administradores) se beneficia da navegação sem recarregamento entre as telas de gestão.

Outro fator determinante foi a sobrecarga que o SSR imporia ao backend. Como todo o processamento pesado — geração de PDF, validação de dados, comunicação com o banco — já está concentrado no servidor de API, adicionar renderização de páginas a esse mesmo processo tornaria a aplicação mais lenta e complexa. Com a separação SPA + API, o frontend é compilado como um conjunto de arquivos estáticos (HTML, CSS, JS) e implantado como *Static Site*, sem necessidade de um runtime

Node.js ativo ou de consumo adicional de recursos do servidor [11]. O backend permanece enxuto, dedicado exclusivamente às suas responsabilidades de API e processamento.

2.1.1 React 19

React é uma biblioteca JavaScript para construção de interfaces de usuário baseada no conceito de componentes reutilizáveis e estado unidirecional [12]. A versão 19 introduz melhorias significativas no modelo de concorrência, como o novo hook `use()`, que permite leitura assíncrona de recursos diretamente no corpo do componente, e as *Actions* para gerenciamento nativo de formulários, reduzindo a necessidade de bibliotecas externas para controle de estado de formulários.

A escolha do React 19 baseou-se em dois fatores principais: (1) maturidade do ecossistema, com ampla disponibilidade de bibliotecas e documentação; e (2) modelo de componentes que favorece a modularidade e a testabilidade do código. Alternativas como Vue.js e Angular foram descartadas respectivamente pelo ecossistema menos consolidado para o tipo de aplicação e pela maior complexidade intrínseca do framework.

2.1.2 TypeScript

TypeScript é um superconjunto tipado de JavaScript que compila para JavaScript puro. Seu sistema de tipos estrutural permite detectar erros comuns em tempo de compilação, como acesso a propriedades inexistentes, passagem de argumentos com tipos incompatíveis e retorno incorreto de funções [13].

A adoção do TypeScript em vez de JavaScript puro deveu-se à necessidade de contratos explícitos entre módulos e à segurança proporcionada pela verificação estática de tipos: erros como acesso a propriedades inexistentes ou argumentos com tipos incorretos são detectados em tempo de compilação, e não em runtime. Alternativas como Flow (Facebook) foram descartadas por seu ecossistema significativamente menor — o Flow não é suportado nativamente por ferramentas como ESLint, Prettier e o próprio Vite — e pela comunidade majoritariamente concentrada no TypeScript, que resulta em melhor documentação, mais tipos publicados (`@types/*`) e suporte de primeira classe em IDEs.

No projeto, o TypeScript foi utilizado em toda a camada frontend e também no backend. O recurso de *generics* possibilitou a criação de hooks e utilitários reutilizáveis com segurança de tipos, enquanto *union types* e *discriminated unions* foram empregados para modelar estados de carregamento, erro e sucesso das requisições assíncronas.

2.1.3 Tailwind CSS

Tailwind CSS é um framework de estilização baseado no paradigma *utility-first*, fornecendo classes atômicas de baixo nível (como `flex`, `p-4`, `text-lg`) que são compostas diretamente no markup HTML/JSX [14]. Diferentemente de abordagens tradicionais como Bootstrap (baseada em componentes pré-construídos), o Tailwind oferece maior flexibilidade de design sem a necessidade de sobrescrever estilos padrão.

Na versão 4, o Tailwind adota uma abordagem *CSS-first*, eliminando a necessidade do arquivo `tailwind.config.js`. A configuração é feita diretamente via CSS com o diretório `@theme`, e o *tree-shaking* é automático: o plugin `@tailwindcss/vite` detecta quais classes são usadas no código-fonte e gera apenas o CSS necessário, resultando em bundles tipicamente inferiores a 10 KB. A consistência visual foi garantida pela definição de um tema personalizado com cores, espaçamentos e tipografia alinhados à identidade visual do sistema.

2.1.4 shadcn/ui

shadcn/ui é uma coleção de componentes reutilizáveis construídos sobre *Radix UI* (primitivas acessíveis e sem estilo) e estilizados com Tailwind CSS [15]. Diferentemente de bibliotecas tradicionais como Material UI ou Chakra UI, o shadcn/ui não é publicado como um pacote npm instalável — os componentes são copiados diretamente para o código-fonte do projeto, concedendo controle total sobre a implementação e personalização.

No sistema, foram utilizados 46 componentes do shadcn/ui: `Button`, `Card`, `Dialog`, `Form`, `Table`, `Sidebar`, `Chart` e `Toast`. O utilitário `class-variance-authority` gerencia variantes de componentes. As classes CSS são resolvidas com `tailwind-merge` e `clsx`. O `Button` oferece as variantes `default`, `destructive`, `outline`, `secondary`, `ghost` e `link`, definidas via `cva()`.

2.1.5 TanStack Router

Em uma arquitetura SPA, o roteamento ocorre integralmente no lado do cliente: as transições entre telas são resolvidas pelo JavaScript sem recarregar a página. Para isso, foi adotado o TanStack Router, biblioteca de roteamento para React que oferece suporte a *search params* tipados, *loaders* e *actions* que integram o carregamento de dados ao ciclo de vida da navegação [16].

No sistema, as rotas foram organizadas em uma árvore aninhada que reflete a hierarquia de navegação do usuário: rotas públicas (validação de certificados), rotas protegidas (dashboard, gestão de eventos) e rotas administrativas (configurações, relatórios). A estrutura de proteção de rotas foi definida com o recurso *beforeLoad*, que redireciona para a tela de login em caso de falta de autenticação. A autorização de acesso é controlada por perfis (*RBAC*): o colaborador comum acessa apenas funcionalidades de emissão e gestão de certificados, enquanto o administrador tem acesso total ao sistema, incluindo configurações e relatórios. O fluxo completo de login, gerenciamento de sessão e renovação de tokens com Supabase Auth está previsto para versões futuras.

O React Router DOM, embora mais difundido, foi preterido por não oferecer tipagem nativa para *search params* e *path params* — o desenvolvedor precisa declarar tipos manualmente ou usar bibliotecas auxiliares como `zod-router`. O TanStack Router infere os tipos automaticamente a partir da definição da rota, eliminando uma fonte comum de erros em tempo de execução. Além disso, seu modelo de *loaders* e *actions* elimina a necessidade de `useEffect` para buscar dados durante a navegação, substituindo-o por funções assíncronas executadas antes da renderização da rota.

2.1.6 React Hook Form

React Hook Form é uma biblioteca para gerenciamento de formulários em React que minimiza o número de re-renderizações ao utilizar refs em vez de estado controlado [17]. Diferentemente do Formik, que mantém o estado do formulário no React state e dispara uma re-renderização a cada tecla digitada, o React Hook Form registra os campos via refs e só re-renderiza componentes específicos quando necessário — o que resulta em melhor desempenho em formulários com muitos campos. Sua integração com o Zod via `@hookform/resolvers` permite que os esquemas de validação definidos com Zod sejam reutilizados como regras de formulário, garantindo que as mesmas validações aplicadas no backend sejam executadas no frontend antes do envio.

No sistema, o padrão adotado combina `useForm` com `zodResolver` para cada formulário (emissão de certificado, cadastro de template, etc.). As mensagens de erro retornadas pelo Zod são mapeadas automaticamente para os campos correspondentes e exibidas via componentes `FormMessage` do shadcn/ui. O modo `onBlur` dispara a validação no momento em que o usuário sai do campo, oferecendo feedback imediato sem a latência da validação a cada tecla.

2.1.7 Sonner

Sonner é uma biblioteca leve de notificações *toast* para React [18]. Diferentemente de alternativas como `react-hot-toast` e `notistack`, o Sonner prioriza simplicidade (exporta apenas `toast` e `Toaster`) e acessibilidade com suporte a leitores de tela.

No sistema, o Sonner é utilizado para exibir feedback ao usuário após operações assíncronas: sucesso na emissão de certificados, erros de validação, confirmação de exclusão e falhas de rede. As notificações são posicionadas no canto superior direito e configuradas para desaparecer automaticamente após 4 segundos, exceto erros que requerem ação manual.

2.1.8 Recharts

Recharts é uma biblioteca de gráficos para React construída sobre os componentes SVG do D3, seguindo uma abordagem declarativa e composível [19]. Cada tipo de gráfico é um componente React independente (`BarChart`, `PieChart`, `LineChart`), facilitando a personalização sem conhecimento aprofundado de SVG.

A escolha do Recharts em vez do react-chartjs-2 (wrapper para Chart.js) deu-se pela integração mais natural com o ecossistema React: enquanto o Chart.js manipula o canvas diretamente e exige imperativamente chamar `update()` para refletir alterações nos dados, o Recharts utiliza SVG declarativo e reage automaticamente a mudanças de props — o que simplifica a manutenção de gráficos dinâmicos que se atualizam conforme o usuário filtra os dados no dashboard. O D3 puro foi descartado por sua API de baixo nível, que demandaria mais código *boilerplate* para alcançar o mesmo resultado.

No sistema, o Recharts é utilizado no dashboard para exibir métricas como total de certificados emitidos por mês (gráfico de barras) e distribuição por tipo de template (gráfico de pizza). Os componentes `ResponsiveContainer`, `Tooltip` e `Leg`

`end` garantem que os gráficos se adaptem ao tamanho da tela e exibam informações contextuais ao passar o mouse.

2.1.9 date-fns

date-fns é uma biblioteca de utilitários para manipulação de datas em JavaScript, caracterizada por sua abordagem modular e funcional [20]. Cada função é importada individualmente (`format`, `parse`, `differenceInDays`, `isAfter`), evitando o aumento desnecessário do bundle com código não utilizado.

Alternativas como Moment.js foram descartadas porque o Moment.js é considerado obsoleto pela própria equipe de mantenedores (modo de manutenção desde 2020), possui API mutável que pode causar efeitos colaterais inesperados e não suporta *tree-shaking* — importar uma única função inclui a biblioteca inteira no bundle final. O dayjs, embora mais leve que o Moment.js, foi preterido por sua API menos expressiva para operações como *locale* customizado e formatação avançada. O date-fns, por sua vez, adota funções puras (sem mutação), é completamente modular e oferece suporte a 80+ locales, incluindo português brasileiro.

No sistema, o date-fns é empregado para formatar datas de validade de certificados nos componentes de listagem, calcular diferenças entre datas para alertas de expiração e padronizar a exibição de datas no formato local (`dd/MM/yyyy`) via função `format`. A função `isAfter` é utilizada no modelo `certificate.ts` para determinar se um certificado está expirado.

2.1.10 Vite

Vite é uma ferramenta de build e servidor de desenvolvimento que utiliza módulos ES nativos durante o desenvolvimento para oferecer *Hot Module Replacement* (HMR) instantâneo, eliminando a necessidade de empacotamento em tempo de desenvolvimento [11]. Em produção, utiliza Rollup para gerar bundles otimizados com *code splitting* automático e *lazy loading*.

No projeto, o Vite foi escolhido como ferramenta de build do frontend por ser a escolha ideal para **aplicações SPA** — enquanto o Next.js, utilizado no backend, é preferível para aplicações que exigem renderização híbrida ou full-stack [11, 21], o Vite oferece uma experiência de desenvolvimento mais leve e rápida para SPAs puras. O backend, por sua vez, utiliza seu próprio sistema de compilação (`next build`). Essa divisão reforça a independência entre as duas aplicações: alterações no frontend não exigem reconstrução do backend, e vice-versa. A escolha pelo Vite se deu por sua configuração mínima, suporte nativo a TypeScript e HMR rápido, alinhando-se à necessidade de um ambiente de desenvolvimento enxuto e produtivo para o frontend.

2.1.11 Arquitetura e Padrões

O frontend adota uma arquitetura em camadas com React, organizada em quatro níveis. A camada *View* reúne os componentes React que renderizam a interface e capturam eventos do usuário, divididos entre páginas (`views/`) e componentes reutilizáveis (`components/`), incluindo os do shadcn/ui. A camada *ViewModel* é composta por custom hooks (`useCertificatesViewModel`, `useDashboardViewModel`, etc.) que encapsulam o estado local com `useState`, efeitos colaterais com `useEffect`, dados

derivados com `useMemo` e expõem ações de forma coesa — cada hook retorna um objeto com o estado e os manipuladores necessários para sua respectiva tela. A camada *Service* centraliza a comunicação externa: o módulo `api.ts` utiliza a `fetch` API nativa para consumir a API REST, e o módulo `envios.ts` gerencia uma fila local de envios de e-mail no `localStorage`, armazenando apenas dados operacionais (nome e e-mail do destinatário, *status* da entrega). Não há armazenamento de tokens de autenticação ou credenciais no `localStorage` — a autenticação será gerenciada pelo Supabase Auth, que utiliza mecanismo de sessão próprio e seguro. A camada *Model* contém as interfaces TypeScript que definem as entidades de domínio (`Certificate`, `Template`, `VerifyResult`, `EnvioEmail`) com tipos inferidos. O TanStack Router orquestra a navegação, mapeando URLs a componentes (`views/`) de forma declarativa. O Tailwind CSS gerencia a estilização em todas as camadas de apresentação. Essa separação permite testar a lógica de apresentação independentemente da interface, utilizando Jest para validar os *ViewModels* sem depender de um navegador.

O frontend comunica-se com um backend externo por meio de uma API REST, cuja URL base é configurada via variável de ambiente `VITE_API_URL`. As requisições são centralizadas em um módulo `api.ts` que utiliza a `fetch` API nativa do JavaScript, encapsulando o tratamento de erros e a serialização JSON. Os endpoints expostos incluem operações CRUD para templates e certificados, emissão de certificados, geração de PDF e verificação de autenticidade.

2.1.12 Zod

Zod é uma biblioteca de validação de esquemas com suporte primário a TypeScript, que permite definir esquemas de dados com inferência automática de tipos [22]. Diferentemente de alternativas como Joi ou Yup, o Zod foi projetado especificamente para integração com TypeScript, eliminando a necessidade de declarar tipos manualmente: o tipo é inferido diretamente do esquema de validação.

No sistema, os esquemas Zod são utilizados em conjunto com o React Hook Form para validar as entradas dos formulários antes do envio à API. Um mesmo esquema pode validar desde o formato de e-mail até a estrutura completa de um cadastro de certificado, com mensagens de erro localizadas exibidas nos componentes `FormMessage` do shadcn/ui. Essa abordagem garante consistência entre a validação no frontend e as regras aplicadas pelo backend.

O backend do sistema é uma API REST desenvolvida com Next.js e executada no Node.js 22 LTS [23], seguindo uma arquitetura em quatro camadas: apresentação (*Route Handlers*), aplicação (*use cases* e *DTOs*), domínio (*entities* e *value objects* sem dependências externas) e infraestrutura (*repositories*, *services* e acesso a banco de dados). As dependências apontam para dentro: a camada de domínio define interfaces que a infraestrutura implementa, permitindo substituir implementações sem alterar a lógica de negócio.

2.1.13 Next.js 16

Next.js é um framework React que, em sua versão 16, oferece o *App Router* com *Route Handlers* para construção de APIs serverless [21]. Cada endpoint é definido como um arquivo em `src/app/api/` exportando funções assíncronas nomeadas

(`GET`, `POST`, `PUT`, `DELETE`) — não há necessidade de um servidor Express ou configuração de rotas manual.

É importante destacar que, neste projeto, o Next.js é utilizado **exclusivamente como servidor de API (*backend-only*)**: não há páginas, layouts, *Server Components* ou qualquer funcionalidade de renderização frontend. O diretório `src/app/` contém apenas subdiretórios `api/*` com *Route Handlers*; não existe um `page.tsx` ou `layout.tsx` no projeto. O frontend é uma SPA completamente separada (descrita na Seção 2.1), que consome a API via HTTP. O fato de o Next.js ser tipicamente associado a aplicações full-stack não invalida seu uso como servidor de API puro — os *Route Handlers* operam de forma independente do sistema de páginas.

No sistema, os *Route Handlers* atuam como a camada mais externa: recebem a requisição, validam os parâmetros com Zod, instanciam os *use cases* com as dependências e retornam a resposta. O `next.config.ts` configura CORS para a URL do frontend, definida via variável de ambiente, e define o PDFKit como pacote exclusivo do servidor.

A separação arquitetural entre o *frontend* (Vite) e a API foi estabelecida para garantir escalabilidade e permitir que, no futuro, o *backend* possa servir outros clientes, como um aplicativo *mobile*. O uso do Next.js exclusivamente como servidor de API — preterindo alternativas como Express, Fastify ou NestJS — justifica-se pela sua superior Experiência do Desenvolvedor (DX): roteamento baseado em arquivos (*Route Handlers*), validação de variáveis de ambiente e suporte *out-of-the-box* a TypeScript sem configurar *middlewares* manuais ou arquivos `tsconfig` complexos. Além disso, o *deploy* das rotas API no Render ocorre de forma *serverless* e altamente otimizada, eliminando a sobrecarga de abstrações que frameworks como NestJS exigiriam.

2.1.14 Zod

Zod é uma biblioteca de validação de esquemas que, na versão 4, oferece inferência automática de tipos e composição de esquemas [22]. No backend, os esquemas são definidos em `application/dtos/index.ts` e utilizados nos *Route Handlers* para validar o corpo das requisições antes de repassá-los aos *use cases*. Essa validação centralizada garante que apenas dados consistentes cheguem à camada de domínio.

2.1.15 Arquitetura e Padrões

A arquitetura em camadas adotada no backend organiza o código em quatro pacotes: `domain`, `application`, `infrastructure` e `app`. O pacote `domain` contém as entidades e interfaces de repositório. O pacote `application` abriga os *use cases* de emissão, geração e verificação, além dos DTOs. O pacote `infrastructure` implementa os repositórios (`SupabaseCertificateRepository`, `SupabaseTemplateRepository`) e serviços (`PDFKitService`). O `app` contém os *Route Handlers*. Cada *use case* possui uma única responsabilidade, e as entidades encapsulam regras de negócio — `Certificate` valida a data antes de alterá-la.

2.2 BANCO DE DADOS

Esta seção descreve a camada de persistência do sistema. A escolha de um banco relacional gerenciado (PostgreSQL por meio do Supabase) foi orientada pela necessidade de integridade referencial, transações ACID e consultas complexas sobre os dados de certificados e templates, conforme detalhado a seguir.

2.2.1 Supabase e PostgreSQL

Supabase é uma plataforma *open-source* que oferece PostgreSQL como serviço gerenciado, com APIs REST auto-geradas e autenticação integrada [24]. Diferentemente do Firebase (Google), que utiliza um banco NoSQL proprietário, o Supabase utiliza PostgreSQL, um banco relacional maduro que permite consultas complexas, integridade referencial e transações ACID.

A escolha do Supabase em vez de alternativas como Firebase foi motivada pelo modelo de dados relacional do sistema: certificados e templates possuem relações bem definidas (um template pode gerar muitos certificados) e exigem integridade referencial via chaves estrangeiras, algo que o Firestore (NoSQL do Firebase) não oferece nativamente, exigindo validações manuais no código da aplicação. O Appwrite [25], outra alternativa *open-source* BaaS, foi descartado por seu ecossistema menos maduro e pela ausência de um cliente oficial com suporte a SSR para Next.js no momento da decisão.

No sistema, o Supabase é utilizado exclusivamente como banco de dados relacional. As consultas são construídas através do *Query Builder* nativo (`@supabase/supabase-js` e `@supabase/ssr`), dispensando o uso de ORMs pesados como Prisma ou TypeORM. O PostgreSQL 16 [26] atua como o banco de dados subjacente. A opção por utilizar o SDK oficial em vez de um ORM tradicional baseou-se na redução da sobrecarga de processamento (*overhead*) e na mitigação do excesso de arquivos de configuração, permitindo usufruir da tipagem gerada automaticamente pelo Supabase de forma muito mais leve e com excelente controle granular das consultas. O esquema contém duas tabelas principais: `templates` (coluna `layout_config` do tipo JSONB) e `certificates` (índices em `recipient_cpf` e `verification_code`). A nomenclatura segue *snake_case* no banco e *camelCase* no código, com mapeamento manual nos repositórios.

2.3 GERAÇÃO DE DOCUMENTOS

Esta seção descreve as bibliotecas responsáveis pela produção dos certificados em PDF no lado do servidor. A geração programática de documentos foi adotada para garantir consistência visual, segurança e escalabilidade, com destaque para a biblioteca PDFKit e a geração de QR Codes de verificação.

2.3.1 PDFKit

PDFKit é uma biblioteca Node.js para geração de documentos PDF no lado do servidor, sem dependência de ferramentas externas como wkhtmltopdf [27] ou LaTeX [28]. Suporta texto com fontes embutidas, imagens, vetores e cores personalizadas.

A escolha do PDFKit em detrimento de alternativas como Puppeteer baseou-se em eficiência de recursos: o Puppeteer exige a execução de um navegador Chromium

headless completo para rasterizar HTML em PDF, consumindo tipicamente mais de 100 MB de memória por instância. Em um ambiente serverless, onde cada requisição pode inicializar uma nova instância, esse custo inviabiliza o uso. O PDFKit opera exclusivamente em Node.js, com footprint de memória da ordem de dezenas de megabytes, e permite controle programático preciso sobre cada elemento do documento — posicionamento absoluto, fontes embutidas e cores arbitrárias — sem a intermediação de um motor de renderização HTML. Bibliotecas como jsPDF foram descartadas por operarem no lado do cliente (navegador). A arquitetura de geração exclusivamente *server-side* (RF023) foi adotada por três fatores arquiteturais críticos: (1) **Segurança e Integridade**: delegar a geração ao cliente abriria brechas para que usuários manipulassem o código no navegador e alterassem dados (nome, carga horária) no PDF final, comprometendo a confiabilidade do documento; (2) **Consistência Visual**: a renderização no servidor atende rigorosamente à restrição não-funcional de fidelidade visual (RNF030), garantindo um arquivo idêntico independente do dispositivo, navegador ou fontes instaladas na máquina do usuário; e (3) **Automação**: a geração no *backend* é um pré-requisito obrigatório para suportar processamento assíncrono em lote e o anexo de certificados em rotinas automáticas de e-mail.

No sistema, o PDFKitService implementa a interface `IPDFService` definida no domínio, seguindo o princípio de inversão de dependência. O certificado é gerado em formato paisagem A4, com fundo colorido, bordas decorativas, título “CERTIFICADO”, nome do destinatário, curso, carga horária, datas de emissão e validade, CPF formatado e código de verificação gerado com UUID v4 [29]. Todas as cores, tamanhos e visibilidade dos elementos são controlados pelo `layoutConfig` do template, armazenado como JSONB no banco de dados. O PDF é retornado como `Uint8Array` no corpo do `NextResponse` (`newNextResponse(newUint8Array(pdfBuffer))`), sem a sobrecarga de uma stream — adequado para o cenário de download individual do MVP.

2.3.2 qrcode

`qrcode` é uma biblioteca Node.js que gera imagens de *QR Code* nos formatos PNG, SVG e UTF-8, sem dependência de navegador [30]. A função `toBuffer()` é utilizada no serviço `PDFKitService` para converter a URL pública de verificação — construída no formato `/verificar?cpf=...&codigo=...` — em um buffer de imagem PNG, que é inserido no certificado via `PDFKit.image()`. Essa abordagem é necessária porque o PDFKit é um motor de renderização de PDF — ele desenha textos, formas e imagens, mas não possui capacidade nativa de codificar *QR Codes*. A geração do *QR Code*, portanto, é delegada a uma biblioteca especializada externa.

A opção de desenhar o *QR Code* manualmente com retângulos do PDFKit foi avaliada e preterida por três motivos: (1) **simplicidade** — `toBuffer(url)` gera o PNG em uma linha, enquanto desenhar manualmente exigiria implementar a codificação Reed-Solomon e a matriz de módulos do QR; (2) **manutenção** — a lógica de QR fica isolada na biblioteca; qualquer atualização de padrão ou segurança vem do pacote, sem necessidade de alterar o `PDFKitService`; e (3) **cor personalizada** — o `qrcode` já suporta `color.dark` e `color.light` de fábrica, tornando trivial usar a cor primária do template (`template.layoutConfig.primaryColor`) sem manipulação adicional de pixels.

No sistema, o *QR Code* é gerado exclusivamente no servidor durante a produção do PDF, utilizando a cor primária do template para manter a identidade visual do documento. A legenda “Verifique o certificado” é posicionada abaixo do código para orientar o usuário. Alternativas como a implementação manual puramente em JavaScript foram descartadas por exigirem a reimplementação da codificação Reed-Solomon e da matriz de módulos, acrescentando complexidade sem ganho real em relação ao binding nativo do libqrencode.

2.3.3 Render

Render é uma plataforma de computação em nuvem que oferece hospedagem simplificada para aplicações web, sites estáticos e serviços, com deploys automáticos a partir de repositórios Git e certificados SSL gratuitos [31].

O frontend, construído como uma SPA com Vite, é compilado para arquivos estáticos (HTML, CSS, JS) e implantado como um *Static Site* no Render. O deploy é acionado automaticamente a cada *push* na branch principal do GitHub. O arquivo `index.html` na raiz do projeto serve como ponto de entrada, e o roteamento no lado do cliente é gerenciado pelo TanStack Router, que utiliza *History API* para navegação sem recarregamento de página.

Alternativas como Vercel e Netlify foram avaliadas. O Render foi escolhido por três motivos principais: (1) diferentemente da Vercel, que detecta automaticamente o tipo de projeto e o configura como *Node.js* mesmo para SPAs — adicionando um runtime desnecessário e consumindo recursos — o Render permite definir manualmente o frontend como *Static Site*, garantindo que apenas arquivos estáticos sejam servidos sem processo Node ativo; (2) a possibilidade de hospedar tanto o frontend quanto o backend na mesma plataforma, centralizando a gestão de infraestrutura em um único provedor — o frontend como *Static Site* e o backend como *Web Service*; e (3) ausência de limite de largura de banda na camada gratuita, diferentemente da Vercel que impõe limites em planos gratuitos. O Cloudflare Pages também foi descartado por seu ecossistema de *Workers* focado em funções *edge*, modelo que não se aplica a um backend Node.js tradicional com dependências como PDFKit.

2.3.4 Jest

Jest é um framework de testes JavaScript desenvolvido pelo Facebook, que oferece um conjunto completo de funcionalidades: *runner* de testes, asserções, *mocking*, *code coverage* e *snapshot testing* [32]. Suporte a TypeScript é provido via `ts-jest`.

Embora reconheça-se que o Vitest seja a ferramenta natural para o ecossistema Vite atual em virtude de sua velocidade extrema, o Jest foi a ferramenta eleita para o escopo deste trabalho. Essa decisão arquitetural fundamenta-se na vasta disponibilidade de material acadêmico e técnico, documentação extensiva de *mocks* e integração nativa consagrada. Em um cenário onde a meta principal era implementar métricas rígidas de cobertura de código e atestar a separação em camadas da *Clean Architecture*, o uso do Jest minimizou os riscos de bloqueios operacionais e incompatibilidades de bibliotecas (`@testing-library/react`, `jest-dom`) durante o curto prazo de desenvolvimento do Produto Mínimo Viável (MVP).

No sistema, o Jest está configurado com cobertura mínima de 75%, validada através da flag `--coverage` no pipeline de CI. Testes unitários foram escritos para as

ViewModels e funções utilitárias, enquanto a configuração de *mocking* permite isolar as chamadas de API durante os testes sem depender de um servidor backend ativo.

2.3.5 ESLint e Prettier

ESLint é uma ferramenta de análise estática de código que identifica padrões problemáticos, erros de sintaxe e violações de boas práticas [33]. Prettier [34] é um formatador de código opinativo que garante consistência estilística em todo o projeto, eliminando discussões sobre espaçamento, aspas e ponto-e-vírgula.

No sistema, ambos os módulos são configurados com ESLint e Prettier. O frontend utiliza o formato `.eslintrc` tradicional com os plugins `eslint-plugin-react-hooks` para hooks, `@typescript-eslint` para tipagem e `eslint-config-prettier` para evitar conflitos. O backend adota a *flat config* do ESLint v9 (`eslint.config.mjs`), com suporte a TypeScript via `typescript-eslint`, integração ao Prettier via `eslint-plugin-prettier` e escopo restrito ao Node.js (sem regras React). Ambos os projetos incluem `.prettierrc` com indentação de 2 espaços, aspas duplas, ponto-e-vírgula obrigatório e *trailing commas*. A integração com o VS Code destaca erros em tempo real e formata o código ao salvar.

2.3.6 Git e GitHub

Git [35] é um sistema de controle de versão distribuído que permite rastrear alterações no código-fonte, gerenciar ramos de desenvolvimento paralelo e colaborar entre múltiplos desenvolvedores. GitHub [36] é a plataforma de hospedagem de repositórios que estende o Git com *Pull Requests*, *Issues*, *Actions* para CI/CD e revisão de código.

O fluxo de trabalho adotado foi o *Git Flow* simplificado, com as branches `main` (produção), `dev` (desenvolvimento) e `feature/*` (funcionalidades específicas). Cada funcionalidade é desenvolvida em uma branch separada e integrada à `dev` via *Pull Request*, que passa por revisão de código antes do merge.

2.3.7 Visual Studio Code

Visual Studio Code [37] é um editor de código-fonte desenvolvido pela Microsoft, baseado no Electron e no protocolo *Language Server Protocol* (LSP), que oferece suporte avançado a TypeScript, depuração integrada, terminal embutido e um ecossistema rico de extensões.

A produtividade foi ampliada com extensões como ESLint (análise estática), Prettier (formatação automática), Tailwind CSS IntelliSense (autocompletar para classes) e GitHub Copilot (sugestões contextuais).

O Quadro 1 apresenta um resumo das tecnologias discutidas neste capítulo, com suas respectivas finalidades no contexto do sistema.

Quadro 1 – Tecnologias utilizadas no desenvolvimento do sistema

Tecnologia	Versão	Finalidade
Frontend		
React	19	Construção da interface com componentes reutilizáveis
TypeScript	5.x	Tipagem estática e segurança em tempo de compilação
Tailwind CSS	4.x	Estilização utilitária com configuração *CSS-first*
shadcn/ui	~2.x	Componentes reutilizáveis baseados em Radix UI
TanStack Router	~1.x	Roteamento declarativo com proteção de rotas
React Hook Form	~7.x	Gerenciamento performático de formulários
Sonner	~2.x	Notificações *toast* acessíveis
Recharts	~2.x	Gráficos declarativos para *dashboard*
date-fns	~4.x	Utilitários de manipulação de datas
Vite	~7.x	Servidor de desenvolvimento e empacotamento
Zod	~3.x / 4.x	Validação de esquemas no *frontend* e *backend*
Backend		
Next.js	16	Framework para API REST com *Route Handlers*
Node.js	22 LTS	Ambiente de execução do servidor
Banco de Dados		
Supabase	~2.x	Plataforma de banco PostgreSQL gerenciado
PostgreSQL	16	Banco relacional com integridade referencial
Geração de Documentos		
PDFKit	~0.18	Geração *server-side* de certificados em PDF
qrcode	~1.5	Geração de *QR Codes* no servidor para PDF
UUID	~14.x	Geração de identificadores únicos para verificação
Infraestrutura e DevOps		
Render	SaaS	Hospedagem do frontend como site estático
Git	~2.x	Controle de versão distribuído
GitHub	SaaS	Repositório remoto e CI/CD via Actions
Ferramentas de Desenvolvimento		
Jest	~30.x	Testes unitários com cobertura mínima de 75%
ESLint	~9.x	Análise estática e *linting* de código
Prettier	~3.x	Formatador opinativo de código
Visual Studio Code	~1.x	Ambiente de desenvolvimento

Fonte: Elaborado pelo Autor (2026).

3 MODELAGEM DO PROJETO

Com base nos requisitos levantados na introdução, este capítulo descreve a modelagem completa do sistema. A discussão que se segue detalha o processo de levantamento de requisitos, os diagramas de casos de uso e classes, a arquitetura adotada, o modelo entidade-relacionamento e as interfaces implementadas.

3.1 LEVANTAMENTO DE REQUISITOS

O levantamento de requisitos foi realizado por meio de um estudo de caso em uma organização do ramo de treinamentos corporativos, utilizando entrevista semi-estruturada com o responsável pela emissão de certificados, juntamente com a observação direta do processo manual executado por ele. Durante a entrevista, foram identificadas as principais dificuldades enfrentadas, como o tempo elevado para emissão manual, a dificuldade em gerenciar grandes volumes de certificados e a ausência de um mecanismo de validação pública. A análise do processo permitiu mapear o

fluxo completo de emissão, desde o cadastro de participantes até a entrega do certificado, servindo como base para a definição dos requisitos funcionais e não funcionais apresentados a seguir.

Requisitos Funcionais:

Os requisitos foram priorizados segundo o método MoSCoW, alinhado à abordagem MVP (Produto Mínimo Viável) adotada no projeto: **[M]** *Must have* — funcionalidades essenciais implementadas no MVP; **[S]** *Should have* — importantes, mas não críticas para a primeira versão; **[C]** *Could have* — desejáveis para versões futuras; **[W]** *Won't have* — fora do escopo do MVP.

3.1.1 Módulo de Autenticação e Controle de Acesso

- RF001 [S] – Permitir login por e-mail e senha.
- RF002 [S] – Permitir redefinição de senha via e-mail.
- RF003 [S] – Validar credenciais antes de conceder acesso.
- RF004 [S] – Permitir cadastro de novos usuários (apenas administrador).
- RF005 [S] – Permitir gerenciamento de perfis e permissões.
- RF006 [S] – Controlar acesso a áreas restritas com base no status de autenticação do usuário.
- RF007 [S] – Implementar bloqueio de múltiplas sessões simultâneas por conta.

3.1.2 Módulo de Templates de Certificados

- RF008 [M] – Permitir criação e configuração de templates por editor visual.
- RF009 [M] – Permitir personalização de templates.
- RF010 [M] – Permitir selecionar template da galeria interna.
- RF011 [C] – Identificar campos dinâmicos automaticamente.
- RF012 [M] – Permitir criação/edição manual de campos dinâmicos (ex: {{nome}}).
- RF013 [M] – Salvar configurações de mapeamento de campos.
- RF014 [C] – Permitir duplicar templates existentes.

3.1.3 Módulo de Entrada de Dados

- RF015 [M] – Permitir cadastro individual de participantes.
- RF016 [M] – Permitir emissão manual para participante único.
- RF017 [C] – Importar dados via CSV.
- RF018 [C] – Importar dados via Excel (.xlsx).
- RF019 [C] – Configurar gatilhos de emissão automática.
- RF020 [C] – Permitir upload de lista em lote.

- RF021 [C] – Processar múltiplos certificados em fila.
- RF022 [C] – Notificar ao término do processamento.

3.1.4 Módulo de Geração de Certificados

- RF023 [M] – Gerar certificados em PDF.
- RF024 [M] – Preencher automaticamente campos dinâmicos.
- RF025 [M] – Gerar código único por certificado.
- RF026 [M] – Gerar *QR Code* no certificado com opções de personalização.
- RF027 [M] – Gerar hash de integridade para cada certificado.
- RF028 [M] – Registrar data, hora e localidade da emissão.
- RF029 [S] – Permitir reemissão mantendo histórico.
- RF030 [M] – Permitir download individual.
- RF031 [S] – Permitir download em lote.

3.1.5 Módulo de Envio e Comunicação

- RF032 [S] – Enviar certificado por e-mail ao participante.
- RF033 [C] – Permitir personalização do corpo do e-mail com variáveis.
- RF034 [S] – Registrar status de envio (pendente, enviado, falha).
- RF035 [C] – Reenviar automaticamente certificados com falha de envio.

3.1.6 Módulo de Validação Pública e Antifraude

- RF036 [M] – Disponibilizar portal público de validação.
- RF037 [M] – Validar certificado via código único.
- RF038 [M] – Exibir informações básicas ao validar (nome, curso, data, validade, autenticidade).

3.1.7 Módulo de Assinaturas Digitais

- RF039 [C] – Permitir adicionar assinatura digitalizada ao template.
- RF040 [C] – Registrar hash de verificação da assinatura.

3.1.8 Módulo de Dashboard e Gestão

- RF041 [M] – Exibir lista completa de certificados emitidos.
- RF042 [M] – Permitir filtros por curso, período, status e instrutor.
- RF043 [M] – Exibir estatísticas mensais de emissão.
- RF044 [C] – Exibir logs de ações realizadas.

3.1.9 Módulo de Auditoria e Segurança Operacional

- RF045 [S] – Registrar responsável por cada certificado gerado.
- RF046 [S] – Registrar data, hora e local de ações relevantes.
- RF047 [S] – Permitir arquivamento de certificados como alternativa à exclusão definitiva.

3.1.10 Módulo de Gestão de Cursos/Eventos

- RF048 [S] – Permitir cadastro de cursos/eventos.
- RF049 [S] – Associar participantes ao curso.
- RF050 [S] – Definir carga horária.
- RF051 [S] – Registrar instrutor responsável.
- RF052 [S] – Configurar datas de início e término.
- RF053 [S] – Listar certificados vinculados ao curso.
- RF054 [S] – Validar vínculo do participante ao curso antes de permitir a emissão.
- RF055 [S] – Registrar local do curso.

3.1.11 Módulo de Participantes

- RF056 [S] – Cadastro completo de participantes (nome, e-mail e CPF).
- RF057 [S] – Permitir edição de dados.
- RF058 [S] – Importar participantes sem duplicidade.
- RF059 [S] – Exibir histórico de certificados por participante.
- RF060 [S] – Permitir busca por nome, e-mail e CPF.

3.1.12 Módulo Multiempresa / Multi-tenant

- RF061 [C] – Permitir cadastro de organizações com dados institucionais (CNPJ, razão social, endereço e contato).
- RF062 [C] – Associar usuários cadastrados à sua respectiva organização.
- RF063 [C] – Permitir que cada organização gerencie seus próprios templates.
- RF064 [C] – Permitir que cada organização gerencie seus próprios usuários e perfis de acesso.

3.1.13 Módulo de Configurações do Sistema

- RF065 [C] – Personalizar mensagens padrão.
- RF066 [C] – Ativar/desativar envio automático.
- RF067 [C] – Configurar regras de reemissão.

3.1.14 Módulo de Armazenamento e Arquivos

- RF068 [M] – Armazenar templates no sistema.

3.1.15 Módulo de Notificações e Alertas

- RF069 [S] – Notificar sucesso na emissão de certificado.
- RF070 [S] – Notificar falhas de emissão.
- RF071 [C] – Notificar sucesso no processamento de planilhas.
- RF072 [C] – Alertar quando template estiver incompleto ou inválido.

3.1.16 Módulo de Monitoramento e Logs

- RF073 [S] – Registrar logs de autenticação e tentativas de acesso.
- RF074 [C] – Registrar eventos críticos (falha geração PDF etc.).
- RF075 [C] – Permitir exportação de logs.

3.1.17 Módulo de Suporte, Operação e Manutenção

- RF076 [C] – Permitir abertura de chamados de suporte.
- RF077 [C] – Exibir status do sistema (online/manutenção).
- RF078 [C] – Permitir atualização de termos de uso e políticas.

Requisitos Não Funcionais:

Os requisitos não funcionais foram priorizados segundo o método MoSCoW, alinhado à abordagem MVP (Produto Mínimo Viável) adotada no projeto: **[M]** *Must have* — requisitos essenciais implementados no MVP; **[S]** *Should have* — importantes, mas não críticos para a primeira versão; **[C]** *Could have* — desejáveis para versões futuras; **[W]** *Won't have* — fora do escopo do MVP.

3.1.18 Segurança

- RNF001 – [S] O sistema deve utilizar o Supabase Auth para gerenciamento seguro de autenticação e armazenamento de senhas com hash criptográfico seguro (bcrypt).
- RNF002 – [M] O sistema deve utilizar conexão segura HTTPS (TLS 1.2 ou superior) em todas as comunicações entre o cliente e o servidor, garantindo confidencialidade e integridade dos dados em trânsito.
- RNF003 – [S] O sistema deve proteger dados sensíveis em repouso utilizando criptografia fornecida pela infraestrutura em nuvem.
- RNF004 – [S] O sistema deve bloquear autenticação após 5 tentativas consecutivas inválidas (conforme política do provedor de autenticação).
- RNF005 – [C] Sessões inativas devem expirar automaticamente após tempo configurável (ex.: 30 minutos).

- RNF006 – [M] O sistema deve implementar controle de acesso baseado em papéis (RBAC).
- RNF007 – [S] O sistema deve seguir boas práticas de segurança conforme OWASP Top 10.
- RNF008 – [S] O sistema deve registrar logs de autenticação e eventos de segurança, incluindo tentativas de login, falhas, acessos negados e alterações de configuração, com data, hora e identificação do usuário.
- RNF009 – [C] O sistema deve garantir isolamento de dados entre empresas (multi-tenant).
- RNF010 – [W] O sistema deve permitir futura implementação de autenticação multi-fator (MFA).
- RNF011 – [C] O sistema deve permitir configurar horários de emissão por perfil de colaborador como política de segurança.

3.1.19 Desempenho e Escalabilidade

Os valores numéricos apresentados a seguir são estimativas definidas pelo desenvolvedor com base no porte esperado da operação, considerando turmas típicas de 20 a 50 participantes, palestras com até 200 participantes e emissões recorrentes ao longo do ano. Os limites estabelecidos consideram uma margem de segurança para absorver picos sazonais sem degradação da experiência do usuário.

- RNF012 – [M] O sistema deve suportar geração simultânea de, estimados, 500 certificados por lote, volume equivalente a duas turmas grandes ou dez turmas médias processadas em única operação.
- RNF013 – [M] O tempo médio de geração individual não deve exceder 5 segundos por certificado, garantindo que um lote de 500 certificados seja concluído em aproximadamente 42 minutos.
- RNF014 – [M] O portal público de validação deve responder em até 2 segundos.
- RNF015 – [S] O sistema deve permitir escalabilidade horizontal do serviço de geração.
- RNF016 – [S] O sistema deve suportar volume estimado de 10.000 certificados/mês, compatível com a projeção de emissões recorrentes acrescida de margem de segurança.
- RNF017 – [M] A geração em lote deve ser processada de forma assíncrona por meio de fila de processamento, sem bloquear a aplicação principal.
- RNF018 – [S] A fila de certificados deve garantir processamento ordenado e controle de concorrência.

3.1.20 Disponibilidade e Confiabilidade

- RNF019 – [S] O sistema deve garantir disponibilidade mínima de 99,5% mensal.

- RNF020 – [S] O sistema deve realizar backup automático diário do banco de dados, com retenção mínima de 30 dias, garantindo recuperação de dados em caso de falha ou corrupção.
- RNF021 – [C] O sistema deve permitir restauração de dados em até 24 horas.
- RNF022 – [S] O sistema deve implementar mecanismo de retentativa automática para tarefas falhas na fila de certificados.
- RNF023 – [M] O sistema deve registrar falhas de processamento para auditoria posterior.

3.1.21 Usabilidade e Acessibilidade

- RNF024 – [M] O sistema deve ser responsivo (desktop, tablet e mobile).
- RNF025 – [M] O sistema deve manter padrão visual consistente.
- RNF026 – [M] O sistema deve fornecer mensagens de erro claras.
- RNF027 – [S] O sistema deve permitir navegação por teclado.
- RNF028 – [C] O sistema deve seguir recomendações WCAG 2.1 nível AA (quando aplicável).

3.1.22 Compatibilidade e Integração

- RNF029 – [M] O sistema deve ser compatível com navegadores modernos (Chrome, Firefox, Edge, Safari).
- RNF030 – [M] A geração de PDF deve preservar fidelidade visual do template original.
- RNF031 – [M] O sistema deve disponibilizar API REST autenticada via token JWT do Supabase.
- RNF032 – [S] O sistema deve aceitar arquivos CSV codificados em UTF-8.
- RNF033 – [C] O sistema deve permitir integração via Webhooks em formato JSON.

3.1.23 Armazenamento e Retenção

- RNF034 – [S] O sistema deve armazenar certificados de forma permanente, garantindo validação indefinida pelo portal público.
- RNF035 – [S] O sistema deve aplicar exclusão lógica (soft delete) quando aplicável.

3.1.24 Conformidade Legal e LGPD

- RNF036 – [C] O sistema deve estar em conformidade com a LGPD (Lei nº 13.709/2018), garantindo tratamento adequado dos dados pessoais coletados, incluindo consentimento, finalidade específica, minimização de dados e direitos do titular.
- RNF037 – [C] O sistema deve permitir exclusão ou anonimização de dados pessoais mediante solicitação.

- RNF038 – [C] O sistema deve registrar consentimento do usuário para o tratamento de dados pessoais, incluindo finalidade, base legal e data da coleta.
- RNF039 – [S] Logs sensíveis devem ser acessíveis apenas por administradores autorizados.
- RNF040 – [S] O sistema deve manter trilha de auditoria inviolável para operações críticas.

3.1.25 Arquitetura e Tecnologia

- RNF041 – [M] O sistema deve ser composto por duas aplicações independentes: uma *Single Page Application* (SPA) para o frontend e uma API REST para o backend, cada uma com seu próprio ciclo de vida de desenvolvimento e deploy.
- RNF042 – [M] O frontend deve adotar arquitetura em camadas com React, com quatro camadas: (i) *View* — componentes de interface; (ii) *ViewModel* — custom hooks que encapsulam estado e ações; (iii) *Service* — cliente de comunicação com a API; (iv) *Model* — interfaces TypeScript que definem as entidades de domínio.
- RNF043 – [M] O backend deve adotar arquitetura em camadas (domínio, aplicação e infraestrutura), com as dependências apontando para a camada de domínio (princípio de inversão de dependências).
- RNF044 – [M] A comunicação entre frontend e backend deve ocorrer exclusivamente via API REST sobre HTTPS, com autenticação baseada em token JWT.
- RNF045 – [S] A geração de certificados em lote deve ser processada de forma assíncrona por meio de fila de processamento, sem bloquear a aplicação principal.
- RNF046 – [M] O sistema deve utilizar banco de dados relacional com integridade referencial (PostgreSQL), gerenciado pelo Supabase.
- RNF047 – [C] O sistema deve implementar estratégia de isolamento multi-tenant (por schema ou coluna organizacional).
- RNF048 – [S] O sistema deve implementar monitoramento e observabilidade centralizada (logs, métricas e rastreamento).
- RNF049 – [M] O frontend deve ser compilado como site estático e implantado em plataforma de hospedagem com deploy contínuo a partir do repositório Git.
- RNF050 – [M] O backend deve ser implantado como serviço web independente em plataforma em nuvem compatível com Node.js.
- RNF051 – [S] O sistema deve permitir extensibilidade, garantindo que novos módulos possam ser adicionados seguindo os mesmos padrões arquiteturais já estabelecidos.

3.1.26 Manutenção e Monitoramento

- RNF052 – [S] O sistema deve permitir atualização com mínima indisponibilidade (deploy contínuo).

- RNF053 – [M] O sistema deve possuir documentação técnica e manual do usuário.
- RNF054 – [S] O sistema deve registrar erros em sistema centralizado de monitoramento.
- RNF055 – [S] O sistema deve permitir extensibilidade sem impacto nas funcionalidades existentes.
- RNF056 – [C] O sistema deve manter histórico de versões de templates, configurações e endpoints da API para fins de auditoria, rastreabilidade e versionamento.

3.1.27 Internacionalização

- RNF057 – [S] O sistema deve permitir configuração de timezone por organização.
- RNF058 – [W] O sistema deve permitir tradução futura da interface.
- RNF059 – [C] O sistema deve suportar diferentes formatos de data e hora.

3.2 REGRAS DE NEGÓCIO

Além dos requisitos funcionais, foram identificadas regras de negócio que impõem restrições ou condições sobre as funcionalidades do sistema:

- RN01 – Não deve ser permitido acesso a áreas restritas sem autenticação prévia.
- RN02 – Certificados não podem ser excluídos definitivamente, devendo ser arquivados.
- RN03 – Certificados só podem ser emitidos para participantes previamente vinculados ao curso ou evento.
- RN04 – O reenvio automático de certificados com falha deve respeitar o limite máximo de 3 tentativas com intervalo mínimo de 10 minutos entre cada uma.

3.2.1 Status de Implementação das Regras de Negócio

As regras de negócio acima representam o modelo completo desejado para o sistema. No entanto, considerando o escopo do MVP (v1.0), algumas regras ainda não foram implementadas:

- **RN02 —** No MVP atual, a exclusão de templates e certificados é realizada por exclusão definitiva (*hard delete*), conforme testado nos casos CT005 e CT056 do plano de testes. A regra de arquivamento (*soft delete*) está prevista para a versão 2.0, conforme o requisito RF047 [S].
- **RN03 —** A validação de vínculo entre participante e curso não é verificada no MVP, pois as entidades Participantes e Cursos ainda não foram materializadas no banco de dados.
- **RN04 —** O reenvio automático com limite de tentativas não foi implementado no MVP; o envio de e-mails é simulado sem controle de retentativas.

A implementação completa dessas regras está vinculada aos requisitos classificados como [S] ou [C], que serão tratados em versões futuras do sistema.

3.3 CRITÉRIOS DE ACEITAÇÃO

Cada requisito funcional possui critérios de aceitação que definem as condições necessárias para considerá-lo atendido. O Quadro 2 apresenta os critérios para todos os RFs.

Quadro 2 – Critérios de aceitação dos requisitos funcionais

RF	Descrição	Critério de Aceitação
RF001	Login por e-mail e senha	Usuário com credenciais válidas acessa o sistema; credenciais inválidas exibem mensagem de erro sem redirecionar.
RF002	Redefinição de senha	Usuário clica em "Esqueci senha", insere e-mail e recebe link de redefinição válido por 1 hora.
RF003	Validar credenciais	Rotas protegidas redirecionam para login quando o usuário não está autenticado; API retorna 401 para requisições sem token.
RF004	Cadastro de usuários	Apenas administrador pode acessar tela de cadastro; formulário exige nome, e-mail e senha; usuário criado aparece na lista.
RF005	Gerenciar perfis	Administrador pode listar, criar, editar e desativar usuários; alterações persistem após recarregar.
RF006	Controle de acesso	Usuário não autenticado é redirecionado para login ao acessar qualquer rota administrativa.
RF007	Sessão única	Login em novo dispositivo invalida sessão anterior; sessão expira após 30 min de inatividade.
RF008	Criar template	Usuário cria novo template via editor visual, configura layout e campos dinâmicos; template aparece na galeria após salvar.
RF009	Personalização	Cores, fontes e bordas configuradas são refletidas no preview visual.
RF010	Selecionar template	Galeria exibe templates salvos com nome e preview; seleção carrega o template para edição.
RF011	Identificar campos automáticos	Sistema sugere campos {{nome}}, {{curso}} ao fazer upload; sugestões são editáveis.
RF012	Criar/editar campos	Usuário adiciona campo {{cpf}}, salva, e o campo aparece na tela de emissão.
RF013	Salvar configurações	Mapeamento de campos persiste após recarregar a página.
RF014	Duplicar templates	Duplicata cria cópia idêntica com nome "(cópia)"; original não é alterado.
RF015	Cadastro individual	Formulário válido salva participante; e-mail ou CPF duplicado exibe alerta; participante aparece na lista.
RF016	Emissão manual	Selecionar template, preencher dados e confirmar gera certificado listado no dashboard.
RF017	Importar CSV	Arquivo CSV com cabeçalho correto importa participantes; linhas com erro são ignoradas e reportadas.
RF018	Importar Excel	Arquivo .xlsx com abas é processado; primeira aba é usada como padrão.
RF019	Gatilhos automáticos	Ao cadastrar participante em curso com gatilho ativo, certificado é emitido automaticamente em até 30s.
RF020	Upload em lote	Arquivo com 100 participantes é processado; certificados são gerados para todos.
RF021	Fila de processamento	Lote de 500 participantes é enfileirado; barra de progresso mostra andamento.

RF	Descrição	Critério de Aceitação
RF022	Notificar término	Processamento concluído envia notificação na tela (toast) com resumo.
RF023	Gerar PDF	PDF gerado preserva layout, cores e fontes do template; campos preenchidos aparecem nas posições corretas; geração leva menos de 5 segundos.
RF024	Preencher campos	Dados do participante substituem corretamente os {{placeholders}} no PDF.
RF025	Código único	Cada certificado possui código único (UUID v4); consulta no banco retorna apenas um registro.
RF028	Data/hora/local	Certificado exibe data e hora da emissão; localização é definida pelo curso ou configuração.
RF029	Reemissão	Reemissão gera novo PDF com novo código; histórico lista versão anterior com data.
RF030	Download individual	PDF é baixado com nome no formato `Nome_Curso_Data.pdf`.
RF031	Download em lote	Selecionar múltiplos certificados e baixar gera arquivo .zip com todos os PDFs.
RF033	Personalizar e-mail	Variáveis {{nome}}, {{link}} são substituídas no corpo do e-mail enviado.
RF034	Status de envio	Certificado exibe status "Pendente", "Enviado"ou "Falha"; status é atualizado em tempo real.
RF035	Reenvio automático	Falha de envio dispara nova tentativa após 10 min, até 3 tentativas.
RF036	Portal de validação	URL pública `/validar` exibe campo para inserir código; página é responsiva.
RF037	Validar via código	Código válido exibe dados do certificado; inválido mostra "Certificado não encontrado".
RF026	*QR Code*	*QR Code* no PDF redireciona para URL de validação com código como parâmetro.
RF038	Exibir informações	Tela de validação mostra nome, curso, data, carga horária e status "Autêntico".
RF039	Assinatura digitalizada	Imagem de assinatura aparece na posição configurada no template.
RF040	Hash da assinatura	Hash SHA-256 da assinatura é armazenado e verificável.
RF041	Lista de certificados	Tabela exibe certificados com colunas: nome, curso, data, status; paginação de 20 itens.
RF042	Filtros	Filtrar por curso, período, status e instrutor atualiza a tabela sem recarregar a página.
RF043	Estatísticas mensais	Dashboard exibe total de emissões por mês em gráfico de barras; dados são atualizados automaticamente.
RF044	Logs de ações	Tabela de logs exibe ação, usuário, data e detalhes; ordenada por data decrescente.
RF045	Responsável	Certificado gerado registra ID do usuário que emitiu; exibido no histórico.
RF046	Data/hora ações	Toda ação de emissão, edição e exclusão registra timestamp.
RF047	Arquivamento	Certificado arquivado não aparece na lista padrão, mas é recuperável via filtro "Arquivados".

RF	Descrição	Critério de Aceitação
RF048	Cadastro cursos	Formulário salva nome, tipo, carga horária, datas e instrutor; curso aparece na lista.
RF049	Associar participantes	Participante é vinculado ao curso no momento da emissão.
RF050	Carga horária	Carga horária definida no curso aparece no campo correspondente do certificado.
RF051	Instrutor	Nome do instrutor é salvo e exibido no certificado.
RF052	Configurar datas	Datas configuradas são preenchidas automaticamente no certificado.
RF053	Listar por curso	Filtro "Curso"exibe apenas certificados daquele curso.
RF054	Validar vínculo	Emissão é bloqueada se participante não pertence ao curso selecionado; mensagem exibe motivo.
RF055	Registrar local	Local do curso é salvo e exibido no certificado.
RF056	Cadastro completo	Nome, e-mail e CPF são obrigatórios; e-mail inválido é rejeitado.
RF057	Edição	Dados editados são salvos e refletidos na tela de emissão.
RF058	Importar sem duplicidade	Mesmo CPF/e-mail não cria registro duplicado; dados existentes são atualizados.
RF059	Histórico	Página do participante exibe todos os certificados emitidos para ele.
RF060	Busca	Busca por nome retorna resultados em menos de 2s; busca vazia retorna lista completa.
RF061	Cadastro organizações	Formulário salva CNPJ, razão social e contato; organização aparece na lista.
RF062	Associar user/org	Usuário é vinculado a uma organização no cadastro; vínculo é editável.
RF063	Templates por org	Organização A não vê templates da organização B.
RF064	Usuários por org	Organização A não vê usuários da organização B.
RF065	Mensagens padrão	Texto padrão de e-mail é configurável por organização.
RF066	Envio automático	Opção "Enviar automaticamente"é ativável/desativável por organização.
RF067	Regras de reemissão	Reemissão é permitida apenas se configurada; limite de reemissões é respeitado.
RF068	Armazenar templates	Template enviado fica imediatamente disponível para uso; download do template original é possível.
RF070	Notificar falhas	Falha de emissão dispara notificação ao administrador com detalhes do erro.
RF071	Notificar sucesso	Processamento concluído com sucesso exibe toast com total de certificados.
RF072	Alertar template inválido	Template sem campos {{nome}} exibe alerta antes da emissão.
RF073	Logs de autenticação	Tentativa de login com senha incorreta é registrada com timestamp e IP.
RF074	Eventos críticos	Falha de geração de PDF é registrada com stack trace e ID do certificado.
RF075	Exportar logs	Logs são exportáveis em CSV com filtro por período.
RF076	Chamados de suporte	Formulário de chamado salva descrição, prioridade e contato; chamado é listado.

RF	Descrição	Critério de Aceitação
RF077	Status do sistema	Página de status exibe "Online", "Em manutenção"ou "Indisponível".
RF078	Termos de uso	Atualização de termos exibe aviso para aceite na próxima autenticação.

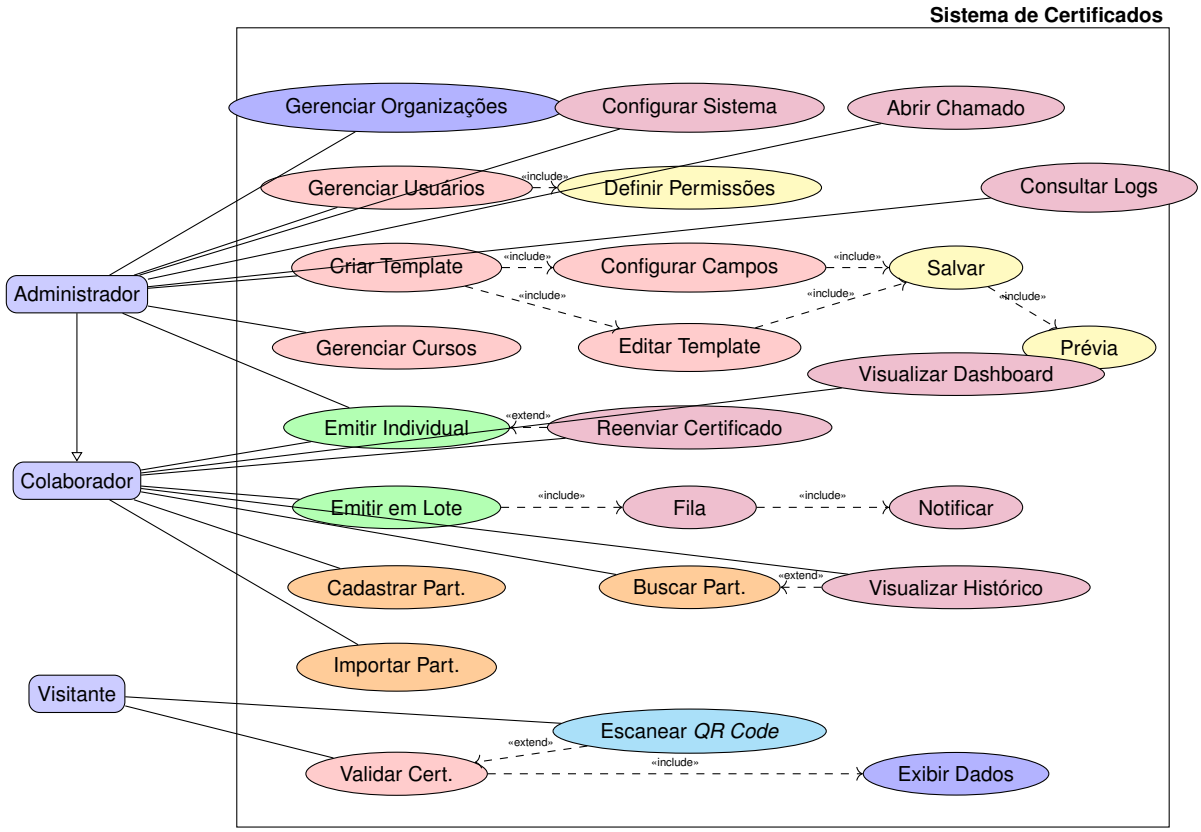
Fonte: Elaborado pelo Autor (2026).

3.4 DIAGRAMAS DE CASOS DE USO

O diagrama de casos de uso da Figura 1 apresenta a visão completa do sistema, incluindo todas as funcionalidades planejadas para o produto final. É importante distinguir os **casos de uso de negócio** (descritos nesta seção) dos **casos de uso de aplicação** (classes do diagrama de classes). Os casos de uso de negócio — como UC004 (Emitir Certificado Individual) e UC002 (Gerenciar Templates) — representam *o que* o sistema faz do ponto de vista do usuário. Já os casos de uso de aplicação — como `EmitCertificateUseCase`, `GeneratePDFUseCase`, `VerifyCertificateUseCase` e `GetCertificateUseCase` — são classes da camada de aplicação da arquitetura DDD que implementam *como* o fluxo de negócio é orquestrado internamente, com granularidade mais fina. No MVP, um caso de uso de negócio pode ser implementado por um ou mais casos de uso de aplicação: por exemplo, UC004 (Emitir Certificado Individual) é orquestrado por `EmitCertificateUseCase` (validação e persistência) e `GeneratePDF UseCase` (geração do arquivo PDF).

As funcionalidades implementadas no escopo do MVP compreendem os casos de uso de negócio UC002 (Gerenciar Templates), UC004 (Emitir Certificado Individual), UC008 (Validar Certificado) e UC014 (Visualizar Dashboard). Os demais casos de uso — UC001 (Gerenciar Usuários), UC003 (Gerenciar Cursos), UC005 (Emitir em Lote), UC006 (Cadastrar Participante), UC007 (Importar Participantes), UC009 (Buscar Participantes), UC010 (Visualizar Histórico), UC011 (Gerenciar Organizações), UC012 (Configurar Sistema), UC013 (Abrir Chamado) e UC015 (Consultar Logs) — estão previstos para versões futuras, mas foram modelados no diagrama completo para preservar a visão de longo prazo do sistema.

Figura 1 – Diagrama de casos de uso do sistema de certificados



Fonte: Elaborado pelo Autor (2026).

As descrições textuais a seguir detalham cada caso de uso identificado no diagrama da Figura 1, seguindo o formato padronizado proposto por Krämer [38].

Quadro 3 – Caso de uso UC001 – Gerenciar Usuários

UC001	**Gerenciar Usuários**	
Dependências	RF001, RF003, RF004, RF005, RF006	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve permitir que o administrador gerencie os usuários da plataforma, incluindo criação, edição e desativação de contas, bem como a definição de perfis de acesso.	
Ator(es)	Administrador	
Pré-condições	**Item**	**Condição**
	1	Administrador autenticado no sistema.
	2	Administrador possui permissão de gerenciamento de usuários.
Sequência Ordinária	**Passo**	**Ação**
	1	O administrador acessa a seção de usuários.
	2	O sistema exibe a lista de usuários cadastrados.
	3	O administrador seleciona criar, editar ou desativar um usuário.
	4	O administrador preenche os dados do usuário (nome, e-mail e senha).
	5	O administrador define os perfis de acesso e permissões do usuário.
	6	O sistema valida os dados informados.
	7	O sistema persiste a alteração no banco de dados.
	8	O sistema exibe confirmação da operação.
Pós-condições	**Item**	**Condição**
	1	Usuário criado, editado ou desativado com sucesso.
	2	Alterações refletidas na lista de usuários.
Exceções	**Passo**	**Condição**
	3	Se o e-mail informado já existir, o sistema rejeita o cadastro e exibe mensagem de erro.
Referências	RF001, RF003, RF004, RF005, RF006	
Comentários	Nenhum.	

Fonte: Elaborado pelo Autor (2026).

Quadro 4 – Caso de uso UC002 – Gerenciar Templates

UC002	**Gerenciar Templates**	
Dependências	RF008, RF009, RF010, RF012, RF013	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve permitir que o administrador gerencie templates de certificados, incluindo upload, personalização visual e configuração de campos dinâmicos.	
Ator(es)	Administrador	
Pré-condições	**Item**	**Condição**
	1	Administrador autenticado no sistema.
Sequência Ordinária	**Passo**	**Ação**
	1	O administrador acessa a seção de templates.
	2	O sistema exibe a galeria de templates existentes.
	3	O administrador seleciona criar um novo template ou editar um existente.
	4	O administrador faz upload de um novo arquivo PPTX ou PDF (ou mantém o template existente).
	5	O sistema valida o formato do arquivo e exibe preview.
	6	O administrador configura campos dinâmicos e personaliza cores e fontes.
	7	O sistema salva o template e retorna à galeria.
Pós-condições	**Item**	**Condição**
	1	Template salvo no sistema.
	2	Template disponível para uso na emissão de certificados.
Exceções	**Passo**	**Condição**
	3	Se o arquivo enviado não for PPTX ou PDF, o sistema rejeita e exibe mensagem de formato inválido.
Referências	RF008, RF009, RF010, RF012, RF013	
Comentários	Nenhum.	

Fonte: Elaborado pelo Autor (2026).

Quadro 5 – Caso de uso UC003 – Gerenciar Cursos

UC003	**Gerenciar Cursos**	
Dependências	RF048, RF049, RF050, RF051, RF052, RF055	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve permitir que o administrador cadastre e gerencie cursos, incluindo nome, carga horária, datas, local e instrutor.	
Ator(es)	Administrador	
Pré-condições	**Item**	**Condição**
	1	Administrador autenticado no sistema.
Sequência Ordinária	**Passo**	**Ação**
	1	O administrador acessa a seção de cursos.
	2	O sistema exibe a lista de cursos cadastrados.
	3	O administrador preenche nome, carga horária, datas e instrutor.
	4	O sistema valida os dados obrigatórios.
	5	O sistema salva o curso no banco de dados.
	6	O sistema exibe confirmação.
Pós-condições	**Item**	**Condição**
	1	Curso cadastrado no sistema.
	2	Curso disponível para vinculação com certificados.
Exceções	**Passo**	**Condição**
	4	Se dados obrigatórios estiverem ausentes, o sistema bloqueia o salvamento e exibe os campos pendentes.
Referências	RF048, RF049, RF050, RF051, RF052, RF055	
Comentários	Nenhum.	

Fonte: Elaborado pelo Autor (2026).

Quadro 6 – Caso de uso UC004 – Emitir Certificado Individual

UC004	**Emitir Certificado Individual**	
Dependências	RF015, RF016, RF023, RF024, RF025, RF028, RF030, RF054, RF034, RF035	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve permitir que o administrador ou colaborador emita um certificado individual, selecionando template, curso e participante, e gerando PDF com código único e *QR Code*.	
Ator(es)	Administrador, Colaborador	
Pré-condições	**Item**	**Condição**
	1	Participante já cadastrado no sistema.
	2	Template disponível para uso.
	3	Usuário autenticado com perfil de colaborador ou administrador.
Sequência Ordinária	**Passo**	**Ação**
	1	O colaborador acessa a seção de emissão.
	2	O sistema exibe formulário com campos para template, curso e participante.
	3	O colaborador seleciona o template e o curso.
	4	O colaborador seleciona ou cadastra o participante.
	5	O sistema valida o vínculo do participante ao curso.
	6	O sistema gera o PDF preenchendo os campos dinâmicos.
	7	O sistema gera código único e insere *QR Code* no PDF.
	8	O sistema registra data, hora e responsável pela emissão.
	9	O sistema exibe o certificado gerado com opção de download.
	10	O sistema envia o certificado por e-mail para o participante (quando configurado).
Pós-condições	**Item**	**Condição**
	1	Certificado gerado em PDF.
	2	Código único registrado no banco de dados.
	3	Certificado disponível para validação pública.
Exceções	**Passo**	**Condição**
	5	Se o participante não estiver vinculado ao curso, o sistema bloqueia a emissão e exibe alerta.
	10	Se o envio por e-mail falhar, o sistema registra a falha e agenda reenvio automático.
Referências	RF015, RF016, RF023, RF024, RF025, RF028, RF030, RF054, RF034, RF035	
Comentários	Nenhum.	

Fonte: Elaborado pelo Autor (2026).

Quadro 7 – Caso de uso UC005 – Emitir Certificados em Lote

UC005	**Emitir Certificados em Lote**	
Dependências	RF017, RF020, RF021, RF022	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve permitir que o administrador ou colaborador emita certificados em lote a partir de uma lista de participantes, processando de forma assíncrona por fila.	
Ator(es)	Administrador, Colaborador	
Pré-condições	**Item**	**Condição**
	1	Administrador ou colaborador autenticado no sistema.
	2	Template de certificado cadastrado.
	3	Arquivo com lista de participantes disponível.
Sequência Ordinária	**Passo**	**Ação**
	1	O colaborador acessa a seção de emissão em lote.
	2	O sistema exibe formulário para seleção de template e arquivo.
	3	O colaborador seleciona o template e envia o arquivo.
	4	O sistema valida os dados do arquivo.
	5	O sistema enfileira as emissões para processamento.
	6	O sistema processa cada item da fila, gerando PDF e código único.
	7	O sistema notifica o colaborador ao término do processamento.
Pós-condições	**Item**	**Condição**
	1	Certificados do lote gerados e registrados.
	2	Colaborador notificado sobre o resultado.
Exceções	**Passo**	**Condição**
	4	Se o arquivo tiver formato inválido, o sistema rejeita e exibe mensagem.
	6	Se alguma linha contiver dados inválidos, o sistema ignora e reporta ao final.
Referências	RF017, RF020, RF021, RF022	
Comentários	Nenhum.	

Fonte: Elaborado pelo Autor (2026).

Quadro 8 – Caso de uso UC006 – Cadastrar Participante

UC006	**Cadastrar Participante**	
Dependências	RF015, RF056, RF057, RF060	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve permitir que o administrador ou colaborador cadastre participantes individualmente, informando nome, e-mail e CPF opcional.	
Ator(es)	Administrador, Colaborador	
Pré-condições	**Item**	**Condição**
	1	Administrador ou colaborador autenticado no sistema.
Sequência Ordinária	**Passo**	**Ação**
	1	O colaborador acessa a seção de participantes.
	2	O sistema exibe formulário com campos de nome, e-mail e CPF.
	3	O colaborador preenche os dados e confirma.
	4	O sistema valida o formato do e-mail.
	5	O sistema salva o participante no banco de dados.
	6	O sistema exibe confirmação.
Pós-condições	**Item**	**Condição**
	1	Participante cadastrado no sistema.
	2	Participante disponível para emissão de certificados.
Exceções	**Passo**	**Condição**
	4	Se o e-mail já estiver cadastrado, o sistema alerta sobre registro existente.
	4	Se o formato do e-mail for inválido, o sistema rejeita e exibe mensagem.
Referências	RF015, RF056, RF057, RF060	
Comentários	Nenhum.	

Fonte: Elaborado pelo Autor (2026).

Quadro 9 – Caso de uso UC007 – Importar Participantes

UC007	**Importar Participantes**	
Dependências	RF017, RF018, RF058	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve permitir que o administrador ou colaborador importe participantes em lote a partir de arquivos CSV ou Excel.	
Ator(es)	Administrador, Colaborador	
Pré-condições	**Item**	**Condição**
	1	Administrador ou colaborador autenticado no sistema.
	2	Arquivo CSV ou Excel disponível.
Sequência Ordinária	**Passo**	**Ação**
	1	O colaborador acessa a opção de importar participantes.
	2	O sistema exibe formulário para upload de arquivo.
	3	O colaborador seleciona e envia o arquivo.
	4	O sistema valida o cabeçalho e os dados das linhas.
	5	O sistema importa os registros válidos.
	6	O sistema exibe resumo com quantidade de registros importados e rejeitados.
Pós-condições	**Item**	**Condição**
	1	Participantes válidos importados e disponíveis para emissão.
	2	Relatório de erros gerado para registros rejeitados.
Exceções	**Passo**	**Condição**
	4	Se o formato do arquivo não for CSV ou XLSX, o sistema rejeita e exibe mensagem.
	4	Se alguma linha contiver dados inconsistentes, o sistema ignora e lista ao final.
Referências	RF017, RF018, RF058	
Comentários	Nenhum.	

Fonte: Elaborado pelo Autor (2026).

Quadro 10 – Caso de uso UC008 – Validar Certificado

UC008	**Validar Certificado**	
Dependências	RF036, RF037, RF026, RF038	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve disponibilizar um portal público onde qualquer visitante possa validar um certificado informando o código único ou escaneando o *QR Code*.	
Ator(es)	Visitante	
Pré-condições	**Item**	**Condição**
	1	Nenhuma (portal público, sem autenticação).
Sequência Ordinária	**Passo**	**Ação**
	1	O visitante acessa o portal público de validação.
	2	O sistema exibe campo para inserção do código único.
	3	O visitante insere o código presente no certificado.
	4	O sistema consulta o código no banco de dados.
	5	O sistema exibe nome, curso, data e status de autenticidade.
Pós-condições	**Item**	**Condição**
	1	Informações do certificado exibidas ao visitante.
Exceções	**Passo**	**Condição**
	3	Se o visitante escanear o *QR Code*, o sistema redireciona com o código preenchido automaticamente.
	4	Se o código não existir, o sistema exibe "Certificado não encontrado".
Referências	RF036, RF037, RF026, RF038	
Comentários	A validação via *QR Code* elimina a necessidade de digitação manual do código.	

Fonte: Elaborado pelo Autor (2026).

Quadro 11 – Caso de uso UC009 – Buscar Participantes

UC009	**Buscar Participantes**	
Dependências	RF060	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve permitir que o administrador ou colaborador busque participantes por nome, e-mail ou CPF, exibindo os resultados de forma rápida e paginada.	
Ator(es)	Administrador, Colaborador	
Pré-condições	**Item**	**Condição**
	1	Administrador ou colaborador autenticado no sistema.
Sequência Ordinária	**Passo**	**Ação**
	1	O colaborador acessa a seção de participantes.
	2	O sistema exibe campo de busca e lista completa de participantes.
	3	O colaborador digita nome, e-mail ou CPF e confirma.
	4	O sistema filtra os registros correspondentes.
	5	O sistema exibe os resultados em menos de 2 segundos.
Pós-condições	**Item**	**Condição**
	1	Lista de participantes filtrada exibida ao colaborador.
Exceções	**Passo**	**Condição**
	4	Se nenhum registro corresponder à busca, o sistema exibe mensagem "Nenhum participante encontrado".
	4	Se a busca estiver vazia, o sistema retorna a lista completa.
Referências	RF060	
Comentários	Nenhum.	

Fonte: Elaborado pelo Autor (2026).

Quadro 12 – Caso de uso UC010 – Visualizar Histórico de Certificados

UC010	**Visualizar Histórico de Certificados**	
Dependências	RF059, RF060	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve exibir o histórico completo de certificados emitidos para um participante específico, incluindo data de emissão, template utilizado e status de validade.	
Ator(es)	Administrador, Colaborador	
Pré-condições	**Item**	**Condição**
	1	Administrador ou colaborador autenticado no sistema.
	2	Participante localizado via busca.
Sequência Ordinária	**Passo**	**Ação**
	1	O colaborador acessa a página do participante.
	2	O sistema exibe os dados cadastrais do participante.
	3	O colaborador seleciona a opção "Histórico de Certificados".
	4	O sistema consulta todos os certificados associados ao participante.
	5	O sistema exibe lista com data, template, curso e status de cada certificado.
Pós-condições	**Item**	**Condição**
	1	Histórico de certificados exibido ao colaborador.
Exceções	**Passo**	**Condição**
	4	Se o participante não possuir certificados, o sistema exibe "Nenhum certificado encontrado".
Referências	RF059, RF060	
Comentários	Estende o caso de uso Buscar Participantes.	

Fonte: Elaborado pelo Autor (2026).

Quadro 13 – Caso de uso UC011 – Gerenciar Organizações

UC011	**Gerenciar Organizações**	
Dependências	RF061, RF062, RF063, RF064	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve permitir que o administrador cadastre e gerencie organizações (multi-tenant), incluindo CNPJ, razão social, endereço e contato, bem como a associação de usuários e templates a cada organização.	
Ator(es)	Administrador	
Pré-condições	**Item**	**Condição**
	1	Administrador autenticado no sistema.
Sequência Ordinária	**Passo**	**Ação**
	1	O administrador acessa a seção de organizações.
	2	O sistema exibe a lista de organizações cadastradas.
	3	O administrador seleciona criar, editar ou desativar uma organização.
	4	O administrador preenche CNPJ, razão social, endereço e contato.
	5	O sistema valida o CNPJ e os dados obrigatórios.
	6	O sistema persiste a organização no banco de dados.
	7	O sistema exibe confirmação da operação.
Pós-condições	**Item**	**Condição**
	1	Organização cadastrada, editada ou desativada com sucesso.
	2	Organização disponível para associação de usuários e templates.
Exceções	**Passo**	**Condição**
	5	Se o CNPJ já estiver cadastrado, o sistema rejeita e exibe mensagem de duplicidade.
	5	Se o CNPJ for inválido, o sistema rejeita e exibe mensagem.
Referências	RF061, RF062, RF063, RF064	
Comentários	Nenhum.	

Fonte: Elaborado pelo Autor (2026).

Quadro 14 – Caso de uso UC012 – Configurar Sistema

UC012	**Configurar Sistema**	
Dependências	RF065, RF066, RF067	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve permitir que o administrador configure parâmetros gerais do sistema, incluindo personalização de mensagens padrão, ativação de envio automático e regras de reemissão de certificados.	
Ator(es)	Administrador	
Pré-condições	**Item**	**Condição**
	1	Administrador autenticado no sistema.
Sequência Ordinária	**Passo**	**Ação**
	1	O administrador acessa a seção de configurações.
	2	O sistema exibe os painéis de configuração disponíveis.
	3	O administrador personaliza mensagens padrão de e-mail.
	4	O administrador ativa ou desativa o envio automático.
	5	O administrador configura limites e regras de reemissão.
	6	O sistema valida e persiste as configurações.
	7	O sistema exibe confirmação da operação.
Pós-condições	**Item**	**Condição**
	1	Configurações do sistema atualizadas.
	2	Novas regras e mensagens aplicadas às próximas operações.
Exceções	**Passo**	**Condição**
	6	Se algum valor obrigatório estiver ausente, o sistema bloqueia o salvamento e exibe os campos pendentes.
Referências	RF065, RF066, RF067	
Comentários	Nenhum.	

Fonte: Elaborado pelo Autor (2026).

Quadro 15 – Caso de uso UC013 – Abrir Chamado de Suporte

UC013	**Abrir Chamado de Suporte**	
Dependências	RF076	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve permitir que o administrador ou colaborador abra chamados de suporte técnico, informando descrição do problema, prioridade e dados de contato.	
Ator(es)	Administrador, Colaborador	
Pré-condições	**Item**	**Condição**
	1	Administrador ou colaborador autenticado no sistema.
Sequência Ordinária	**Passo**	**Ação**
	1	O administrador acessa a seção de suporte.
	2	O sistema exibe o formulário de abertura de chamado.
	3	O administrador preenche descrição, prioridade e contato.
	4	O sistema valida os dados obrigatórios.
	5	O sistema registra o chamado no banco de dados.
	6	O sistema exibe confirmação com número do chamado.
Pós-condições	**Item**	**Condição**
	1	Chamado registrado no sistema.
	2	Chamado disponível para acompanhamento pela equipe de suporte.
Exceções	**Passo**	**Condição**
	4	Se campos obrigatórios estiverem ausentes, o sistema bloqueia o envio e exibe os campos pendentes.
Referências	RF076	
Comentários	Nenhum.	

Fonte: Elaborado pelo Autor (2026).

Quadro 16 – Caso de uso UC014 – Visualizar Dashboard

UC014	**Visualizar Dashboard**	
Dependências	RF041, RF042, RF043	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve exibir um dashboard com a lista de certificados emitidos, filtros por curso, período, status e instrutor e estatísticas mensais de emissão.	
Ator(es)	Administrador, Colaborador	
Pré-condições	**Item**	**Condição**
	1	Usuário autenticado no sistema.
Sequência Ordinária	**Passo**	**Ação**
	1	O usuário acessa a seção de dashboard.
	2	O sistema exibe a lista de certificados emitidos com filtros disponíveis.
	3	O usuário aplica filtros por curso, período, status ou instrutor.
	4	O sistema exibe os resultados filtrados.
	5	O sistema exibe estatísticas mensais de emissão.
Pós-condições	**Item**	**Condição**
	1	Dashboard exibido com dados atualizados.
Exceções	**Passo**	**Condição**
	3	Se nenhum certificado corresponder aos filtros, o sistema exibe "Nenhum resultado encontrado".
Referências	RF041, RF042, RF043	
Comentários	Nenhum.	

Fonte: Elaborado pelo Autor (2026).

Quadro 17 – Caso de uso UC015 – Consultar Logs

UC015	**Consultar Logs**	
Dependências	RF073, RF074, RF075	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve permitir que o administrador consulte logs de autenticação e eventos críticos do sistema, com opção de exportação dos registros.	
Ator(es)	Administrador	
Pré-condições	**Item**	**Condição**
	1	Administrador autenticado no sistema.
Sequência Ordinária	**Passo**	**Ação**
	1	O administrador acessa a seção de logs.
	2	O sistema exibe os logs de autenticação e eventos críticos.
	3	O administrador aplica filtros por período ou tipo de evento.
	4	O sistema exibe os registros filtrados.
	5	O administrador solicita a exportação dos logs.
	6	O sistema exporta os registros no formato selecionado.
Pós-condições	**Item**	**Condição**
	1	Logs consultados e/ou exportados com sucesso.
Exceções	**Passo**	**Condição**
	3	Se não houver registros no período, o sistema exibe "Nenhum log encontrado".
Referências	RF073, RF074, RF075	
Comentários	Nenhum.	

Fonte: Elaborado pelo Autor (2026).

3.5 DIAGRAMAS DE CLASSE

O diagrama de classes da Figura 2 modela as entidades implementadas no MVP do sistema de certificados, abrangendo as camadas de domínio, aplicação, infraestrutura e apresentação. A arquitetura segue o padrão *Domain-Driven Design* (DDD) com separação clara entre as responsabilidades de cada camada.

A hierarquia `Pessoa` (abstrata) $\rightarrow$ `Usuario` / `Participante` modela os atores do sistema no nível de domínio, mas não possui tabelas correspondentes no banco de dados do MVP — é uma abstração conceitual que será concretizada em versões futuras via estratégia *joined-table*. No centro do modelo está a classe `Certificate`, que representa o certificado emitido. Seus atributos incluem `id`, `recipientName`, `recipientCPF`, `courseName`, `courseHours`, `verificationCode`, `validityDate`, `templateId`, `issuedAt`, `createdAt` e `updatedAt`. A classe expõe os métodos `isValid()`, `updateValidity(newDate)`, `update(details)` e `toJSON()`. O código de verificação único é modelado como uma classe separada `VerificationCode`, que encapsula a lógica de geração (`generate()`), reconstrução a partir de string (`fromString()`), comparação (`equals(other)`) e formatação (`toString()`), com constantes estáticas `CHARSET` e `LENGTH=8`.

A classe `Template` gerencia os modelos de certificado, com atributos `id`, `name`, `description`, `layoutConfig` (`TemplateLayout`), `createdAt` e `updatedAt`. `TemplateLayout` é uma classe separada que define as propriedades visuais do template: `backgroundColor`, `primaryColor`, `borderColor`, `titleFontSize`, `bodyFontSize`, `showLogo` e `showBorder`. A relação entre `Certificate` e `Template` é N:1 (um template pode ser usado por vários certificados), e entre `Template` e `TemplateLayout` é 1:1 (composição).

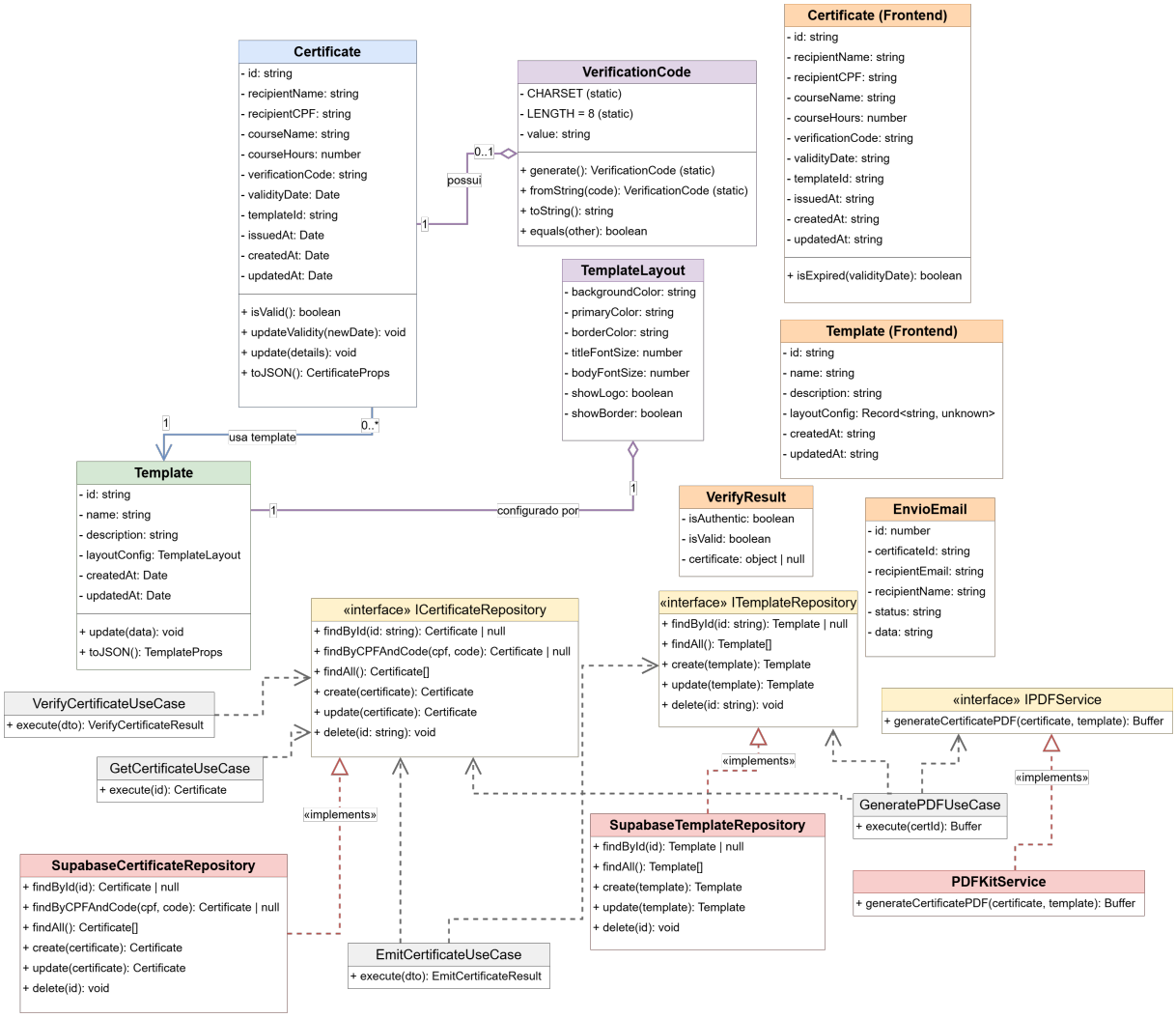
As interfaces de repositório (*repository contracts*) são definidas como `ICertificateRepository` e `ITemplateRepository`, estabelecendo os contratos de persistência com operações CRUD (`findById`, `findAll`, `create`, `update`, `delete`), além de `findByCPFAndCode` no repositório de certificados. A interface `IPDFService` define o contrato para geração de PDF com o método `generateCertificatePDF(certificate,template)`.

A implementação concreta dos repositórios é feita por `SupabaseCertificateRepository` e `SupabaseTemplateRepository`, que utilizam o Supabase como banco de dados. O serviço de PDF é implementado por `PDFKitService`, que gera os arquivos PDF dos certificados. As setas tracejadas com o estereótipo «implements» indicam a realização das interfaces.

Os casos de uso (*use cases*) da camada de aplicação são: `EmitCertificateUseCase` (emissão de certificado), `GeneratePDFUseCase` (geração do PDF), `VerifyCertificateUseCase` (verificação de autenticidade) e `GetCertificateUseCase` (consulta de certificado), cada um expondo um método `execute()` com seus respectivos parâmetros e retornos.

Na camada de apresentação (*frontend*), as classes `Certificate` e `Template` refletem os mesmos atributos do domínio, porém com tipos serializáveis (strings para datas). `VerifyResult` encapsula o resultado da verificação com `isAuthentic`, `isValid` e `certificate`. `EnvioEmail` modela a estrutura de um registro de envio de e-mail com `id`, `certificateId`, `recipientEmail`, `recipientName`, `status` e `data`; no escopo do MVP, essa classe opera exclusivamente no *frontend* com persistência simulada em `localStorage`, sem integração com provedor SMTP nem tabela correspondente no banco de dados.

Figura 2 – Diagrama de Classes — MVP do Sistema de Certificados



Fonte: Elaborado pelo Autor (2026).

3.6 ARQUITETURA DO SISTEMA

A arquitetura do sistema foi projetada para atender aos requisitos de independência entre camadas, testabilidade e facilidade de manutenção. Optou-se por uma abordagem de duas aplicações independentes — frontend e backend — que se comunicam exclusivamente via API REST. Essa separação permite que cada aplicação seja desenvolvida, testada, implantada e escalada de forma independente, sem acoplamento tecnológico entre as camadas de apresentação e lógica de negócio.

O backend adota uma arquitetura em quatro camadas, organizadas pelo princípio de inversão de dependências (do inglês *Dependency Inversion Principle* — DIP), em que camadas externas dependem de camadas internas:

1. **Apresentação (*App*)** — *Route Handlers* do Next.js que expõem os endpoints REST (`/api/certificates`, `/api/templates`, `/api/verify`) e o utilitário `handleApiError`, que converte exceções de domínio e erros de validação Zod em respostas HTTP padronizadas. Esta camada recebe a requisição, faz o *parse* do JSON, aciona a validação do DTO e delega a execução ao *use case*.

2. **Aplicação (*Application*)** — *Use cases* que orquestram o fluxo de negócio (Emitir, Consultar, Verificar e Gerar PDF) e **DTOs** definidos como esquemas Zod (`EmitCertificateDTO`, `UpdateValidityDTO`, `VerifyCertificateDTO`, etc.), que validam os dados de entrada antes de alcançarem o domínio.

3. **Domínio (*Domain*)** — Núcleo da aplicação, sem dependências externas. Contém as **entidades** (`Certificate`, `Template`) com suas regras de negócio encapsuladas, **interfaces de repositório** (`ICertificateRepository`, `ITemplateRepository`), **interfaces de serviço** (`IPDFService`), ***Value Objects*** (`VerificationCode`, com geração e validação de código único) e **erros de domínio** (`CertificateNotFoundError`, `InvalidCertificateDataError`, etc.).

4. **Infraestrutura (*Infrastructure*)** — Implementações concretas das interfaces definidas no domínio: `SupabaseCertificateRepository`, `SupabaseTemplateRepository` e `PDFKitService`, além dos clientes de acesso ao banco PostgreSQL gerenciado pelo Supabase.

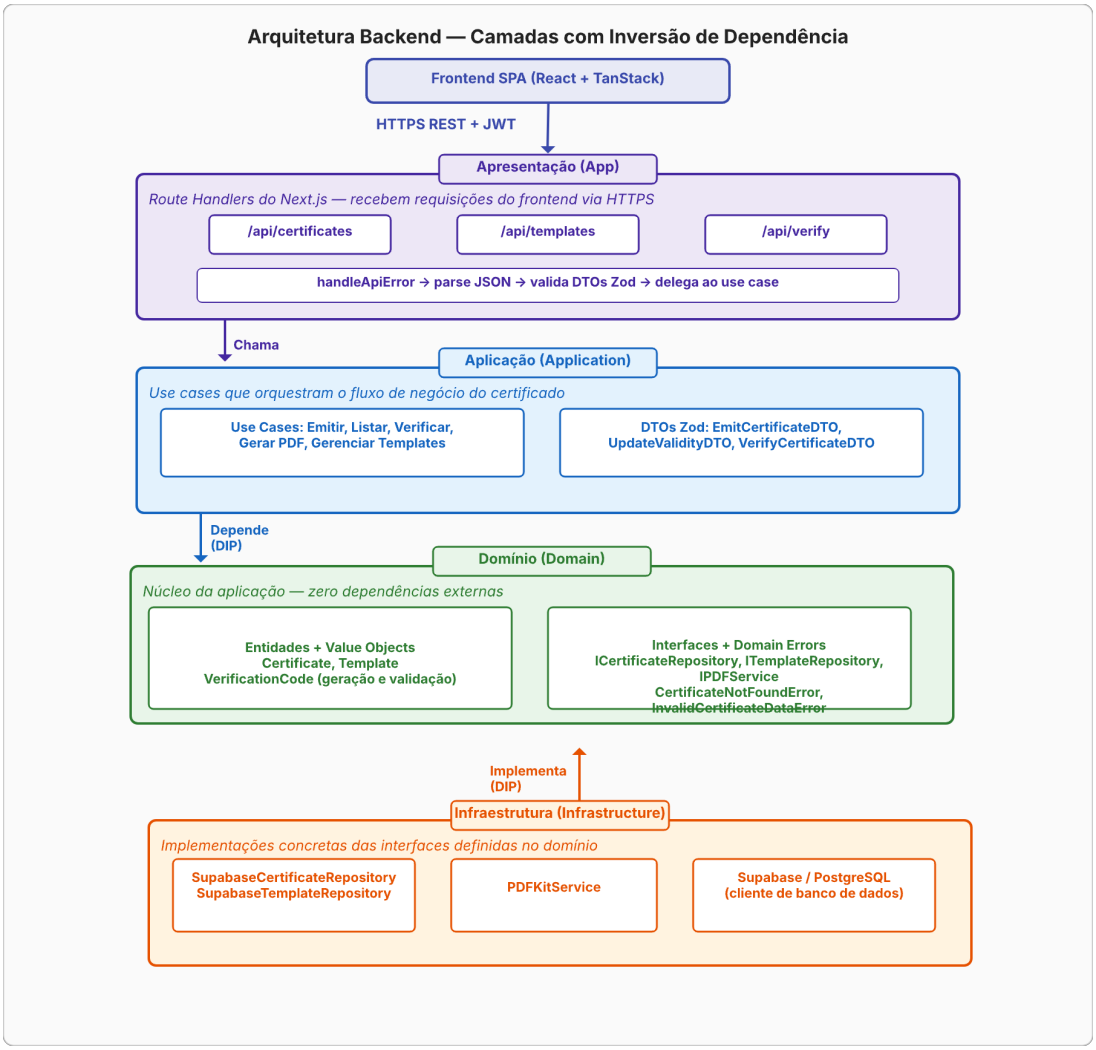
O fluxo de dados percorre as camadas da seguinte forma: uma requisição HTTP chega ao *Route Handler* (Apresentação), que valida o corpo da requisição contra o esquema Zod (Aplicação) e invoca o *use case*. O *use case* orquestra as operações de negócio utilizando entidades e *Value Objects* do Domínio, e persiste os dados por meio das interfaces de repositório — cujas implementações concretas (Infraestrutura) são injetadas via construtor. A resposta segue o caminho inverso até ser retornada ao cliente. Esse isolamento garante que as regras de negócio permaneçam independentes de frameworks, bancos de dados ou serviços externos. A inversão de dependência é obtida por meio de interfaces definidas no domínio e implementadas na infraestrutura — por exemplo, `ICertificateRepository` e `IPDFService` são contratos que o *use case* de emissão consome sem conhecer a implementação concreta. Entre as limitações desta abordagem, destaca-se o aumento do número de arquivos e interfaces, o que pode elevar a complexidade inicial do projeto em comparação com arquiteturas mais simples como *controllers* monolíticos.

O frontend adota uma arquitetura em camadas com React, composta por quatro camadas que separam responsabilidades de forma clara. A **View** (apresentação) contém os componentes React que renderizam a interface do usuário, divididos em páginas (`views/`) e componentes reutilizáveis (`components/`), incluindo os do shadcn/ui. A **ViewModel** (lógica de apresentação) é formada por custom hooks (`useCertificatesViewModel`, `useDashboardViewModel`, etc.) que encapsulam estado (`useState`), efeitos colaterais (`useEffect`) e dados derivados (`useMemo`), expondo um objeto com estado e manipuladores para a camada View. O **Service** (infraestrutura) centraliza a comunicação com o backend: o módulo `api.ts` consome a API REST via `fetch`, e o módulo `envios.ts` gerencia a fila local de envios de e-mail no `localStorage` — armazenando apenas dados operacionais (nome e e-mail do destinatário, *status* da entrega), sem tokens de autenticação ou credenciais. O **Model** (domínio) reúne as interfaces TypeScript que definem as entidades do sistema (`Certificate`, `Template`, `VerifyResult`, `EnvioEmail`). O TanStack Router orquestra a navegação entre as Views, e o Tailwind CSS gerencia a estilização em todas as camadas de apresentação. Essa separação permite testar a lógica de apresentação independentemente da interface, utilizando Jest para validar os ViewModels sem depender de um navegador. Como limitação, o

aumento do número de ViewModels eleva a quantidade de arquivos e hooks, o que pode tornar a navegação entre camadas mais custosa em projetos de grande escala.

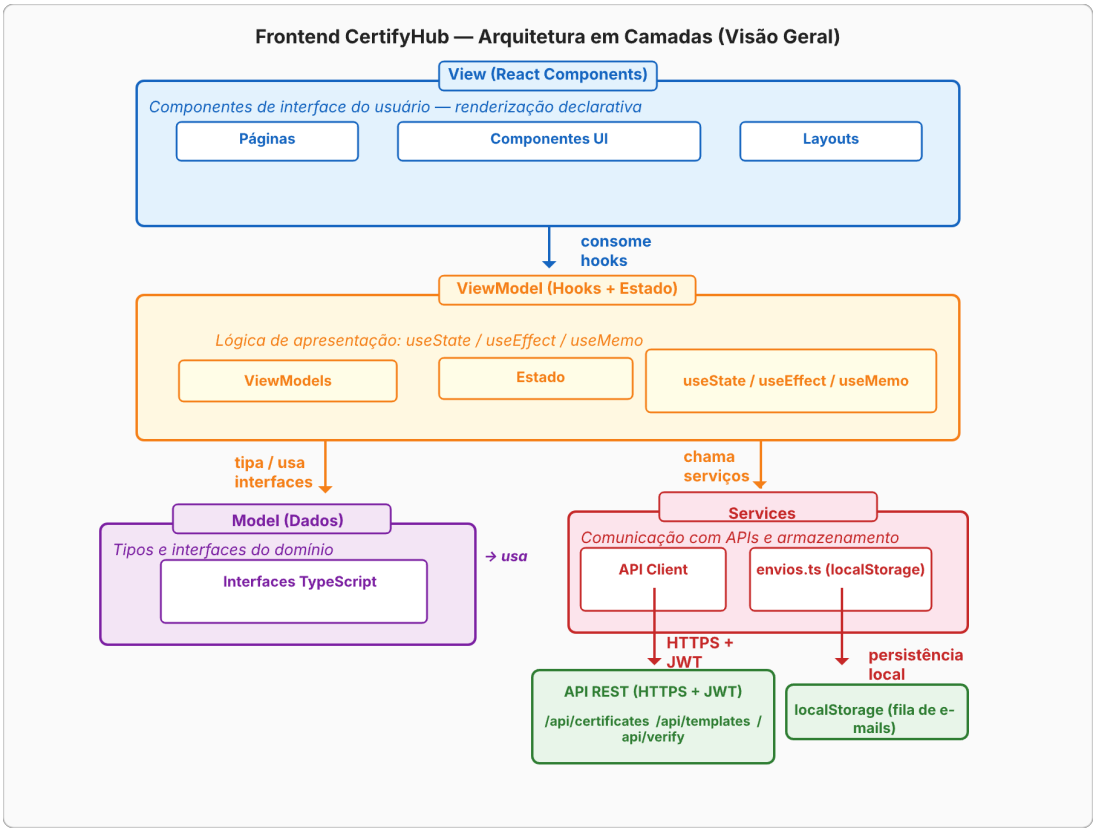
O fluxo de comunicação entre as duas aplicações ocorre da seguinte forma: o frontend SPA, executado no navegador, faz requisições HTTPS aos *Route Handlers* do backend por meio do módulo `api.ts` (camada Service), que encapsula a `fetch` API nativa. As requisições trafegam sobre HTTPS (TLS 1.2), conforme RNF044. No escopo do MVP, a autenticação via JWT com Supabase Auth não foi implementada — o middleware de autenticação existe e foi validado em testes unitários (CT053), mas o fluxo completo de login, gerenciamento de sessão e renovação de tokens está previsto para versões futuras. O backend processa a requisição através de suas quatro camadas — Apresentação, Aplicação, Domínio e Infraestrutura — e retorna a resposta padronizada ao frontend. Esse fluxo ponta a ponta está representado nos diagramas das Figuras 3 e 4.

Figura 3 – Arquitetura do Backend — Quatro camadas (App, Application, Domain, Infrastructure) com DTOs, Value Objects e fluxo de dependências



Fonte: Elaborado pelo Autor (2026).

Figura 4 – Arquitetura do Frontend — Quatro camadas (View, ViewModel, Service, Model) com React



Fonte: Elaborado pelo Autor (2026).

3.7 DIAGRAMA ENTIDADE-RELACIONAMENTO

O Diagrama Entidade-Relacionamento da Figura 5 modela a estrutura do banco de dados implementada no MVP do sistema, composta por duas entidades principais: `CERTIFICADOS` e `TEMPLATES`. A notação adotada é a pé-de-galinha (Engenharia da Informação), onde o símbolo de barra (—) indica o lado "1" e o símbolo de três pontas (–<) indica o lado "N" (muitos). Ambas as entidades utilizam UUID como identificador primário.

A entidade `CERTIFICADOS` armazena os certificados emitidos e contém os seguintes atributos: `id` (PK, UUID), `recipientName` (varchar 200, NOT NULL), `recipientCPF` (varchar 11, NOT NULL, indexado), `courseName` (varchar 300, NOT NULL), `courseHours` (integer, NOT NULL), `verificationCode` (varchar 8, UNIQUE, NOT NULL), `validityDate` (timestamptz, NOT NULL), `templateId` (FK, UUID, NOT NULL), `issuedAt` (timestamptz, NOT NULL), `createdAt` (timestamptz, NOT NULL) e `updatedAt` (timestamptz, NOT NULL).

A entidade `TEMPLATES` gerencia os modelos de certificado disponíveis para emissão, com os atributos: `id` (PK, UUID), `name` (varchar 100, NOT NULL), `description` (text), `layoutConfig` (jsonb), `createdAt` (timestamptz, NOT NULL) e `updatedAt` (timestamptz, NOT NULL).

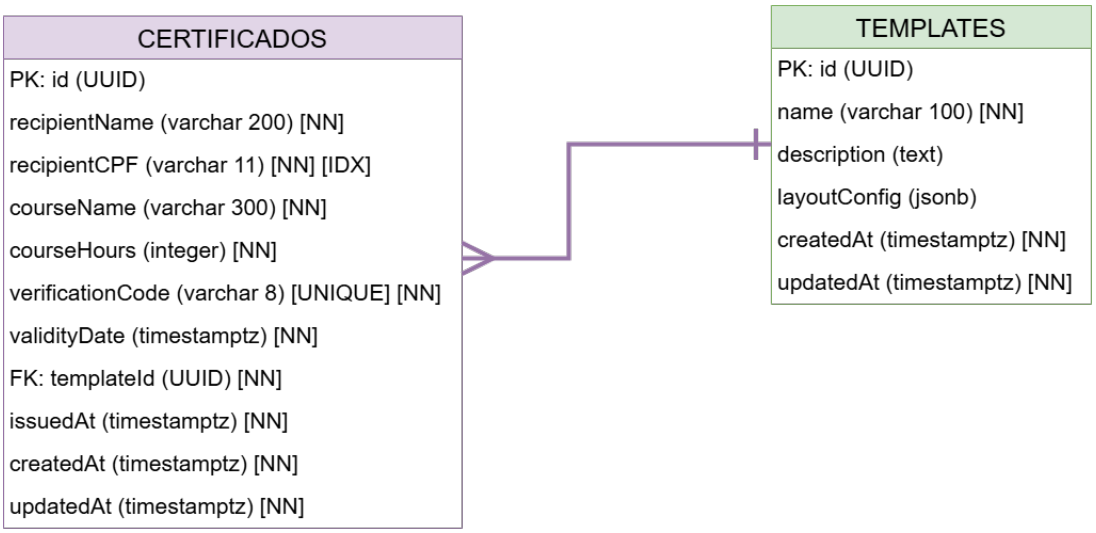
O relacionamento entre as entidades é de 1:N (um template pode ser usado por

vários certificados), materializado pela chave estrangeira `templateId` na entidade `CERTIFICADOS` que referencia `TEMPLATES(id)`. Essa modelagem reflete o escopo reduzido do MVP, que contempla exclusivamente as funcionalidades de emissão de certificados com suporte a templates personalizáveis, sem as entidades de usuários, organizações, participantes, cursos e auditoria que serão incorporadas em versões futuras do sistema.

Vale ressaltar que, no estado atual do sistema, as informações relativas a participantes e cursos são armazenadas de forma denormalizada diretamente na entidade `CERTIFICADOS`, por meio dos atributos `recipientName`, `recipientCPF`, `courseName` e `courseHours`. Essa abordagem elimina a necessidade de chaves estrangeiras para entidades de participantes e cursos, simplificando a implementação do MVP, porém introduz redundância de dados e compromete a integridade referencial. Em versões futuras, as entidades `PARTICIPANTES` e `CURSOS` serão materializadas no banco de dados, permitindo a associação por meio de chaves estrangeiras (`participantId` e `courseId`), eliminando a duplicação de dados e habilitando funcionalidades como histórico de certificados por participante, listagem de certificados por curso e relatórios gerenciais por unidade de treinamento.

O Diagrama de Classes (Figura 2) apresenta a hierarquia `PESSOA` → `USUARIO` / `PARTICIPANTE` como uma generalização conceitual oriunda do modelo de domínio. No banco de dados físico do MVP, essas entidades não foram materializadas — apenas `CERTIFICADOS` e `TEMPLATES` existem como tabelas. A implementação futura dessa herança seguirá a estratégia *joined-table* (ou *table-per-type*), na qual `PESSOA` seria uma tabela concreta com os atributos comuns (`id`, `nome`, `email`) e cada subtipo (`USUARIO`, `PARTICIPANTE`) teria sua própria tabela com chave primária que é também chave estrangeira referenciando `PESSOA(id)` — garantindo integridade referencial sem armazenamento redundante.

Figura 5 – Diagrama Entidade-Relacionamento (DER) — MVP do Banco de Dados



Fonte: Elaborado pelo Autor (2026).

3.8 INTERFACES

No escopo do MVP, foram implementadas sete telas principais que atendem aos requisitos funcionais prioritários (*Must have* e *Should have*). As telas estão organizadas entre páginas públicas (acessíveis sem autenticação) e páginas do painel administrativo (com sidebar de navegação). As figuras a seguir apresentam cada tela com sua respectiva descrição e os requisitos funcionais atendidos.

3.8.1 Tela Inicial

A Figura 6 apresenta a *landing page* do sistema, que exibe o nome "CertifyHub", uma breve descrição do propósito do sistema e dois botões de ação principais: "Começar agora"(direciona ao painel administrativo) e "Verificar um certificado"(direciona à página pública de verificação). A seção inferior apresenta quatro cards com os principais recursos: Templates, Emissão, Envio por e-mail e Verificação. Esta tela não está vinculada a um requisito funcional específico, servindo como porta de entrada para as demais funcionalidades do sistema.

Figura 6 – Tela inicial — *Landing page*



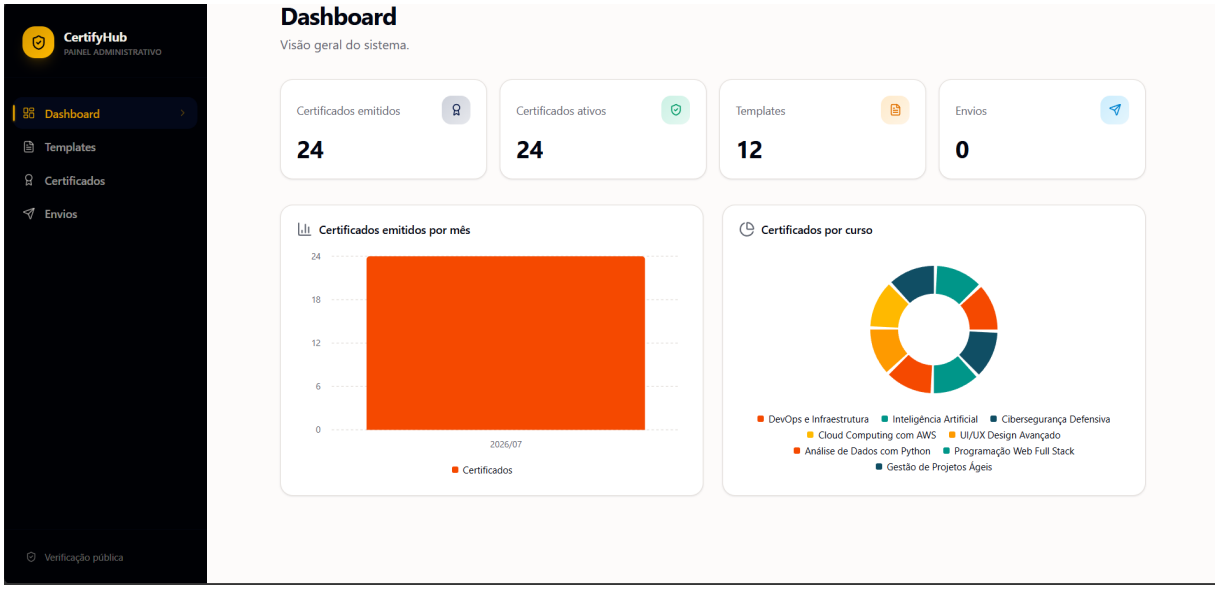
Fonte: Elaborado pelo Autor (2026).

3.8.2 Dashboard

A Figura 7 exibe o painel de controle principal do sistema, acessível a partir do menu lateral. O dashboard apresenta quatro cartões de estatísticas (certificados emitidos, certificados ativos, templates e envios), um gráfico de barras de emissões por mês e um gráfico de pizza com a distribuição de certificados por curso. O cartão "Envios"exibe valor zero porque o módulo de envio automático de e-mails não

foi implementado no escopo do MVP — o registro de envios no *frontend* (`envios.ts`) é apenas uma estrutura simulada em armazenamento local, sem integração com provedores SMTP. O gráfico de pizza contempla oito áreas temáticas: DevOps e Infraestrutura, Inteligência Artificial, Cibersegurança Defensiva, Cloud Computing com AWS, UI/UX Design Avançado, Análise de Dados com Python, Programação Web Full Stack e Gestão de Projetos Ágeis. Esta tela atende aos requisitos **RF041** (exibir lista de certificados), **RF042** (filtros) e **RF043** (estatísticas de emissão).

Figura 7 – Dashboard — Visão geral do sistema

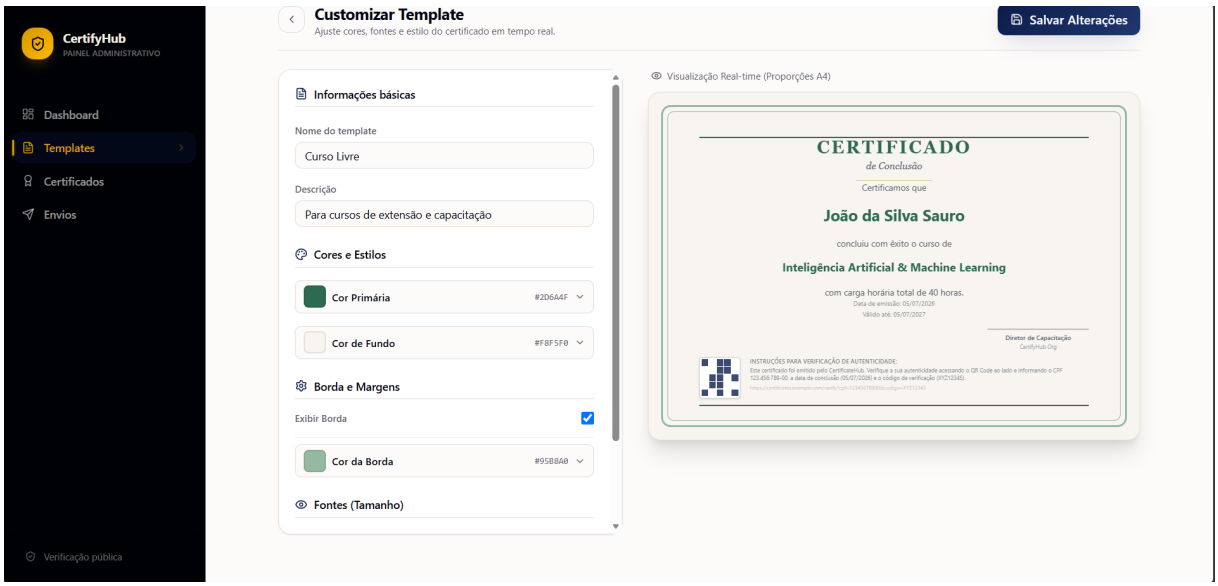


Fonte: Elaborado pelo Autor (2026).

3.8.3 Customização de Templates

A Figura 8 exibe o editor visual de templates do sistema. Nele, o usuário pode personalizar a aparência do certificado ajustando cores (fundo, primária, borda), fontes (tamanho do título e do corpo) e a visibilidade de elementos como borda e logotipo, com pré-visualização em tempo real das alterações. O editor é acessado ao clicar em "Editar"em um template na lista de templates. Esta tela atende aos requisitos **RF009** (personalização), **RF012** (criação de campos dinâmicos) e **RF013** (salvar configurações de mapeamento).

Figura 8 – Editor visual de template com pré-visualização em tempo real

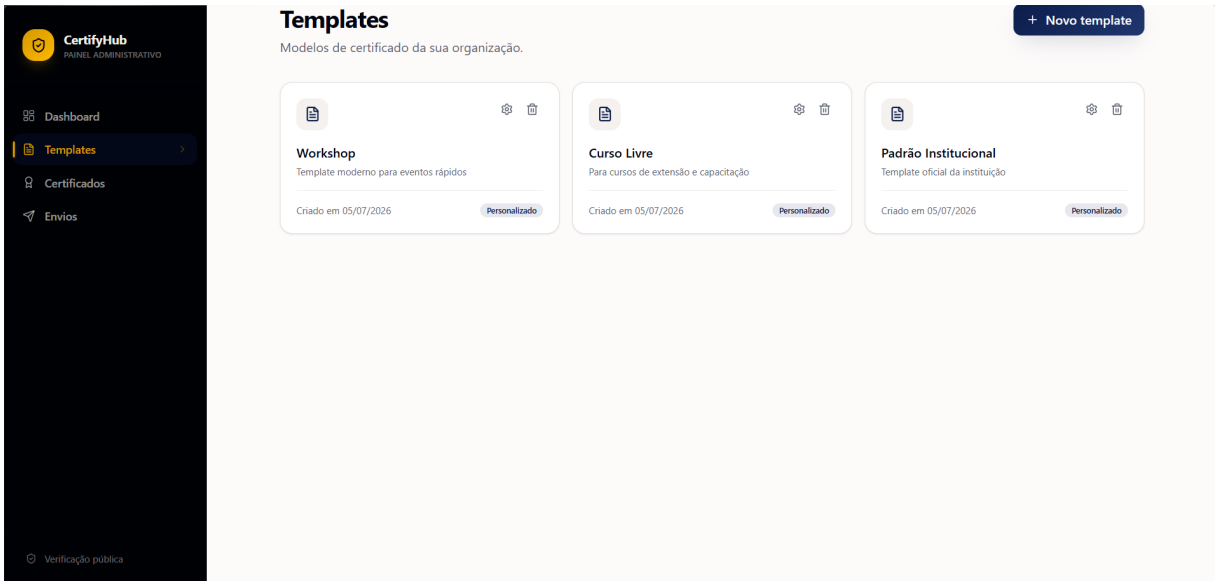


Fonte: Elaborado pelo Autor (2026).

3.8.4 Lista de Templates

A Figura 9 apresenta a página de listagem de templates, onde o usuário pode visualizar os modelos cadastrados, criar novos templates e acessar o editor de personalização. Esta tela atende aos requisitos **RF008** (criação de templates via editor visual) e **RF010** (selecionar template da galeria).

Figura 9 – Lista de templates cadastrados

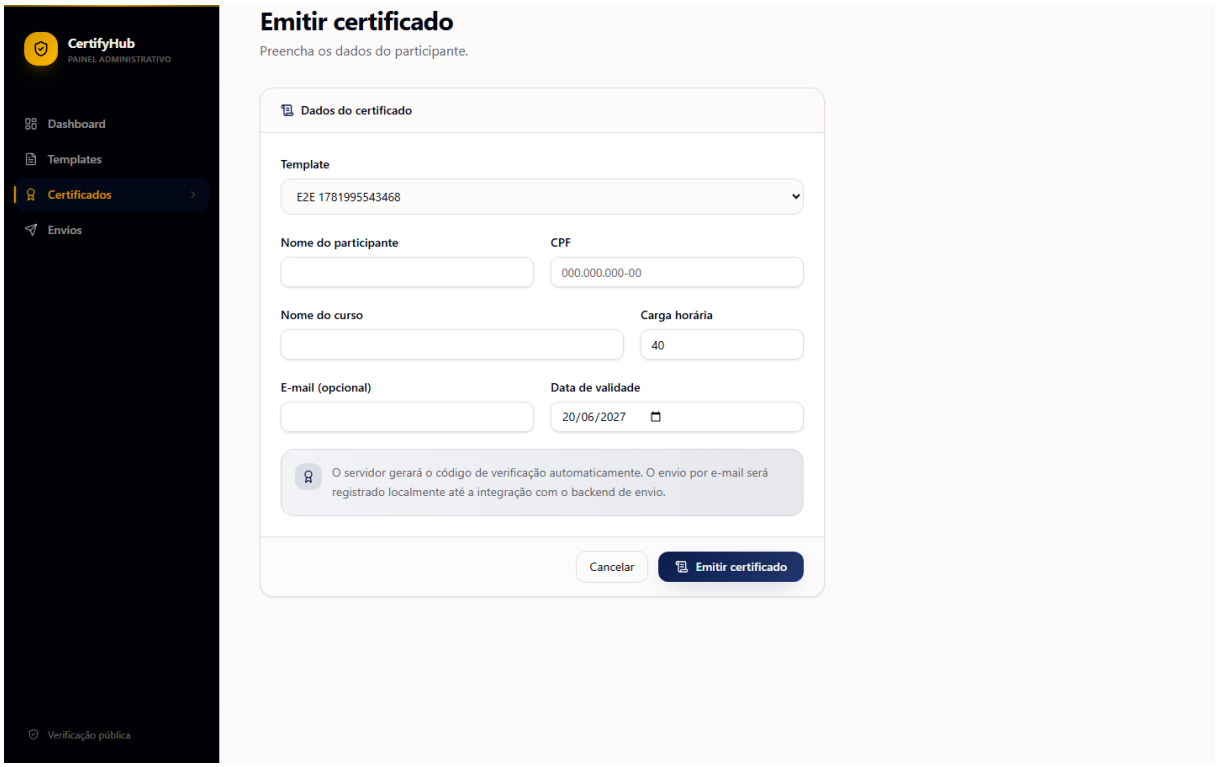


Fonte: Elaborado pelo Autor (2026).

3.8.5 Emissão de Certificados

A Figura 10 mostra o formulário de emissão de certificados. O usuário seleciona um template, preenche os dados do participante (nome, CPF, curso, carga horária, e-mail e data de validade) e confirma a emissão. O sistema gera automaticamente um código de verificação único — o envio automático por e-mail, embora planejado, não foi implementado no escopo do MVP, ficando o download do PDF disponível para distribuição manual. Esta tela atende aos requisitos **RF015** (cadastro de participantes), **RF016** (emissão manual), **RF023** (gerar PDF), **RF024** (preenchimento automático de campos), **RF025** (código único), **RF028** (registro de data e hora) e **RF030** (download individual).

Figura 10 – Formulário de emissão de certificado

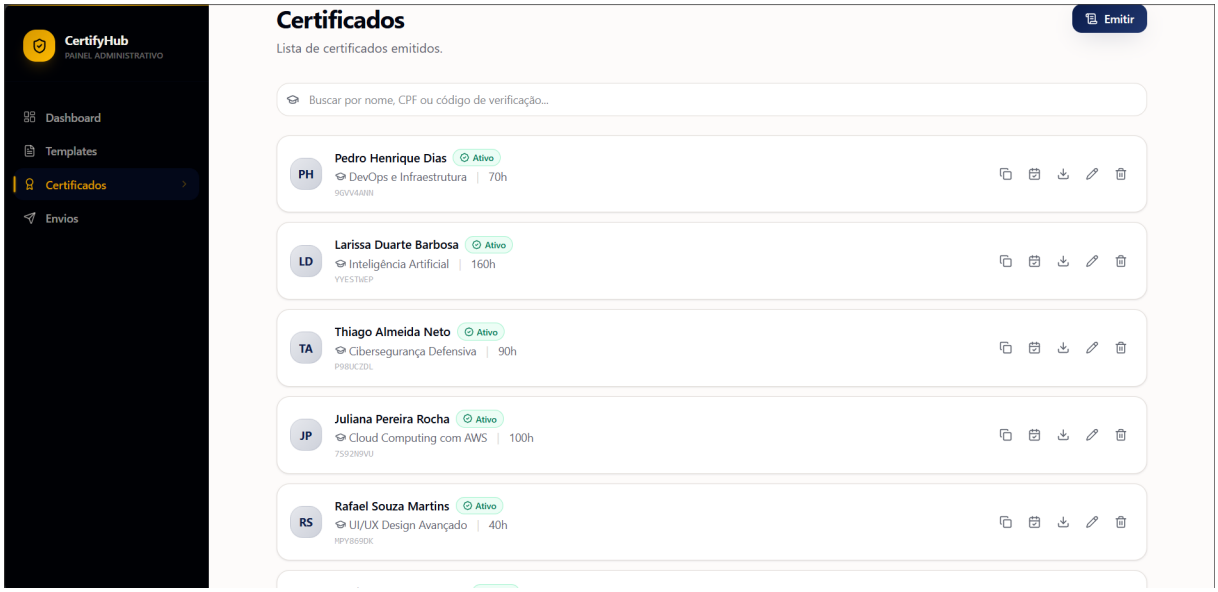


Fonte: Elaborado pelo Autor (2026).

3.8.6 Lista de Certificados

A Figura 11 exibe a lista de certificados emitidos, com campo de busca por nome, CPF ou código de verificação. Cada certificado exibe o status (ativo ou vencido), o curso, a carga horária e opções de ação: copiar código de verificação, editar validade, baixar PDF e editar dados do certificado. Esta tela atende aos requisitos **RF030** (download individual), **RF041** (lista completa de certificados) e **RF042** (filtros por curso, período e status). O requisito **RF029** (reemissão mantendo histórico) não foi implementado nesta versão — a edição de certificados sobrescreve os dados existentes sem gerar novo código de verificação ou preservar a versão anterior.

Figura 11 – Lista de certificados emitidos

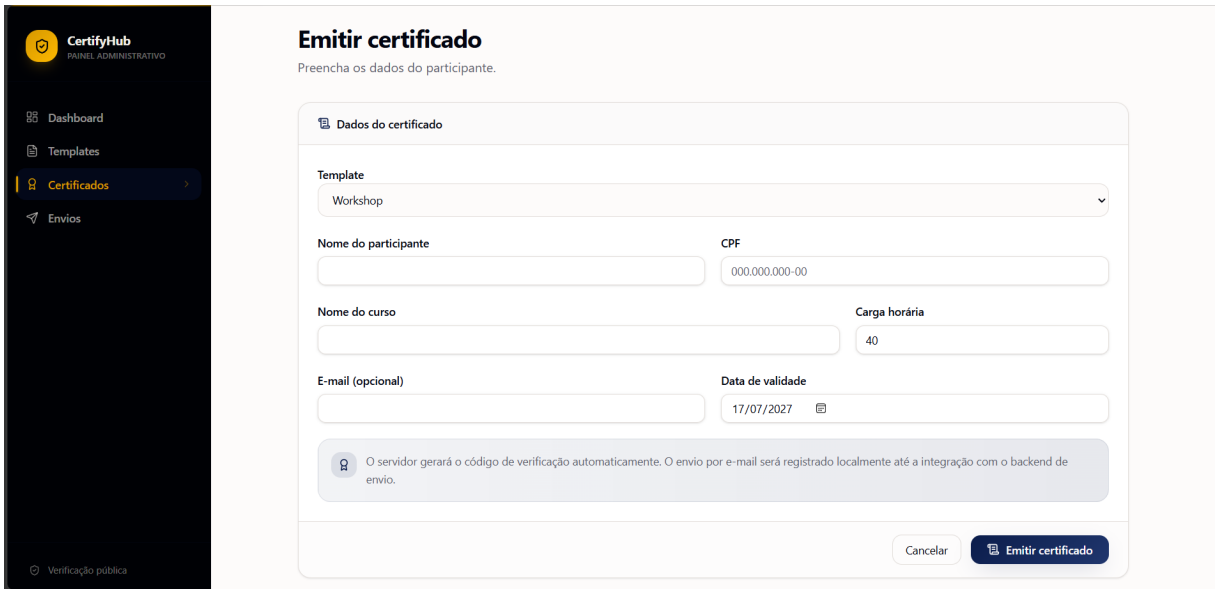


Fonte: Elaborado pelo Autor (2026).

3.8.7 Envios

A Figura 12 apresenta o histórico de envios de e-mail — funcionalidade planejada, mas não implementada no MVP. O frontend registra localmente no `localStorage` os metadados da tentativa (destinatário, participante, data e status), servindo como estrutura preparada para integração futura com um provedor SMTP. Esta tela atende ao requisito **RF034** (registrar status de envio).

Figura 12 – Histórico de envios de e-mail



Fonte: Elaborado pelo Autor (2026).

3.8.8 Certificado Gerado

A Figura 13 apresenta o certificado gerado pelo sistema no formato PDF, em layout paisagem A4, com bordas decorativas, nome do participante, curso, carga horária, CPF, código de verificação, datas de emissão e validade e o *QR Code* de verificação no canto superior direito. O *QR Code* codifica a URL pública de validação (`/verificar?cpf=...&codigo=...`), permitindo que qualquer pessoa autentique o certificado apontando a câmera do celular para o código. Este resultado é produzido pela combinação das bibliotecas PDFKit e qrcode no backend, conforme detalhado na Seção 2.3.2. Esta tela atende aos requisitos **RF023** (gerar PDF), **RF025** (código único), **RF026** (*QR Code*) e **RF030** (download individual).

Figura 13 – Certificado gerado com *QR Code* de verificação

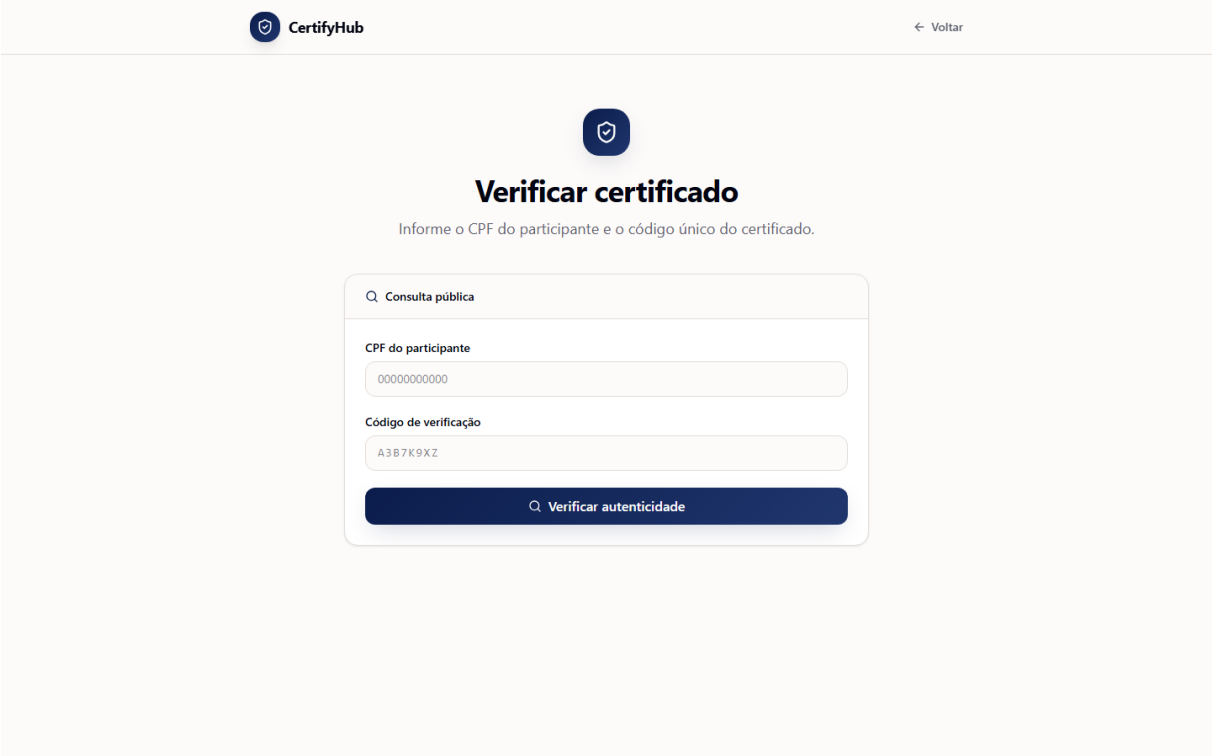


Fonte: Elaborado pelo Autor (2026).

3.8.9 Verificação Pública

A Figura 14 mostra a página pública de verificação de autenticidade de certificados. O usuário informa o CPF do participante e o código de verificação presente no certificado. O sistema consulta a base de dados e retorna o resultado: certificado válido, expirado ou não encontrado. Esta tela atende aos requisitos **RF036** (portal público de validação), **RF037** (validação via código único) e **RF038** (exibir informações do certificado).

Figura 14 – Página pública de verificação de certificados



Fonte: Elaborado pelo Autor (2026).

3.9 MATRIZ DE RASTREABILIDADE

O Quadro 18 apresenta a matriz de rastreabilidade que relaciona os requisitos funcionais implementados no MVP aos casos de uso, às principais classes do diagrama de classes e às tabelas do banco de dados, garantindo a rastreabilidade entre os artefatos do projeto. A página foi disposta em orientação paisagem (rotacionada 90°) para preservar a legibilidade de seu conteúdo, uma vez que possui quatro colunas com textos extensos.

Quadro 18 – Matriz de rastreabilidade entre requisitos funcionais, casos de uso, classes e tabelas implementados no MVP

Módulo / RFs	**Casos de Uso**	**Classes**	**Tabelas**
Templates de Certificados (RF008–RF014)	Gerenciar Templates (UC002), Criar Template, Editar Template, Salvar Template	Template, TemplateLayout	TEMPLATES
Entrada de Dados (RF015–RF022)	Emitir Certificado Individual (UC004)	Certificate, EmitCertificateUseCase, ICertificateRepository	CERTIFICADOS
Geração de Certificados (RF023–RF029)	Emitir Certificado (UC004), Gerar PDF, Obter Certificado	Certificate, VerificationCode, EmitCertificateUseCase, GeneratePDFUseCase, GetCertificateUseCase, ICertificateRepository, ITemplateRepository, IPDFService, SupabaseCertificateRepository, SupabaseTemplateRepository, PDFKitService	CERTIFICADOS
Validação Pública (RF033–RF036)	Validar Certificado (UC008)	VerifyCertificateUseCase, VerifyResult, Certificate (*Frontend*)	CERTIFICADOS
Dashboard e Gestão (RF039–RF042)	Visualizar Dashboard (UC014)	Certificate (*Frontend*), Template (*Frontend*)	CERTIFICADOS, TEMPLATES
Armazenamento de Templates (RF066)	Gerenciar Templates (UC002)	Template, SupabaseTemplateRepository	TEMPLATES

Fonte: Elaborado pelo Autor (2026).

4 SOFTWARE

O capítulo anterior apresentou a modelagem do sistema, definindo requisitos, diagramas e arquitetura. Este capítulo detalha a implementação prática do software, incluindo o ambiente de implantação, exemplos de código relevantes e a estratégia de testes adotada.

4.1 IMPLANTAÇÃO

O sistema é composto por duas aplicações independentes implantadas na plataforma Render, a partir de dois repositórios no GitHub: frontend (`https://github.com/igordev23/certificate-hub`) e backend (`https://github.com/igordev23/Certificate-server`). O sistema é publicado sob subdomínios automáticos fornecidos pela plataforma, garantindo tráfego criptografado via SSL/HTTPS de forma nativa. O frontend pode ser acessado, por exemplo, em `https://certificate-hub.onrender.com`, enquanto a API do backend responde em `https://certificate-server-zd0m.onrender.com`.

4.1.1 Arquitetura e Execução

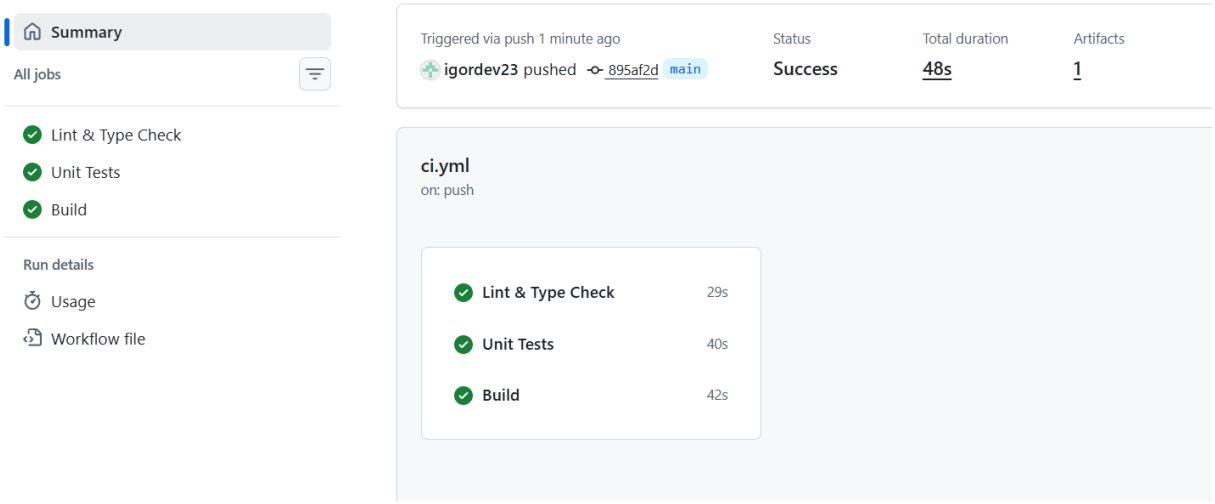
Para a execução e construção do projeto, é estipulado o uso do Node.js, configurado no Render através da variável de ambiente `NODE_VERSION`. A implantação distingue a arquitetura das duas aplicações:

- **Frontend (Static Site):** Trata-se de uma *Single Page Application* (SPA) em Vite. O serviço de *Static Site* serve exclusivamente arquivos estáticos (HTML, CSS e JavaScript gerados na pasta `dist`) diretamente, sem manter um processo de execução ativo (*runtime* Node). O roteamento no lado do cliente é delegado ao TanStack Router via *History API*.
- **Backend (Web Service):** Construído em Next.js (utilizando recursos de servidor Node.js), opera como um *Web Service*, ou seja, mantém um processo vivo que processa requisições HTTP dinâmicas em tempo real. O processo de implantação exige a execução do comando de compilação (`npm run build`) antes da inicialização do servidor com `npm start`.

4.1.2 CI/CD e Pipeline de Implantação

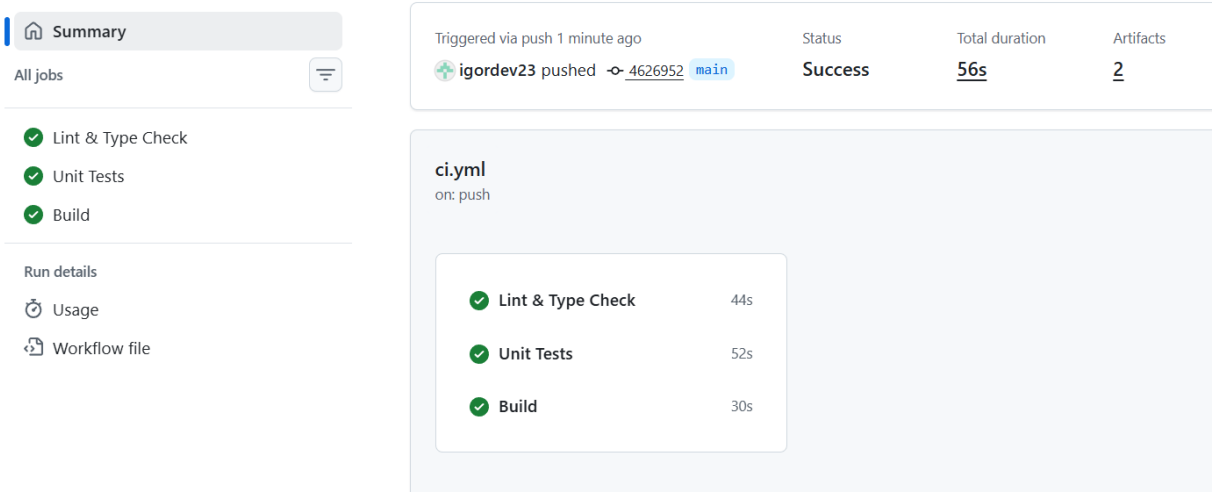
O fluxo de implantação baseia-se em um pipeline de CI/CD (Integração e Entrega Contínuas). A etapa de *Continuous Integration* (CI) é gerenciada pelo GitHub Actions, que executa automaticamente as suítes de testes a cada *push* na branch `main`. Se os testes passarem com sucesso, a etapa de *Continuous Deployment* (CD) é acionada no Render, que compila e publica a nova versão. Em caso de falha nos testes (no GitHub Actions) ou no *build* (no Render), o processo é interrompido, impedindo que código defeituoso chegue a produção. Caso um comportamento inesperado ocorra pós-deploy, a plataforma fornece uma estratégia simples de *rollback*, permitindo reverter imediatamente para o *deploy* anterior funcional com um único clique. A Figura 17 ilustra o fluxo.

Figura 15 – Pipeline de CI do backend no GitHub Actions



Fonte: Elaborado pelo Autor (2026).

Figura 16 – Pipeline de CI do frontend no GitHub Actions



Fonte: Elaborado pelo Autor (2026).

4.1.3 Configuração de Ambiente

Para assegurar o isolamento das configurações sensíveis, utilizam-se variáveis de ambiente não expostas no código. Os modelos de configuração adotados incluem:

- **Frontend:** `VITE_API_URL` (aponta para a URL do Web Service backend).
- **Backend:**
  - `DATABASE_URL` (string de conexão PostgreSQL);
  - `NEXT_PUBLIC_SUPABASE_URL` (URL do projeto Supabase);
  - `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY` (chave pública do Supabase);
  - `NEXT_PUBLIC_FRONTEND_URL` (URL do frontend para configuração de CORS).

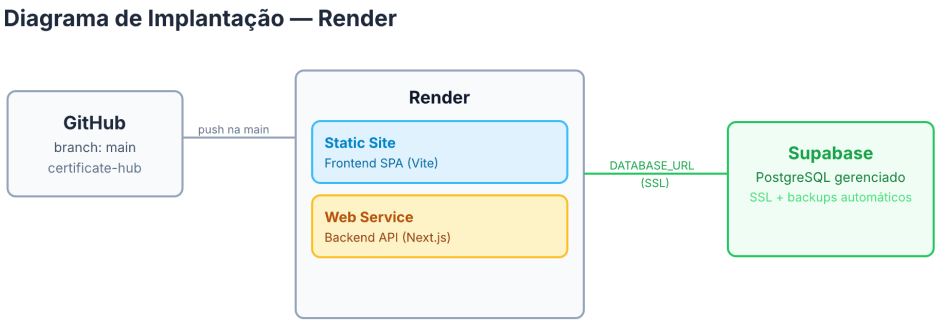
4.1.4 Banco de Dados

O armazenamento é delegado ao PostgreSQL hospedado via Supabase. O banco de dados utiliza *connection pooling* nativo (gerenciamento de conexões otimizado) para alta concorrência e conta com política de backup automático gerida pela plataforma. As atualizações estruturais (*schema*) são aplicadas através de migrações executadas automaticamente na fase de *build*.

4.1.5 Monitoramento e Observabilidade

Por fim, a observabilidade do sistema e o monitoramento (*health checks*) são garantidos pelos logs em tempo real nativos do Render, permitindo diagnóstico ágil de eventuais gargalos de desempenho e verificação de saúde da aplicação.

Figura 17 – Diagrama de Implantação — Fluxo de deploy no Render a partir do GitHub, com frontend como Static Site, backend como Web Service e banco PostgreSQL gerenciado pelo Supabase



Fonte: Elaborado pelo Autor (2026).

4.2 EXEMPLOS DE CÓDIGO

Esta seção apresenta trechos reais do código-fonte do sistema que ilustram as principais funcionalidades implementadas. Os exemplos foram extraídos do backend (CertifyHub Server), construído com Next.js e TypeScript seguindo os princípios de arquitetura limpa.

4.2.1 Entidade de Domínio: Certificate

O Código 1 apresenta a entidade `Certificate`, que encapsula as regras de negócio relacionadas à emissão e validação de certificados. A classe utiliza um construtor privado e métodos estáticos de fábrica (`create` e `fromPersistence`) para garantir que apenas instâncias válidas sejam criadas. O método `isValid()` (linha 13) implementa a regra de negócio que determina se um certificado está dentro do prazo de validade, comparando a data de validade com a data atual.

Código 1 – Entidade Certificate com validações de domínio

```
1 export class Certificate {
2   private constructor(private props: CertificateProps) {}
```

```
3
4     static create(props: Omit<CertificateProps, "createdAt" | "
          updatedAt">): Certificate {
5       Certificate.validate(props);
6       return new Certificate({ ...props, createdAt: new Date(),
            updatedAt: new Date() });
7     }
8
9     static fromPersistence(props: CertificateProps): Certificate {
10      return new Certificate({ ...props, validityDate: new Date(props
            .validityDate) });
11    }
12
13    isValid(): boolean {
14      return this.props.validityDate > new Date();
15    }
16
17    updateValidity(newDate: Date): void {
18      if (newDate <= new Date()) throw new InvalidValidityDateError()
            ;
19      this.props.validityDate = newDate;
20      this.props.updatedAt = new Date();
21    }
22
23    private static validate(props: Omit<CertificateProps, "createdAt"
          | "updatedAt">): void {
24      if (!props.recipientName?.trim()) throw new
            InvalidCertificateDataError("Nome é obrigatório");
25      if (!props.recipientCPF || props.recipientCPF.length !== 11)
26        throw new InvalidCertificateDataError("CPF deve ter 11 dí
              gitos");
27      if (!props.courseHours || props.courseHours <= 0)
28        throw new InvalidCertificateDataError("Carga horária deve ser
               maior que zero");
29      if (!props.templateId) throw new InvalidCertificateDataError("
            Template é obrigatório");
30    }
31  }
```

Fonte: Elaborado pelo Autor (2026).

4.2.2 Route Handler de Emissão

O Código 2 mostra o *Route Handler* responsável pela emissão de certificados (RF023). A requisição `POST/api/certificates/emit` recebe os dados do certificado, valida-os com Zod (`emitCertificateSchema.parse`), instancia os repositórios concretos e delega a execução ao caso de uso `EmitCertificateUseCase`. O uso de injeção de dependência nos repositórios (linhas 6 e 7) permite substituir as implementações sem alterar a lógica de negócio.

Código 2 – Route Handler de emissão de certificado

```
1  export async function POST(request: Request) {
2    try {
```

```
    const body = await request.json();
    const validated = emitCertificateSchema.parse(body);

    const certificateRepo = new SupabaseCertificateRepository();
    const templateRepo = new SupabaseTemplateRepository();
    const useCase = new EmitCertificateUseCase(certificateRepo,
       templateRepo);

    const { certificate } = await useCase.execute(validated);

    return NextResponse.json(
      { data: certificate.toJSON(), message: "Certificado emitido
         com sucesso" },
      { status: 201 }
    );
  } catch (error) {
    return handleApiError(error);
  }
}
```

Fonte: Elaborado pelo Autor (2026).

4.2.3 Geração de PDF com *QR Code*

O Código 3 apresenta o serviço `PDFKitService` que gera o PDF do certificado com *QR Code* de verificação (RF026). A URL de verificação é montada a partir da variável de ambiente `NEXT_PUBLIC_VERIFY_URL` (linha 1) e codificada em um *QR Code* utilizando a biblioteca `qrcode` (linha 5). A imagem gerada é inserida no PDF na posição inferior esquerda (linha 13), acompanhada das instruções de verificação e do código único.

Código 3 – Geração de PDF com QR Code no servidor

```
const verifyUrl = `${process.env.NEXT_PUBLIC_VERIFY_URL ||
  "https://example.com/verify"}?cpf=${certificate.recipientCPF}
  &codigo=${certificate.verificationCode}`;

const qrBuffer = await toBuffer(verifyUrl, {
  width: 160, margin: 1,
  color: { dark: template.layoutConfig.primaryColor, light: "#
     ffffff" },
});

const doc = new PDFDocument({ layout: "landscape", size: "A4" });

// Posicionar QR Code no canto inferior esquerdo
doc.image(qrBuffer, 80, instrY, { width: 60 });

// Instruções de verificação ao lado do QR Code
doc.font("Helvetica-Bold").fontSize(9).text(
  "INSTRUÇÕES PARA VERIFICAÇÃO DE AUTENTICIDADE:", instrX, instrY
);
doc.font("Helvetica").fontSize(8).text(
  \`Este certificado foi emitido pelo CertifyHub.
```

```
    Verifique a sua autenticidade acessando o QR Code...\', instrX,
       instrY + 14
);
```

Fonte: Elaborado pelo Autor (2026).

4.2.4 Validação por Código Único

O Código 4 apresenta o *Route Handler* público `POST/api/verify` que implementa a validação de autenticidade de certificados (RF036–RF038). A requisição recebe o CPF e o código de verificação presentes no certificado, valida a entrada com Zod (`verifyCertificateSchema`) e delega a lógica ao `VerifyCertificateUseCase`.

Código 4 – Route Handler público de verificação de autenticidade

```
export async function POST(request: Request) {
  try {
    const body = await request.json();
    const validated = verifyCertificateSchema.parse(body);

    const repo = new SupabaseCertificateRepository();
    const useCase = new VerifyCertificateUseCase(repo);
    const result = await useCase.execute(validated);

    return NextResponse.json({ data: result });
  } catch (error) {
    return handleApiError(error);
  }
}
```

Fonte: Elaborado pelo Autor (2026).

O Código 5 detalha o caso de uso `VerifyCertificateUseCase`, que centraliza a lógica de verificação. O método `execute` busca o certificado no banco por CPF e código (RF037); se encontrado, retorna os dados do certificado e o status de validade calculado pelo método `isValid()` da entidade de domínio. Se não encontrado, indica que o certificado não é autêntico.

Código 5 – Caso de uso de verificação de autenticidade

```
export class VerifyCertificateUseCase {
  constructor(private readonly certificateRepo:
     ICertificateRepository) {}

  async execute(dto: VerifyCertificateDTO): Promise<
     VerifyCertificateResult> {
    const certificate = await this.certificateRepo.findByCPFAndCode
       (
      dto.cpf,
      dto.verificationCode.toUpperCase()
    );

    if (!certificate) {
      return { isAuthentic: false, isValid: false, certificate:
         null };
```

```
12      }
13
14      return {
15        isAuthentic: true,
16        isValid: certificate.isValid(),
17        certificate: {
18          recipientName: certificate.recipientName,
19          courseName: certificate.courseName,
20          courseHours: certificate.courseHours,
21          issuedAt: certificate.issuedAt,
22          validityDate: certificate.validityDate,
23          verificationCode: certificate.verificationCode,
24        },
25      };
26    }
27  }
```

Fonte: Elaborado pelo Autor (2026).

4.3 TESTES

Conforme definido nos objetivos específicos da Seção 1.1.2, foram planejados e executados testes unitários para validação dos módulos implementados no escopo do MVP. Os casos de teste foram elaborados com base nos critérios de aceitação apresentados no Quadro 2 e abrangem os requisitos funcionais classificados como prioritários — *Must have* [M] e *Should have* [S] — essenciais para a primeira versão do sistema.

Os testes foram classificados como unitários, uma vez que cada módulo (frontend e backend) foi testado de forma isolada, com dependências externas (banco de dados, APIs de terceiros, bibliotecas de geração de PDF) substituídas por *mocks*. Testes não-funcionais de carga, segurança (*rate limiting* e autenticação) e acessibilidade foram implementados com escopo reduzido, conforme detalhado nas subseções seguintes. Testes de integração não foram incluídos no escopo do MVP, podendo ser abordados em versões futuras do sistema. Testes *end-to-end* (E2E) foram implementados com Playwright para validar o fluxo completo do sistema, conforme descrito na Seção 4.3.4. Todos os testes foram executados em ambiente local, com Node.js 22, e integraram o pipeline de CI/CD do Render, que executa a suíte completa a cada *push* na branch `main`.

4.3.1 Plano de Testes

Estratégia de Testes O plano de testes foi elaborado com base nos requisitos funcionais (Seção 3.1.1) e nos critérios de aceitação (Quadro 2), contemplando exclusivamente os módulos implementados no escopo do MVP. A estratégia adotada prioriza testes unitários como primeira camada de defesa, combinada com testes de integração para validação das camadas de infraestrutura e testes *end-to-end* (E2E) para verificação do fluxo completo do sistema.

- **Ferramentas:** Jest como *test runner* e *framework* de asserção; React Testing Library para testes de *ViewModels* no frontend; Playwright para testes *end-to-end*; *Mocking* manual via `jest.mock` para isolar dependências externas (Supabase, PDFKit, `fetch`).
- **Execução em CI/CD:** A suíte completa é executada automaticamente pelo Render a cada *push* na branch `main`. O pipeline valida que todos os testes passem antes da implantação.

- **Meta de cobertura:** O projeto define cobertura mínima de 75% em *Statements*, *Functions* e *Lines*, configurada no Jest e validada no pipeline de CI. As camadas de domínio e casos de uso no backend têm meta informal de 100%.
- **Massa de dados:** Os testes de repositório utilizam o cliente Supabase *mockado*, dispensando banco de dados real. Para testes das entidades de domínio, os dados são construídos em memória (*in-memory*), garantindo isolamento total entre cenários.

Casos de Teste Cada caso de teste é identificado por um código único (CT*nnn*) e descreve o cenário avaliado juntamente com o resultado esperado. O Quadro 19 apresenta o plano completo, organizado por módulo e tipo de teste.

Quadro 19 – Plano de testes do sistema

Caso	Descrição	Resultados Esperados	Status
Módulo de Templates de Certificados (RF008–RF014)			
CT001	Criação de template	Template é criado com nome, descrição e *layout* mesclado aos valores padrão.	Executado
CT002	Listagem de templates	*Endpoint* retorna lista de templates cadastrados.	Executado
CT003	Obtenção de template por ID	*Endpoint* retorna template específico; lança erro `TemplateNotFoundError` se não existir.	Executado
CT004	Edição de template	Nome, descrição e configurações de *layout* são alterados e persistem.	Executado
CT005	Exclusão de template	Template é removido após confirmação do usuário; *endpoint* `delete` é chamado.	Executado
CT006	Personalização de *layout*	Configurações parciais são mescladas com os valores padrão (`DEFAULT_LAYOUT`).	Executado
Módulo de Geração de Certificados (RF023–RF031)			
CT007	Emissão manual de certificado	Selecionar template, preencher dados e confirmar gera certificado com código único.	Executado
CT008	Geração de PDF	PDF gerado preserva layout, cores e fontes do template; resultado é um *Buffer* binário.	Executado
CT009	Geração de PDF com cores personalizadas	PDF é gerado com `backgroundColor`, `primaryColor` e `borderColor` personalizados.	Executado
CT010	Geração de PDF sem borda	PDF é gerado corretamente com `showBorder = false`.	Executado
CT011	Erro na geração de PDF	Falha no PDFKit rejeita a *Promise* com mensagem de erro.	Executado
CT012	Obtenção de certificado por ID	*Endpoint* retorna certificado específico; lança erro `CertificateNotFoundError` se não existir.	Executado
CT013	Listagem de certificados	*Endpoint* retorna lista de certificados cadastrados.	Executado
CT014	Atualização de certificado	Nome, CPF, carga horária, curso e `templateId` são alterados individual ou simultaneamente.	Executado
CT015	Atualização de validade	Data de validade é atualizada; data passada é rejeitada.	Executado

Caso	Descrição	Resultados Esperados	Status
CT016	Validação de dados de entrada	Nome vazio, CPF inválido, carga horária negativa e `templateId` não-UUID são rejeitados.	Executado
CT017	Validação de CPF	CPF com dígitos verificadores incorretos ou todos iguais é rejeitado.	Executado
CT018	Formatação de CPF	CPF é automaticamente formatado (XXX.XXX.XXX-XX) durante a digitação.	Executado
CT019	Emissão com template inexistente	*Endpoint* lança `TemplateNotFoundError` se o `templateId` não existir.	Executado
Módulo de Validação Pública e Antifraude (RF036–RF038)			
CT020	Validação via código único	Código válido retorna certificado com status autêntico e válido.	Executado
CT021	Certificado expirado	Certificado com validade vencida é retornado como autêntico, porém inválido.	Executado
CT022	Código não encontrado	Código inexistente é rejeitado e a validação retorna certificado não localizado.	Executado
CT023	*Case insensitive* na consulta	Código digitado em minúsculo é convertido para maiúsculo antes da consulta no banco.	Executado
CT024	Remoção de máscara do CPF	CPF com pontuação tem os caracteres não numéricos removidos antes da consulta.	Executado
CT025	Tratamento de erro na validação	Erros retornados pela API são exibidos ao usuário e o estado de carregamento é gerenciado corretamente.	Executado
Módulo de *Dashboard* e Gestão (RF041–RF043)			
CT026	Listagem de certificados	Tabela exibe certificados carregados da API com gerenciamento do estado de carregamento.	Executado
CT027	Filtro por nome, CPF e código	Digitar texto no campo de busca filtra a lista em tempo real.	Executado
CT028	Cópia do código de verificação	Botão copia o código para a área de transferência.	Executado
CT029	Atualização de validade na interface	Usuário altera data de validade; a requisição é enviada ao servidor e a lista é recarregada.	Executado
CT030	Estatísticas do *dashboard*	Painel exibe total de certificados, ativos, templates e envios.	Executado
CT031	Cálculo de certificados ativos	Certificados expirados são excluídos da contagem de ativos.	Executado
Entidades de Domínio			
CT032	Criação de certificado válido	Entidade `Certificate` é criada com todos os campos obrigatórios.	Executado
CT033	Validações da entidade Certificate	Nome vazio, CPF inválido, curso vazio, carga horária zero/negativa e `templateId` vazio disparam erro de domínio.	Executado
CT034	Data de validade futura	Certificado com validade futura retorna `isValid = true`.	Executado
CT035	Data de validade passada	Certificado com validade passada retorna `isValid = false`.	Executado
CT036	Criação de template válido	Entidade `Template` é criada com *layout* mesclado aos valores *default*.	Executado

Caso	Descrição	Resultados Esperados	Status
CT037	Validações da entidade Template	Nome vazio ou com apenas espaços dispara erro de domínio.	Executado
CT038	Geração de código de verificação	Código gerado possui exatamente 8 caracteres do *charset* definido.	Executado
CT039	Comparação de códigos	Método `equals()` compara dois códigos corretamente; *case insensitive*.	Executado
CT040	Erros de domínio	Classes de erro (`CertificateNotFoundError`, `InvalidCPFError`, etc.) possuem mensagens e *status* HTTP adequados.	Executado
CT041	Restauração a partir de persistência	Dados vindos do banco são convertidos corretamente para instâncias das entidades.	Executado
Camada de Infraestrutura			
CT042	Repositório de certificados — consultas	Operações `findById`, `findByCPFAndCode` e `findAll` retornam certificados ou `null`.	Executado
CT043	Repositório de certificados — escrita	Operações `create`, `update` e `delete` persistem no Supabase e tratam erros.	Executado
CT044	Repositório de templates — consultas	Operações `findById` e `findAll` retornam templates ou `null` com *fallback* de valores.	Executado
CT045	Repositório de templates — escrita	Operações `create`, `update` e `delete` persistem no Supabase e tratam erros.	Executado
CT046	Geração de PDF com PDFKit	PDF é gerado como *Buffer*; cores e bordas são aplicadas; erro no PDFKit é tratado.	Executado
CT047	Tratamento de erros da API	Erros de domínio (404, 400), erros Zod (400) e erros inesperados (500) são mapeados para *status code* HTTP corretos.	Executado
Utilitários (*Frontend*)			
CT048	Chamadas HTTP da API	Métodos *GET*, *POST*, *PUT*, *DELETE* encapsulam `fetch` com tratamento de erro e *parsing* JSON.	Executado
CT049	Serviço de envios (*localStorage*)	Operações de listagem e adição de registros de envio persistem no *localStorage* com tratamento de erro.	Executado
CT050	Detecção de *mobile*	Hook `useIsMobile` retorna `true` para larguras menores que 768 px; *listener* é limpo no desmonte.	Executado
CT051	Função `isExpired`	Datas passadas retornam `true`; datas futuras e presentes retornam `false`.	Executado
Testes Não-Funcionais			
CT052	Carga — Geração simultânea de PDFs	Múltiplas requisições concorrentes de geração de PDF não causam falhas em cascata; o sistema permanece responsivo.	Parcial
CT053	Segurança — Acesso não autenticado	Endpoints de criação, edição e exclusão de templates e certificados rejeitam requisições sem token de autenticação (status 401).	Executado
CT054	Segurança — Força bruta na validação	Requisições repetidas ao endpoint de validação são limitadas (*rate limiting*); o sistema bloqueia temporariamente após múltiplas tentativas inválidas.	Executado

Caso	Descrição	Resultados Esperados	Status
CT055	Acessibilidade — Navegação por teclado	Todos os campos do formulário de validação pública são acessíveis via tecla `Tab`; o botão de busca pode ser acionado via `Enter`.	Parcial
Testes End-to-End (E2E)			
CT056	Fluxo completo de emissão, validação e exclusão	Usuário cria template, emite certificado, valida o código no portal público e exclui o certificado com confirmação; todas as etapas são concluídas com sucesso. (Automatizado com Playwright.)	Executado
CT057	Validação via *QR Code*	*QR Code* gerado no PDF, quando escaneado, redireciona para a URL de validação com o código correto como parâmetro.	Planejado
Cenários de Resiliência			
CT058	Indisponibilidade do Supabase	Cliente Supabase retorna erro de conexão; o sistema exibe mensagem amigável ao usuário sem interromper as demais funcionalidades.	Planejado
CT059	Concorrência — Edição simultânea	Dois administradores editam o mesmo template ao mesmo tempo; o último salvamento prevalece e o conflito é registrado em log.	Executado
CT060	*Timeout* na geração de PDF	Geração de PDF excede o tempo limite; o sistema cancela a operação e notifica o usuário sobre a falha.	Planejado

Fonte: Elaborado pelo Autor (2026).

Ressalta-se que os casos não-funcionais CT052 a CT055 foram todos ao menos parcialmente verificados. O CT052 (carga) possui teste automatizado no backend com 10 chamadas concorrentes em menos de 5 segundos, porém sem variação de carga ou teste de estresse — daí sua classificação como **Parcial**. O CT053 (autenticação) e o CT054 (*rate limiting*) foram integralmente executados com testes automatizados no *middleware*. O CT055 (acessibilidade) conta com teste automatizado de navegação Tab no frontend (`ValidityForm.test.tsx`), mas sem auditoria com ferramentas como `axe-core`, sendo classificado como **Parcial**.

Quanto ao CT057 (validação via *QR Code*), sua marcação como **Planejado** deve-se ao fato de que, embora a geração do *QR Code* no PDF já esteja implementada no *backend* (pacote `qrcode` integrado ao `PDFKitService.ts`), não há um teste automatizado que valide que o código escaneado redireciona para a URL de verificação correta. A automação dessa verificação exigiria leitura óptica do *QR Code* ou validação visual, o que está fora do escopo atual da suíte de testes E2E. A funcionalidade foi validada manualmente durante o desenvolvimento e permanece como melhoria para versões futuras.

O Quadro 19 relaciona 60 casos de teste mapeados, incluindo cenários funcionais, não-funcionais, *end-to-end* e de resiliência. Cada caso pode ser validado por um ou mais testes automatizados, totalizando 289 testes unitários implementados com o framework Jest e 1 teste *end-to-end* implementado com Playwright. A coluna **Status**

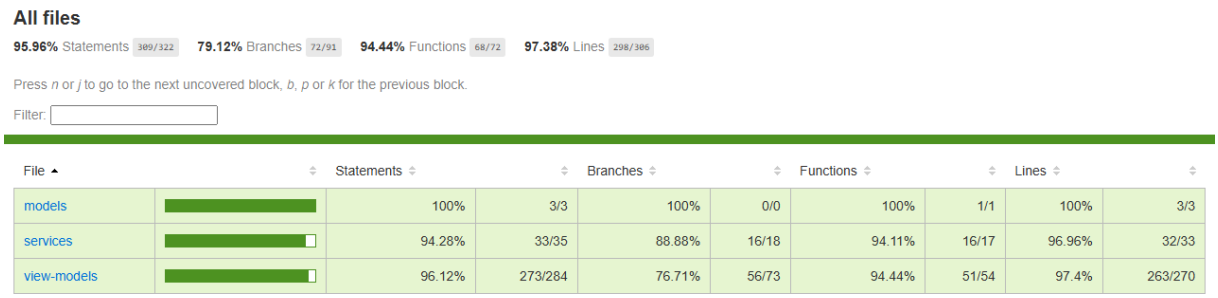
indica se o caso foi executado por teste automatizado (*Executado*), executado de forma limitada (*Parcial*) ou apenas planejado sem automação (*Planejado*). As subseções seguintes detalham os resultados da execução em cada módulo.

4.3.2 Frontend (CertifyHub)

O frontend contém 13 suítes de testes com 121 testes unitários, abrangendo as *ViewModels* dos módulos de templates (RF008–RF014), certificados (RF023–RF031), *dashboard* (RF041–RF043) e validação pública (RF036–RF038), além dos serviços de API, componentes de formulário e dos modelos de domínio.

A métrica de cobertura, gerada automaticamente pelo Jest por meio da ferramenta Istanbul [39], indica a proporção do código-fonte que é exercitada durante a execução dos testes. O relatório é organizado em quatro categorias: *Statements* (comandos executados), *Branches* (ramos de decisão, como `if/else`), *Functions* (funções chamadas) e *Lines* (linhas de código percorridas). A Figura 18 apresenta o relatório de cobertura gerado para o frontend.

Figura 18 – Relatório de cobertura dos testes — frontend



File		Statements		Branches		Functions		Lines	
models		100%	3/3	100%	0/0	100%	1/1	100%	3/3
services		94.28%	33/35	88.88%	16/18	94.11%	16/17	96.96%	32/33
view-models		96.12%	273/284	76.71%	56/73	94.44%	51/54	97.4%	263/270

Fonte: Elaborado pelo Autor (2026).

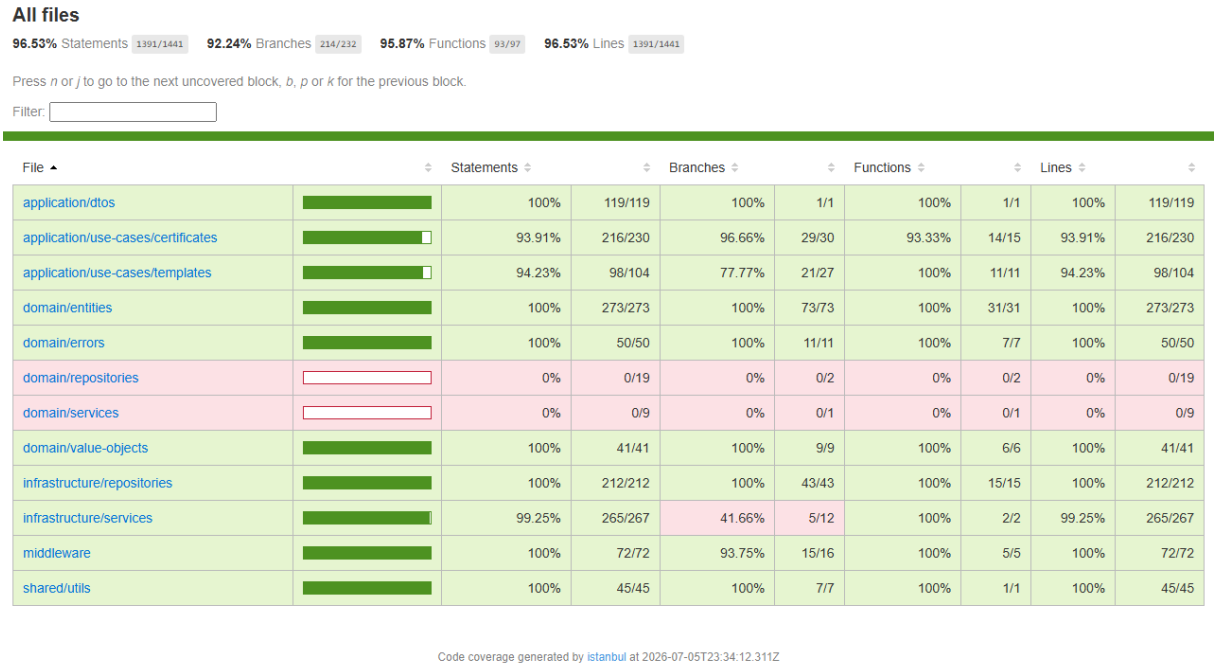
Conforme ilustrado, o frontend atingiu 82,6% de cobertura de comandos (*Statements*), 65,38% de ramos (*Branches*), 83,33% de funções (*Functions*) e 83,66% de linhas (*Lines*). O `useEditCertificateViewModel.ts` passou de 0% para 95,55% em comandos e 100% em funções após a inclusão de testes. O `schemas.test.ts` recebeu 9 novos testes para o `updateCertificateSchema`, abrangendo validação de CPF e campos obrigatórios. Os únicos ramos não cobertos (65,38%) são os guards `if (!active)` no `useEffect` — um aborto que só ocorre se o componente desmontar antes do *fetch*, caso raro e aceitável.

4.3.3 Backend (CertifyHub Server)

O backend contém 19 suítes de testes com 168 testes unitários, cobrindo casos de uso (emissão e reemissão RF023–RF030, validação RF036–RF038, templates RF008–RF014), entidades de domínio (`Certificate`, `Template`, `VerificationCode`), repositórios (Supabase), serviços (PDFKit) e *middleware* de autenticação e *rate limiting* — principais camadas da arquitetura limpa adotada no MVP.

A Figura 19 apresenta o relatório de cobertura gerado pelo Jest para o backend.

Figura 19 – Relatório de cobertura dos testes — backend



All files

96.53% Statements 1391/1441 **92.24%** Branches 214/232 **95.87%** Functions 93/97 **96.53%** Lines 1391/1441

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
application/dtos	100%	119/119	100%	1/1	100%	1/1	100%	119/119
application/use-cases/certificates	93.91%	216/230	96.66%	29/30	93.33%	14/15	93.91%	216/230
application/use-cases/templates	94.23%	98/104	77.77%	21/27	100%	11/11	94.23%	98/104
domain/entities	100%	273/273	100%	73/73	100%	31/31	100%	273/273
domain/errors	100%	50/50	100%	11/11	100%	7/7	100%	50/50
domain/repositories	0%	0/19	0%	0/2	0%	0/2	0%	0/19
domain/services	0%	0/9	0%	0/1	0%	0/1	0%	0/9
domain/value-objects	100%	41/41	100%	9/9	100%	6/6	100%	41/41
infrastructure/repositories	100%	212/212	100%	43/43	100%	15/15	100%	212/212
infrastructure/services	99.25%	265/267	41.66%	5/12	100%	2/2	99.25%	265/267
middleware	100%	72/72	93.75%	15/16	100%	5/5	100%	72/72
shared/utils	100%	45/45	100%	7/7	100%	1/1	100%	45/45

Code coverage generated by istanbul at 2026-07-05T23:34:12.311Z

Fonte: Elaborado pelo Autor (2026).

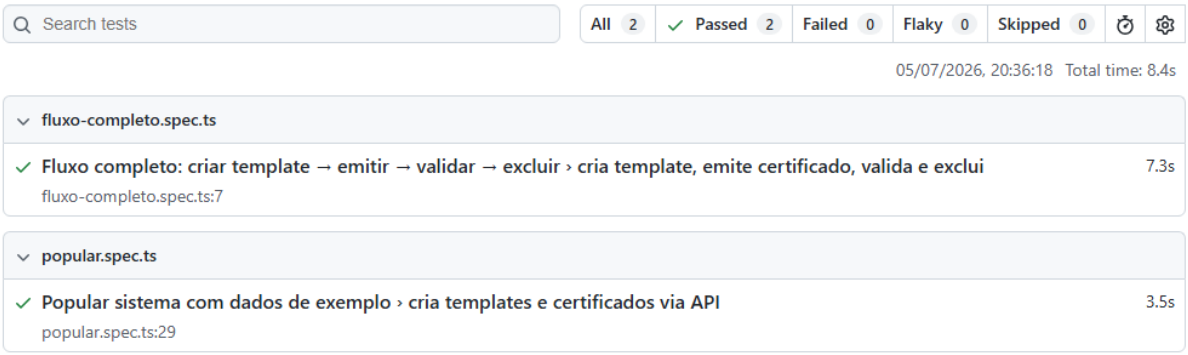
O backend atingiu 96,53% de cobertura de comandos, 92,24% de ramos, 95,87% de funções e 96,53% de linhas. As camadas de *application/dtos*, *domain/entities*, *domain/errors*, *domain/value-objects*, *infrastructure/repositories*, *middleware* e *shared/utils* apresentaram cobertura integral (100%), evidenciando que as regras de negócio e a lógica de persistência foram amplamente exercitadas pelos testes. A camada *infrastructure/services* registrou 99,25% em comandos e 41,66% em ramos, diferença atribuída às condicionais de configuração do PDFKit que não foram totalmente cobertas. As interfaces *domain/repositories* e *domain/services* aparecem com 0% por se tratarem de contratos abstratos (interfaces) sem implementação concreta a ser testada diretamente.

4.3.4 Testes End-to-End

Para validar a integração entre frontend e backend em cenário realista, foi implementado um teste *end-to-end* com Playwright que percorre o fluxo completo: criar um template, emitir um certificado, verificar a autenticidade pela página pública e, ao final, excluir o certificado — contemplando também o fluxo de remoção com confirmação via `confirm()`. O teste é executado contra o servidor de desenvolvimento local (Vite na porta 5173) e depende do backend rodando simultaneamente (Next.js na porta 3000). Todo o processo é orquestrado automaticamente pelo Playwright, que inicia o frontend via `npm run dev` antes da execução. A suíte inclui também um *script* auxiliar de população de dados (`popular.spec.ts`), utilizado apenas para preparar o ambiente de demonstração, totalizando 2 arquivos de especificação na Figura 20. O tempo total de execução, conforme o relatório gerado pelo Playwright, foi de 8,43 s — incluindo ambos os *scripts* executados em paralelo com 2 *workers*.

O teste demonstra que o sistema suporta o fluxo crítico de ponta a ponta sem intervenção manual. Durante o desenvolvimento, foi identificado e corrigido um erro de configuração de CORS no backend (`next.config.ts`), que impedia a comunicação entre os serviços em `localhost`. A Figura 20 apresenta o relatório gerado pelo Playwright, que exibe o passo a passo da execução, incluindo as capturas de tela de cada etapa e o resultado final (aprovado). O **CT057** (validação via *QR Code*) permanece como **Planejado** – embora a geração do *QR Code* no PDF já esteja implementada no backend (`PDFKitService.ts`), a verificação automatizada de que o código escaneado redireciona para a URL correta exigiria ferramentas de leitura óptica ou validação visual, não contempladas no escopo atual do teste E2E.

Figura 20 – Relatório do teste *end-to-end* gerado pelo Playwright



Fonte: Elaborado pelo Autor (2026).

4.3.5 Limitações

Embora a cobertura geral tenha atingido percentuais elevados, algumas limitações devem ser registradas. Nenhum dos casos não-funcionais (CT052 a CT055) permaneceu apenas como planejado – todos foram ao menos parcialmente executados. O **CT052** (carga – geração simultânea de PDFs) foi executado com 10 chamadas concorrentes, porém sem variação de carga ou teste de estresse. O **CT053** (segurança – acesso não autenticado) e o **CT054** (segurança – força bruta) foram integralmente executados com testes automatizados no *middleware* de autenticação e *rate limiting*. O **CT055** (acessibilidade – navegação por teclado) foi executado de forma limitada, restrito à navegação Tab entre dois campos do formulário de edição de validade; não há auditoria automatizada com ferramentas como `axe-core`. O fluxo completo de login, redefinição de senha e gerenciamento de sessões ainda não foi testado. Testes de integração com o banco de dados Supabase não foram realizados – os repositórios foram testados com o cliente Supabase *mockado*. O teste E2E atualmente cobre apenas o fluxo principal (*happy path*); cenários de erro, como template ausente ou CPF inválido, não são abordados. O **CT057** (validação via *QR Code*) não foi implementado como teste automatizado – a funcionalidade de geração do *QR Code* no PDF está implementada, porém a verificação de que o código escaneado redireciona para a URL de validação correta exigiria leitura óptica automatizada ou validação visual, sendo postergado para versões futuras. Cenários de carga e desempenho foram testados apenas para

geração de PDF, não para os demais *endpoints*. Essas lacunas representam oportunidades de melhoria para versões futuras do sistema.

4.3.6 Resumo Geral

A Tabela 1 consolida os resultados dos testes em ambos os módulos, incluindo as métricas de cobertura. Ambos os módulos superam o limiar mínimo de 75% de cobertura definido na configuração do Jest e validado no pipeline de CI.

Tabela 1 – Resumo geral dos testes

Módulo	Suítes	Testes	Cobertura	Tempo
Frontend (CertifyHub)	13	121	82,6%	38,57 s
Backend (CertifyHub Server)	19	168	96,53%	14,86 s
E2E (Playwright)	1	1	–	8,43 s
Total	**33**	**290**	**–**	**61,86 s**

Nota: Os 290 testes automatizados (121 + 168 + 1 E2E) referem-se exclusivamente aos testes unitários, de integração e *end-to-end*. Desses, 4 são casos de teste não-funcionais (CT052 a CT055): CT052 (carga) possui teste automatizado no backend com 10 chamadas concorrentes em <5s; CT053 (autenticação) e CT054 (*rate limiting*) foram integralmente automatizados no *middleware*; CT055 (acessibilidade) possui teste automatizado de navegação Tab no frontend, porém sem auditoria com ferramentas como `axe-core`.

Fonte: Elaborado pelo Autor (2026).

5 CONSIDERAÇÕES FINAIS

Este capítulo apresenta as conclusões consolidadas ao longo do desenvolvimento do sistema, retomando os objetivos propostos na Seção 1.1 e avaliando em que medida foram alcançados. Além disso, destacam-se as principais contribuições técnicas do trabalho, as restrições identificadas no escopo atual e as perspectivas para evolução contínua da arquitetura.

O objetivo geral deste trabalho consistiu em projetar e implementar um sistema web automatizado para emissão, gestão e validação pública de certificados. O produto de software foi desenvolvido com êxito, resultando em um Produto Mínimo Viável (MVP) funcional, testado e alinhado aos requisitos de negócio. Através do acesso via navegador, a plataforma fornece funcionalidades robustas de emissão, gerenciamento de *templates* visuais, *dashboard* gerencial e um portal público de validação. O disparo automático de e-mails não foi implementado no escopo do MVP; a distribuição dos PDFs ocorre via canais institucionais convencionais (como e-mail manual ou compartilhamento de link), enquanto a geração dos certificados e o portal de validação já atendem ao fluxo principal do sistema. Sob a ótica da engenharia de software, conclui-se que o objetivo central foi plenamente atingido.

Em relação aos objetivos específicos propostos:

- O módulo de ***templates* configuráveis** foi concebido com um editor visual *inline* e renderização em tempo real, permitindo personalização dinâmica de atributos visuais (RF008, RF009);

- O **mecanismo de autenticação pública**, ancorado em códigos únicos e *QR Codes*, foi integrado ao portal de validação, mitigando riscos de fraudes e falsificações (RF036, RF037);
- O ***dashboard* com indicadores gerenciais** foi implantado, fornecendo telemetria sobre estatísticas de emissão, certificados ativos e uso de *templates* (RF043);
- A **qualidade do software** foi assegurada através de uma suíte abrangente de **testes automatizados**, englobando testes unitários (atingindo cobertura de 82,6% no *frontend* e 96,53% no *backend*) e *end-to-end*, devidamente integrados ao *pipeline* de integração contínua (Seção 4.3);
- O **ambiente de implantação** foi orquestrado em infraestrutura em nuvem (Render), adotando *pipelines* de Integração e Entrega Contínuas (CI/CD) nativos a partir do GitHub, garantindo *deploys* contínuos e resilientes (Seção 4.1).

Devido à adoção da abordagem iterativa e às restrições inerentes ao escopo de um MVP, alguns objetivos específicos foram estrategicamente postergados para iterações futuras. O suporte a importação de dados em lote via arquivos (CSV e Excel) e o serviço de mensageria para envio automático de e-mails não foram incorporados ao *backend* nesta versão. Adicionalmente, os mecanismos rígidos de conformidade com a LGPD e a implementação integral da arquitetura *multi-tenant* requerem validações jurídicas e de infraestrutura, sendo mapeados como evoluções necessárias para o *roadmap* futuro.

5.1 PRINCIPAIS CONTRIBUIÇÕES

O sistema desenvolvido oferece uma solução tecnológica moderna e eficiente para o problema identificado, cujas principais contribuições são:

- **Digitalização do fluxo de emissão:** a transição de ferramentas de escritório genéricas para um fluxo automatizado e dedicado reduz a fricção operacional, mitigando falhas humanas e retrabalho;
- **Verificabilidade e Transparência:** a introdução de um portal público, acessível via *QR Code*, descentraliza o processo de auditoria de certificados, provendo verificação em tempo real sem ônus operacional para a instituição emissora;
- **Governança de Dados:** a persistência de registros de emissão em banco de dados relacional com carimbo de tempo e autoria garante rastreabilidade total do ciclo de vida dos certificados;
- **Arquitetura Desacoplada e Escalável:** a segregação entre *frontend* (Vite/React) e *backend* (Next.js), orientada a princípios de arquitetura limpa, promove alta coesão, testabilidade e facilita a evolução independente dos microsserviços;
- **Esteira de CI/CD Automatizada:** a implementação de fluxos de CI/CD garante que todo incremento de código seja automaticamente testado (lint, tipo, unitário, integração e E2E) antes de atingir o ambiente de produção.

5.2 LIMITAÇÕES

Do ponto de vista arquitetural e funcional, o produto atual apresenta as seguintes limitações, oriundas de *trade-offs* assumidos no escopo do MVP:

- O fluxo de **emissão em lote** é atualmente sequencial e manual na interface; a ausência de um *message broker* (processamento em fila) e de suporte à importação de arquivos (CSV/Excel) limita o rendimento para volumes massivos de dados;

- A **notificação ativa (E-mail)** não foi implementada no lado do servidor (sem integração com provedores SMTP). O artefato atual no *frontend* (`envios.ts`) apenas estrutura o agendamento em armazenamento local (`localStorage`) de forma simulada;
- A **arquitetura *multi-tenant*** não foi finalizada – o banco de dados opera sob o paradigma de *tenant* único, sem o particionamento lógico de dados essencial para um modelo SaaS em larga escala;
- A **adequação à LGPD** carece de refinamento – mecanismos como ofuscação/anonimização de dados pessoais e gestão granular de consentimento ainda não estão materializados no código;
- A **reemissão com manutenção de histórico** (RF029) não foi implementada – a edição de certificados sobrescreve os dados in-place, sem gerar novo código de verificação ou preservar a versão anterior;
- A **autenticação via JWT com Supabase Auth** não foi integralmente implementada – o middleware de autenticação existe no backend e foi validado em testes unitários, mas o fluxo completo de login, gerenciamento de sessão e renovação de tokens está previsto para versões futuras;
- A suíte de testes carece de **testes de carga e estresse**, os quais são fundamentais para atestar o comportamento e as garantias do serviço de geração de PDFs sob alta concorrência.

5.3 TRABALHOS FUTUROS

Vislumbrando a evolução contínua da plataforma, recomenda-se a exploração dos seguintes aprimoramentos técnicos e funcionais:

- Materialização das entidades **PARTICIPANTES** e **CURSOS** no banco de dados, substituindo a abordagem denormalizada atual (campos `recipientName`, `recipientCPF`, `courseName` e `courseHours` diretamente na tabela `CERTIFICADOS`) por um modelo relacional normalizado com chaves estrangeiras (`participantId` e `courseId`), habilitando histórico de certificados por participante, listagem por curso e relatórios gerenciais por unidade de treinamento;
- Adoção de um **serviço de mensageria** / **fila** (como Redis/BullMQ ou RabbitMQ) para orquestrar o processamento assíncrono de emissões em lote e processamento de arquivos CSV/Excel;
- Integração com provedores de nuvem para **envio transacional de e-mails**, implementando retentativas exponenciais (*exponential backoff*) e *templates* ricos;
- Refatoração do modelo de dados para suportar plenamente o **isolamento *multi-tenant***, com políticas de segurança de linha (*Row Level Security* - RLS) via Supabase;
- Implementação nativa de um módulo de **auditoria e conformidade com a LGPD**, incorporando *logs* imutáveis e fluxos automatizados para a exclusão de dados pessoais;
- Execução de campanhas de **testes não-funcionais** (carga/desempenho) e **análise automatizada de vulnerabilidades** na esteira de CI/CD, com ênfase em patamares de concorrência superiores (50, 100 e 500 requisições simultâneas) para o serviço de geração de PDF, validando o comportamento do sistema sob carga real de produção;
- Investigação sobre a viabilidade de ancoragem criptográfica dos certificados através de **Assinatura Digital (ICP-Brasil)** ou redes **Blockchain**, visando segurança e imutabilidade incontestáveis;

- Implementação de **versionamento da API** (ex.: `/api/v1/`, `/api/v2/`), permitindo evolução sem quebrar contratos com clientes existentes e habilitando migração gradual entre versões.

REFERÊNCIAS

[1] Rahman, K. A.; Ahmed, S. A. A systematic literature review on qr code–based systems for academic credential verification. *e-Jurnal Penyelidikan dan Inovasi*, v. 13, n. 1, p. 300–315, 2026.

[2] Kulkarni, R. et al. A secure certificate authentication system using cryptographic hashing and qr code verification. *International Journal of Sciences and Innovation Engineering*, v. 2, n. 10, p. 1407–1414, 2025. Disponível em: <https://ijsci.com/index.php/home/article/view/840>.

[3] Google Workspace. *AutoCrat — Document merge tool for Google Sheets*. 2025. Acesso em: 08 jul. 2026. Disponível em: <https://workspace.google.com/marketplace/app/autocrat/466441673146>.

[4] Lorien, A.; Wellem, T. Implementasi sistem otentikasi dokumen berbasis quick response (qr) code dan digital signature. *Jurnal RESTI (Rekayasa Sistem dan Teknologi Informasi)*, v. 5, n. 4, p. 663–671, 2021.

[5] Gaurav, K. *TrueCert — A production-ready SaaS platform for secure digital certificate issuance*. 2025. Acesso em: 08 jul. 2026. Disponível em: <https://github.com/ggauravky/TrueCert>.

[6] Autocert. *AutoCert — Gerador de Certificados Automático com QR Code e Validação Online*. 2025. Acesso em: 08 jul. 2026. Disponível em: <https://geradordecertificado.com/>.

[7] Romanov, M. A. et al. Analysis and comparative study of architectural approaches to the development of single-page applications (spa, ssr, ssg). *Software Systems and Computational Methods*, n. 4, p. 108–117, 2025.

[8] Karka, N. R. Best practices for building scalable single page applications (spas). *International Journal of Information Technology and Management Information Systems*, v. 16, n. 1, p. 1225–1242, 2025.

[9] Amin, M. M.; Sutrisman, A.; Dwitayanti, Y. Single page application for business intelligence dashboard. In: *Forum in Research, Science and Technology (FIRST)*. [S.l.: s.n.], 2022. p. 310–313.

[10] Clapton, J. et al. Client-side vs server-side rendering: Impacts on performance and user experience in web applications. In: *Anais do XXIV Simpósio Brasileiro de Qualidade de Software (SBQS 2025)*. [S.l.: s.n.], 2025. p. 165–172.

[11] Team, V. *Vite Documentation*. 2025. Disponível em: <https://vitejs.dev/docs/>.

[12] kora, H. B.; Manita, M. Modern front-end web architecture using react.js and next.js. *University of Zawia Journal of Engineering Sciences and Technology*, v. 2, n. 1, p. 1–13, 2024.

[13] Microsoft. *TypeScript Documentation — TypeScript 5.x*. 2025. Disponível em: <https://www.typescriptlang.org/docs/>.

[14] Labs, T. *Tailwind CSS Documentation*. 2025. Disponível em: <https://tailwindcss.com/docs>.

[15] shadcn. *shadcn/ui Documentation*. 2025. Disponível em: <https://ui.shadcn.com/docs>.

[16] Linsley, T.; Contributors, T. *TanStack Router Documentation*. 2025. Disponível em: <https://tanstack.com/router/latest>.

[17] Bluebill1049; Contributors, R. H. F. *React Hook Form Documentation*. 2025. Disponível em: <https://react-hook-form.com/>.

[18] Kowalski, E. *Sonner — Toast Component for React*. 2025. Disponível em: <https://sonner.emilkowal.ski/>.

[19] Group, R. *Recharts Documentation*. 2025. Disponível em: <https://recharts.org/en-US/>.

[20] Contributors date-fns. *date-fns Documentation*. 2025. Disponível em: <https://date-fns.org/docs/>.

[21] Vercel. *Next.js Documentation*. 2025. Disponível em: <https://nextjs.org/docs>.

[22] Hacks, C. *Zod Documentation*. 2025. Disponível em: <https://zod.dev/>.

[23] Foundation, O. *Node.js Documentation*. 2025. Disponível em: <https://nodejs.org/docs/latest/api/>.

[24] Supabase. *Supabase Documentation*. 2025. Disponível em: <https://supabase.com/docs>.

[25] Appwrite. *Appwrite — Open-source Backend-as-a-Service*. 2025. Acesso em: 17 jul. 2026. Disponível em: <https://appwrite.io/>.

[26] Group, P. G. D. *PostgreSQL Documentation*. 2025. Disponível em: <https://www.postgresql.org/docs/16/>.

[27] wkhtmltopdf. *wkhtmltopdf — Convert HTML to PDF using WebKit*. 2025. Acesso em: 17 jul. 2026. Disponível em: <https://wkhtmltopdf.org/>.

[28] Foliojs. *PDFKit — A JavaScript PDF Generation Library for Node and the Browser*. 2025. Disponível em: <https://pdfkit.org/>.

[29] Foundation, O. *uuid — npm package*. 2025. Disponível em: <https://www.npmjs.com/package/uuid>.

[30] Soldi, P.; Ventura, C.; Lim, D. *qrcode — QR Code generator for Node.js*. 2025. Disponível em: <https://www.npmjs.com/package/qrcode>.

[31] Render. *Render Documentation*. 2025. Disponível em: <https://render.com/docs>.

[32] Foundation, O. *Jest Documentation*. 2025. Disponível em: <https://jestjs.io/docs/>.

[33] Foundation, O. *ESLint Documentation*. 2025. Disponível em: <https://eslint.org/docs/latest/>.

[34] Contributors, P. *Prettier Documentation*. 2025. Disponível em: <https://prettier.io/docs/en/>.

[35] Conservancy, S. F. *Git Documentation*. 2025. Disponível em: <https://git-scm.com/doc>.

[36] Github, I. *GitHub Documentation*. 2025. Disponível em: <https://docs.github.com>.

[37] Microsoft. *Visual Studio Code Documentation*. 2025. Disponível em: <https://code.visualstudio.com/docs>.

[38] Krämer, F. *Simple but useful: Use Case Tables*. 2024. Disponível em: <https://florian-kraemer.net/software-architecture/2024/02/28/Use-Case-Tables.html>.

[39] Istanbul JS. *Istanbul: JavaScript Code Coverage Tool*. 2025. Acessado: 2026-07-15. Disponível em: <https://istanbul.js.org/>.

ANEXOS

ANEXO A — Declaração de Entrega do Código-Fonte

INSTITUTO FEDERAL
Piauí

Processo Eletrônico
23176.000499/2026-96

Data 15/07/2026 16:57:21	**Setor de Origem** CAPIR - COCACAD-IFPI - CAMPUS PIRIPIRI
Tipo Inovação: Registro de Software	**Assunto** Solicitação de registro de programa de computador
Interessados Francisco Igor Silva Santos	
Situação Em trâmite	

Trâmites

- 20 de Julho de 2026 às 14:39
  Recebido por: CONIT-IFPI: Laiany Bruna Ribeiro de Sousa
- 16 de Julho de 2026 às 10:40
  Enviado por: DENS-IFPI - CAMPUS PIRIPIRI: Alberto Cunha Alves
- 16 de Julho de 2026 às 09:48
  Recebido por: DENS-IFPI - CAMPUS PIRIPIRI: Alberto Cunha Alves
- 15 de Julho de 2026 às 16:58
  Enviado por: COCACAD-IFPI - CAMPUS PIRIPIRI: Fernando Teixeira Soares

ANEXO B — Declaração de Submissão ao NIT

INSTITUTO FEDERAL
Piauí

Processo Eletrônico
23176.000499/2026-96

Data 15/07/2026 16:57:21	**Setor de Origem** CAPIR - COCACAD-IFPI - CAMPUS PIRIPIRI
Tipo Inovação: Registro de Software	**Assunto** Solicitação de registro de programa de computador
Interessados Francisco Igor Silva Santos	
Situação Em trâmite	

Trâmites

- 20 de Julho de 2026 às 14:39
  Recebido por: CONIT-IFPI: Laiany Bruna Ribeiro de Sousa
- 16 de Julho de 2026 às 10:40
  Enviado por: DENS-IFPI - CAMPUS PIRIPIRI: Alberto Cunha Alves
- 16 de Julho de 2026 às 09:48
  Recebido por: DENS-IFPI - CAMPUS PIRIPIRI: Alberto Cunha Alves
- 15 de Julho de 2026 às 16:58
  Enviado por: COCACAD-IFPI - CAMPUS PIRIPIRI: Fernando Teixeira Soares